Rust for RTL Verification

Part I: Rust for the Python-Fluent (Chapters 1-14, draft)

Ray Salemi

Contents

Chapter 1: Why Rust?

Rust for RTL Verification is a book for verification engineers who have outgrown an interpreter and for Rust programmers who want to learn the Universal Verification Methodology (UVM). Mostly, though, it is a book for readers of *Python for RTL Verification* who are ready for a second language — one that trades a little of Python's ease for a lot of speed and an entirely new superpower: a compiler that finds testbench bugs before the simulator ever runs.

This book teaches you Rust the way the last book taught you Python: just enough of the language, arriving just in time, to build testbenches with rustvm-sim (our cocotb equivalent) and rustvm (our pyuvm equivalent). By the final chapter you will have rebuilt the TinyALU testbench — the same TinyALU, the same testbench architecture, versions 1.0 through 8.0 — in a language that compiles to native code and races through regressions.

The book assumes you know the Python story

Python for RTL Verification assumed you knew how to program. This book assumes more: that you know how to program *testbenches*, the way that book taught them. When we meet a sequence in Chapter 36, I will not explain what a sequence is for — you know. I will explain what it looks like in Rust and why it looks that way. If you have not read the Python book but know cocotb and pyuvm well, you will be fine. If neither is true, read that book first; this one will still be here.

A note on what you do *not* need: any Rust. Not one line. If you have heard alarming rumors about a thing called the borrow checker, you have heard correctly, and we will make friends with it in Chapter 5.

Two revolutions, thirty years apart

The Python book told the story of how we came to verify hardware with software — from e and SUPERLOG through the methodology wars to the UVM, and then sideways, off the simulator entirely, to Python. That story had a moral: testbenches are software, and software deserves a software language.

Rust's story rhymes with it. In 2006, a Mozilla engineer named Graydon Hoare started a personal project to answer an uncomfortable question: why, decades into the software era, were our foundational programs — browsers, kernels, the code we bet everything on — still written in languages that let one stray pointer corrupt everything? C and C++ were fast because they trusted the programmer completely, and every security bulletin showed what that trust cost.[2]

The conventional answer was garbage collection: let a runtime babysit memory, and accept the slowdown. That is Python's answer, and for testbenches it is a fine one — until it isn't. Rust proposed something genuinely new: what if the *compiler* proved memory safety, at compile time, and the finished program paid nothing at all? No garbage collector, no interpreter, no runtime babysitter. The rules that make this possible — ownership and borrowing — are the subject of Chapter 5, and they will bend your brain exactly once, after which you will wonder how you ever tracked object lifetimes in your head.

Rust 1.0 shipped in 2015. Since then the language has spent year after year at the top of developer-survey "most admired" lists, and it has done something no other language managed in half a century: it convinced the Linux kernel, Windows, and Android teams to admit a second systems language into their codebases. That is not fashion. That is an industry deciding that compile-time correctness is worth learning something hard.

[2] The security community eventually put numbers on it: both Microsoft and Google's Chrome team reported that roughly 70% of their serious security bugs were memory-safety bugs — the exact category Rust eliminates at compile time.

Why Rust for verification?

It is fair to ask why a verification engineer with a working Python flow should care. I will give you the honest engineering answer in four parts.

Speed. Python is an interpreted language, and for all of cocotb's cleverness, every signal read, every transaction compare, every scoreboard update runs through the interpreter. For the TinyALU it does not matter. For a regression farm running thousands of seeds against a large SoC, testbench overhead is real money and real schedule. Rust compiles to the same kind of native code as the simulator itself. When the testbench stops being the bottleneck, you buy back regression time without touching your license count.

Correctness before simulation. This is the deeper reason, and the one this book keeps returning to. In Python, a typo'd signal name, a transaction handed to the wrong port, a scoreboard mutated by two tasks at once — all of these are discovered *at runtime*, which in our world means *after the simulator license is checked out, the design is elaborated, and forty minutes have passed*. Rust moves an astonishing fraction of these discoveries to the compile step, which takes seconds and costs nothing. A TLM connection mistake that pyuvm reports as a runtime `UVMTLMConnectionError` simply does not compile in rustvm. The Python book warned you about a race between two tasks sharing `transaction_data` and taught you to avoid it by discipline. The Rust compiler *rejects that code*. Discipline is good; proof is better.

Testbench refactoring without fear. Verification code lives longer than we admit and gets modified by more hands than we would like. In Python, renaming a field means grepping and praying; the interpreter will tell you what you missed, one AttributeError at a time, over the following month. In Rust, the compiler produces the complete list of every place that must change, and the testbench does not build until you have addressed all of them. This changes how boldly you can improve old testbenches.

One binary, no environment. A Rust testbench compiles to a single library that the simulator loads. There is no interpreter version to match, no virtual environment to activate, no `pip install` on the farm machines. If it built, it runs.

And one more, which deserves its own paragraph: **unit testing without a simulator.** Because Rust testbench components are ordinary structs, the pure-software parts of your testbench — predictors, transaction operations, coverage logic — can be tested with `cargo test` in milliseconds, on your laptop, with no simulator license anywhere in sight. The Python stack could do some of this; the Rust toolchain makes it so frictionless that this book does it constantly, starting in Chapter 14.

What it costs

I owe you the other side of the ledger, because there is one.

Rust is harder to learn than Python. Not a little harder — the ownership system is a genuinely new idea, and for your first weeks the compiler will reject code you are certain is fine. (It is almost never fine. This is the maddening part. Later it becomes the endearing part.) The Python book could teach its whole language in fourteen short chapters because Python works hard to be unsurprising; Rust holds opinions, and it holds them at compile time.

The edit-run loop is slower. Python starts instantly; Rust compiles first. For testbench work the compile is usually seconds, not minutes, and it replaces the forty-minute runtime failure — but the rhythm is different and you will feel it.

The verification ecosystem is younger. Python has cocotb, pyuvm, and years of conference papers; Rust verification is early. That is part of why this book exists — someone gets to write the early chapters of that story, and it may as well be us.

There is no REPL. Python let you poke at ideas interactively; Rust asks you to write a small program. The playground projects in Part I are this book's substitute, and honestly, `cargo` makes them cheap enough that you will not miss the prompt much.

If your testbenches are small, your regressions short, and your team fluent in Python, Python remains a fine answer, and I will not pretend otherwise. This book is for when one of those three stops being true.

Code examples

The conventions are the ones you know. Every example has a figure number; code is followed by `--` and then its output. Examples live in the `rust4uvm_examples` repository, in directories named after their chapters, with instructions in each `README.md`. Early chapters use standalone cargo projects the way the Python book used Jupyter notebooks; from Chapter 17 on, examples are simulation directories.

We should not break a two-book tradition. In figure 1, we create a program with `cargo new`, Rust's project generator — meet `cargo` now, because like `pip`, `venv`, `make`, and `pytest` fused into one tool, it will be everywhere.

```
# Figure 1: Creating our first program

% cargo new hello
    Creating binary (application) `hello` package
```

`cargo new` writes a tiny project containing `src/main.rs`, which is where figure 2 lives. Where Python let us type `print("Hello, world.")` naked at a prompt, Rust asks for a function — `fn main()` is where every Rust program begins. The exclamation point on `println!` marks it as a *macro* rather than a function, a distinction that will matter a great deal in Chapter 21 and not at all before then.

```
# Figure 2: The classic first program

fn main() {
    println!("Hello, world.");
}
```

```
% cargo run
   Compiling hello v0.1.0
    Finished `dev` profile
     Running `target/debug/hello`
Hello, world.
--
Hello, world.
```

Notice what happened between `cargo run` and the greeting: a compile. Nothing about our two-line program was checked until we asked to run it — but *everything* about it was checked before it ran. Hold on to that trade; it is the whole book in miniature.

The plan

Part I (Chapters 1–14) teaches the Rust you need, always against the Python you know: ownership where Python had garbage collection, traits where Python had inheritance, `Result` where Python had exceptions, `match` where Python had `if` chains and envy. Part II (Chapters 15–20) reaches the simulator: async/await — whose engine, you will be pleased to hear, works on the same principles as the cocotb scheduler you already understand — then triggers, signal handles, and testbenches 1.0 and 2.0. Part III (Chapter 21) is a single load-bearing chapter on macros, Rust's answer to decorators and metaclasses. Part IV (Chapters 22–39) rebuilds the UVM: components, the lifecycle, configuration, the factory's job (done, you may be surprised to learn, by closures), component communication, and sequences, ending at testbench 8.0 — the same summit as last time, by a steeper and more scenic route. Part V wraps the complete testbench into a reference template and looks ahead.

The TinyALU is waiting. It has not gotten any bigger, and this time, neither will our tolerance for runtime errors.

Chapter 2: Rust Concepts

> **In Python we...** learned that everything is an object, and every object is an instance of a class. We called `type(5)` and Python told us, at runtime, that `5` was an instance of `int`. Variables held handles to objects, not bits, so we never worried about data sizes, and dynamic typing let us ask forgiveness rather than permission: try any operation on any object, and Python would raise an `AttributeError` if the object couldn't oblige. *(Python for RTL Verification, "Python concepts")*

The Python book opened its concepts chapter with a little language history: programming began with engineers pushing bits around, types were invented so that a 16-bit `int` couldn't silently trample an 8-bit `char`, and languages arranged themselves along a spectrum of strictness. VHDL and Pascal demanded permission for everything. C and SystemVerilog were permissive to a fault — SystemVerilog would happily chop the top eight bits off a 16-bit value to cram it into a byte. Python opted out of the argument entirely: since variables hold handles to objects rather than bits, there is nothing to chop, and the question of type compatibility gets answered at runtime, one operation at a time.

Rust takes a position on that spectrum you have seen before — it is statically typed, like VHDL — but it occupies the position so differently that the comparison will mislead you if you stop there. This chapter is about the difference. Not the syntax (that starts in Chapter 3), but the worldview: where the types live, when the checking happens, and why the strictest compiler you have ever met is going to become the most useful colleague you have.

Where the objects went

In Python, the number `5` is an object of class `int`, and it knows it. The type information travels with the object at runtime; that is what makes `type(5)` possible, and it is what makes dynamic typing work — every operation on every object begins with the interpreter looking up, right then, whether the object can do the thing.

Rust's `5` has a type too — but the type exists only in the compiler's head. Every value in a Rust program has exactly one type, known completely before the program runs. The compiler uses that knowledge to check every operation in the program, and then it throws the types away. The compiled binary contains no type tags, no class objects, no lookup tables — just the machine code that the types proved correct. There is no `type()` function in Rust for the same reason there is no ghost in a finished building: by the time the program runs, the types have done their work and left.[1]

This means the checking that Python spreads across the whole runtime, Rust performs all at once, up front. The Python book demonstrated dynamic typing with a program that called a string method on an integer — and the key detail was *when* the program failed. Let's port that exact example. In figure 1, `ends_with` is a real method on Rust strings, just as `endswith()` was in Python, and calling a method looks exactly the way it looked in Python: `mystring.ends_with("orld")`. Then we try it on an integer.

```
# Figure 1: Calling an undefined method

fn main() {
```

```
    let mystring = "Hello, World";
    println!("{}", mystring.ends_with("orld"));
    let myint: u8 = 42;
    println!("{}", myint.ends_with("a whimper"));
}
```

```
--
% cargo run
   Compiling concepts v0.1.0
error[E0599]: no method named `ends_with` found for type `u8` in the current scope
 --> src/main.rs:5:26
  |
5 |     println!("{}", myint.ends_with("a whimper"));
  |                          ^^^^^^^^^ method not found in `u8`

error: could not compile `concepts` (bin "concepts") due to 1 previous error
```

Compare this to the Python book's version of the same mistake. There, the program *ran*: it printed `True` for the string, created the integer, and only then died with an `AttributeError` at line 4. The first three lines executed; the trouble was discovered in the act of transgressing.

Here, nothing ran. Not the broken line, and — look carefully — not the correct lines either. Rust refused to produce a program at all. That is the trade in its purest form: Python checks each operation the moment it happens, so a mistake costs you a run; Rust checks every operation before any of them happen, so a mistake costs you a compile. In Chapter 1 we noted that for verification work this trade is nearly a gift, because in our world "a run" is not a millisecond of interpreter time — it is a simulator license, an elaboration, and a lunch break. Delete the offending line, as in figure 2, and the program compiles and runs.

```
# Figure 2: The corrected program

fn main() {
    let mystring = "Hello, World";
    println!("{}", mystring.ends_with("orld"));
}
```

```
--
true
```

One reassurance before we move on: statically typed does not mean verbosely typed. You may remember pages of SystemVerilog declarations and be bracing for their return. Rust's compiler performs *type inference* — in figure 1 we never told it that `mystring` was a string; it worked that out from the value. You will write far fewer type annotations than you did in SystemVerilog, and the ones you do write (like the `u8` above) tend to be the ones a verification engineer *wants* to write, because in our business the sizes are the specification. The TinyALU's A port really is eight bits wide, and Chapter 3 will make that `u8` official.

[1] Rust does keep a sliver of type information around for one feature we'll meet much later (trait objects, Chapter 10), but the rule to internalize is: no runtime type lookup, because no runtime types.

The compiler as collaborator

Everything in this book now depends on a mental adjustment, so let's make it explicitly.

When the Python interpreter raised an exception, it was reporting an accident that had already happened: the program was running, it hit something it couldn't do, and it stopped. When the Rust compiler rejects your program, nothing has happened yet. A rejection is not a failure report — it is a *prediction*, delivered

while the mistake is still free. The compiler is not an adversary blocking the door to the simulator. It is a reviewer who reads every line of your testbench, every time, in seconds, and never gets bored or skims.

The catch is that this reviewer communicates in a format you have to learn to read, and reading it well is a genuine skill — the single most valuable skill in this book, which is why our figures will so often show error messages rather than output. Let's dissect one. Figure 3 makes a mistake with TinyALU flavor: an ADD of two 8-bit operands can carry into nine bits, so the result belongs in a `u16` — but we try to store it back into a `u8` register. This is precisely the assignment the Python book warned about in its history lesson, the one where SystemVerilog "will happily chop off the top eight bits."

```
# Figure 3: The mistake SystemVerilog would have allowed

fn main() {
    let a: u8 = 0xFF;
    let b: u8 = 0x01;
    let result: u16 = a as u16 + b as u16;
    let reg: u8 = result;
    println!("{}", reg);
}
```

```
--
error[E0308]: mismatched types
 --> src/main.rs:5:19
  |
5 |     let reg: u8 = result;
  |              --   ^^^^^^ expected `u8`, found `u16`
  |              |
  |              expected due to this
  |
help: you can convert a `u16` to a `u8` and panic if the converted value
      doesn't fit
  |
5 |     let reg: u8 = result.try_into().unwrap();
  |                          ++++++++++++++++++++
For more information about this error, try `rustc --explain E0308`.
```

Read it from the top, because the compiler writes for readers who do:

- **The headline**: `error[E0308]: mismatched types`. Every error has a code, and the last line of the output tells you that `rustc --explain E0308` will print a small essay about this class of error, with examples. That command is the book behind the book.
- **The location and the arrows**: file, line, column, then your own code quoted back with carets under the guilty expression. The compiler doesn't just point at the line — it points at the exact expression, and the `expected due to this` arrow points at the *other* place involved, the annotation that created the expectation. Two pointers, because a type mismatch always has two ends.
- **The `help` block**: a suggested fix, often as a ready-to-paste diff (those `+` signs mark what to insert). And notice what this particular suggestion says out loud: converting a `u16` to a `u8` *can lose data*, so the suggested conversion is one that checks at runtime and panics if the value doesn't fit. Rust will let you chop bits — with `result as u8`, exactly the truncation SystemVerilog performs silently — but you must write the chop yourself, in ink, where a reviewer can see it.

Figure 4 takes the honest fix: our register was simply too small for an ADD result, so we widen it. The point of the figure is how little drama the fix involves once the message has told you both ends of the mismatch.

```
# Figure 4: The corrected program

fn main() {
```

```
    let a: u8 = 0xFF;
    let b: u8 = 0x01;
    let result: u16 = a as u16 + b as u16;
    let reg: u16 = result;
    println!("{}", reg);
}
```

```
--
256
```

The compiler's helpfulness extends past types into plain proofreading. The Python book admitted that a typo'd name in Python surfaces as a runtime `AttributeError`, possibly weeks later, possibly on the one branch of the testbench that only executes when the DUT misbehaves. In Rust:

```
# Figure 5: The compiler as proofreader

fn main() {
    let result = 42;
    println!("{}", resutl);
}
```

```
--
error[E0425]: cannot find value `resutl` in this scope
 --> src/main.rs:3:20
  |
3 |     println!("{}", resutl);
  |                    ^^^^^^ help: a local variable with a similar name
  |                             exists: `result`
```

It found the typo, and then it found the variable you meant. This is the texture of working in Rust: the error messages are not walls, they are directions. When this book's later chapters claim that a misconnected TLM port or a mistyped configuration "becomes a compile error," figures like these are what that will look like — and by then you will read them at a glance.

A word of honest preparation, though. The errors in this chapter are the friendly kind, because the mistakes were simple. Starting in Chapter 5, the compiler will begin rejecting programs for *ownership* reasons — code that looks obviously fine to a Python-trained eye — and those messages take real practice to read. The habit to build now, on easy errors, is the one that will carry you through the hard ones: read the first error first (later errors are often echoes of it), read the *whole* message including the help block, and trust that the compiler is describing a real problem even when you don't yet see it. In several years of Rust's existence, the compiler has been wrong about this far less often than its users have.[2]

[2] The Rust project treats confusing error messages as bugs and accepts bug reports about them. It is the only compiler I know with a customer-service department.

No interpreter, no garbage collector, no GIL

Three pieces of Python machinery are simply absent from a running Rust program, and each absence will matter to us.

No interpreter. Python source is executed by a program that reads it — every signal comparison, every scoreboard update pays the interpreter's overhead, and every deployment needs the right interpreter version installed. Rust source is *compiled away*: `cargo run` in Chapter 1 produced a native binary, the same species of artifact as the simulator itself, and the source code is not consulted again. This is where the speed comes from, and it is also where the "no environment" benefit from Chapter 1 comes from — there is nothing to install on the farm machines because the program is already finished.

No garbage collector. Python's GC is why the Python book could show two variables holding handles to one object and never ask who was responsible for it — memory management was somebody else's job, done invisibly, at times of the runtime's choosing. Rust has no such somebody. Memory is reclaimed at points the compiler determines *while compiling*, according to rules about which variable owns which value. Those rules are the famous part of Rust — ownership — and they are the subject of Chapter 5, where we will need a full chapter to replace the instinct that "assignment copies a handle and both names stay alive." For now, one sentence of preview: in Rust, assignment usually *moves* a value to its new owner, and that single idea eliminates the garbage collector, the pauses, and — more interesting to us — a whole family of "two tasks touched one transaction" testbench bugs.

No GIL. Python's Global Interpreter Lock serializes threads, which is why cocotb never pretended to give you parallelism — everything ran cooperatively on one thread. Rust has no GIL; its compiler can prove threaded code free of data races, and "fearless concurrency" is a genuine Rust selling point. I mention it mostly to lower your expectations: our testbenches will still run cooperatively on the simulator's single thread, because the simulator interface demands it, and the design of our libraries enforces that rule at compile time rather than in documentation. The GIL's absence is real; our use for it, in this book, is modest.

The toolchain

The Python book could assume you had Python and then teach `pip` when packages arrived. Rust's tooling deserves a proper introduction, because it is unusually good and because you will touch all of it in the next few chapters. The pleasant surprise is that the whole kit arrives as a set with well-defined jobs — a verification engineer who has juggled `pip`, `venv`, `pyenv`, `black`, and `pylint` will recognize every silhouette here, minus the juggling.

`rustup` installs and manages Rust itself — the job `pyenv` did for Python versions. One command installs the whole toolchain; `rustup update` moves you to the latest release. Rust ships a new stable version every six weeks, and — this matters — the project holds a strong backward-compatibility promise, so updating is routine rather than an event.

Which raises the obvious worry for anyone who lived through Python 2-to-3: what happens when the language needs to change incompatibly? Rust's answer is **editions**. Every few years (2015, 2018, 2021, 2024) the project bundles its rare breaking changes into a named edition, and each crate — Rust's word for a package, official introductions in Chapter 14 — declares in its manifest which edition it speaks. The compiler supports all of them, forever, and crates from different editions link together in one program. It is Python 3 without the decade of schism: old code keeps compiling, new code gets the improvements, and nobody writes a farewell blog post. The projects in this book use the 2024 edition, which is what `cargo new` selects for you.

`cargo` you met in Chapter 1 — `pip`, `venv`, `make`, and `pytest` fused into one tool. It creates projects (`cargo new`), builds them (`cargo build`), builds-and-runs them (`cargo run`), fetches dependencies declared in `Cargo.toml`, and runs tests (`cargo test` — the star of Chapter 14, where we will unit-test testbench components with no simulator in sight). Because every Rust project is a cargo project with the same layout, there is no per-project incantation to learn; every example in this book builds the same way.

`rustfmt` reformats your source into the community-standard style, the job `black` did in Python: `cargo fmt`, and the formatting debate is over. This book's figures are all `rustfmt`-clean, so what you read is what your editor will produce.

`clippy` is the linter — `pylint`'s job — but with a compiler's leverage, since it analyzes the same typed, checked view of your program the compiler sees. It catches correctness hazards, but its everyday gift to a Rust learner is idiom: clippy knows what fluent Rust looks like and will tell you, kindly and specifically, when you have written Python in Rust syntax. Run it as `cargo clippy`; a representative complaint (output abridged) looks like this:

```
# Figure 6: Clippy teaching idiom

fn main() {
    let done = true;
    if done == true {
        println!("finished");
    }
}
```

```
--
% cargo clippy
warning: equality checks against true are unnecessary
 --> src/main.rs:3:8
  |
3 |     if done == true {
  |        ^^^^^^^^^^^^ help: try simplifying it as shown: `done`
```

Notice the shape: the same location-caret-help anatomy as a compiler error, because it comes from the same machinery. Reading one teaches you to read the other. Make `cargo clippy` a habit early — while you are learning, it is the closest thing to a Rust mentor watching over your shoulder, and unlike a mentor it never sighs.

Summary

Python's core idea was that everything is an object carrying its type at runtime, checked operation by operation as the program runs. Rust's core idea is that every value has a type known *before* the program runs, checked exhaustively by a compiler that then erases the types entirely — leaving a native binary with no interpreter, no garbage collector, and no GIL. The cost is that the compiler rejects programs Python would have happily started; the payoff is that it rejects them in seconds, with error messages that name the problem, point at both ends of it, and usually propose the fix. Learning to read those messages is the skill this book will exercise constantly, and the toolchain — `rustup` for the compiler, `cargo` for everything, `rustfmt` for style, `clippy` for idiom — is the same set of jobs your Python toolbelt did, consolidated and sharpened.

Concepts in hand, it is time to write some actual Rust. Chapter 3 starts where the Python book's basics chapter started — variables, numbers, and printing — and the TinyALU's 8-bit operands are about to get types that say so.

Chapter 3: Rust Basics

> **In Python we...** learned that everything is an object. When we wrote `xx = 5`, Python instantiated an `int` object of value 5 on the heap and handed `xx` a reference to it. There was no maximum `int`, no bit width, and no declared type — the *object* had a type, the *variable* was just a name, and we could point that name at anything else a line later. The Python book put it this way: when you say `byte xx = 5` in SystemVerilog, the program sets aside an 8-bit memory location; when you say `xx = 5` in Python, you get a handle to an `int` object. This chapter is about coming back from that trip.

Chapter 2 introduced the compiler as a collaborator. This chapter puts it to work on the smallest possible material: variables, numbers, strings of output, and functions. None of it is hard, but nearly all of it is *different* from Python in ways that will matter every day, so we will move through it the way the Python book moved through its basics chapter — with small figures you can run and diff against the ones you already know.

If you want to follow along (you should), make a playground the way we did in Chapter 1:

```
# Figure 1: A playground for this chapter

% cargo new basics
    Creating binary (application) `basics` package
```

Everything in this chapter goes inside `fn main()` in `src/basics/src/main.rs` unless a figure shows its own function.

`let`: bindings, not name tags

In Python, a variable is a name tag you can peel off one object and stick on another. In Rust, `let` creates a **binding**: it associates a name with a value, and — here is the first surprise — that binding is **immutable by default**. Assigning to it a second time is not merely discouraged. It does not compile.

```
# Figure 2: Assigning twice to an immutable binding

fn main() {
    let xx = 5;
    println!("xx: {xx}");
    xx = 6;
    println!("xx: {xx}");
}
```

```
% cargo run
error[E0384]: cannot assign twice to immutable variable `xx`
 --> src/main.rs:4:5
  |
```

```
2 |     let xx = 5;
  |         -- first assignment to `xx`
3 |     println!("xx: {xx}");
4 |     xx = 6;
  |     ^^^^^^ cannot assign twice to immutable variable
  |
help: consider making this binding mutable
  |
2 |     let mut xx = 5;
  |         +++
```

Read that error the way Chapter 2 taught you: it names the rule, points at both the first assignment and the offending one, and then *tells you the fix*. If you want a variable that varies, you ask for one with `mut`:

```
# Figure 3: A mutable binding

fn main() {
    let mut xx = 5;
    println!("xx: {xx}");
    xx = 6;
    println!("xx: {xx}");
}
```

—

```
xx: 5
xx: 6
```

Coming from Python, immutability-by-default feels backwards for about a week. Then you start reading other people's testbench code and discover what it buys you: every `mut` in a Rust program is a signpost saying *this value changes — watch it*. In Python, every variable carried that warning implicitly, which is the same as no variable carrying it at all. When you read a Rust monitor and see that only one binding in it is `mut`, you know where the state lives. The compiler is not restricting you; it is making your intentions legible.[1]

[1] It will also nag you in the other direction: declare something `mut` and never mutate it, and the compiler warns you to take the `mut` off. It wants the signposts accurate in both directions.

Scalar types: the bits are back

The Python book's basics chapter needed exactly three number types — `bool`, `int`, and `float` — and Python's `int` had no maximum value. Rust returns us to the world hardware people never really left: integers have widths, and the widths are in the names.

The scalar types you will actually use:

- **Integers, unsigned:** `u8`, `u16`, `u32`, `u64` — 8 to 64 bits, no sign bit. There is also `usize`, the pointer-width integer that Rust uses for indexing and lengths.
- **Integers, signed:** `i8`, `i16`, `i32`, `i64` — two's complement, exactly as your DUT stores them. Bare integer literals like `5` default to `i32`.
- **Floats:** `f32` and `f64`. Bare float literals like `2.0` default to `f64`, which is Python's `float`.
- **`bool`:** `true` and `false` — lowercase now, and the *only* things a condition accepts. Python's habit of treating `None`, `0`, and empty containers as falsy does not exist here; an `if` takes a `bool`, full stop.
- **`char`:** a single Unicode character, in single quotes: `'A'`. Not a one-character string — a distinct type, four bytes wide.

Why should a verification engineer care, beyond nostalgia for SystemVerilog's `byte`? Because *your DUT already thinks this way*, and now your testbench can agree with it. The TinyALU's A and B legs are eight

bits wide. In the Python book we drove them from an `int` and relied on discipline (and the BFM) to keep values in range — nothing in the language stopped a careless test from generating `a = 300`. In Rust, a TinyALU operand is a `u8`, and 300 *is not a value that type can hold*. The type system now knows something true about the hardware, and it never forgets it:

```
# Figure 4: The TinyALU's A leg really is a u8

fn main() {
    let aa: u8 = 0xFF;
    let bb: u8 = 300;
    println!("aa: {aa}, bb: {bb}");
}
```

```
% cargo run
error: literal out of range for `u8`
 --> src/main.rs:3:18
  |
3 |     let bb: u8 = 300;
  |                  ^^^
  |
  = note: the literal `300` does not fit into the type `u8`
    whose range is `0..=255`
```

Notice the `let aa: u8 = 0xFF;` syntax: a colon and a type after the name is a **type annotation**. Most of the time you can leave it off and Rust infers the type from context — this is why Figures 2 and 3 didn't need one — but when you mean *eight bits*, say so.

One honest wrinkle while we are here: what happens when arithmetic *overflows* a `u8` at runtime — say, `200 + 100` where both values arrived from a random generator? In a debug build, the program panics (halts with an error) at the overflowing operation; in a release build, the value wraps around modulo 256, the way the hardware would. If wrapping is what you *mean* — and in ALU-prediction code it often is — Rust provides methods like `wrapping_add` that say so explicitly. We will use exactly that when we write the TinyALU predictor, because "the model overflows exactly like the DUT, on purpose, in writing" is the kind of sentence verification sign-off meetings love.

No implicit conversions — diffing against the Python book

Here is the Python book's very first figure, which showed that a `float` in an operation means a `float` result — `ii + ff` quietly promoted the `int`, and `ii/ii` produced a `float` even from two `int`s. Let's port it one-for-one and watch Rust refuse to play:

```
# Figure 5: A float in operations means... a compile error

fn main() {
    let ii: i32 = 1;
    let ff: f64 = 2.0;
    let ss = ii + ff;
    println!("ss: {ss}");
}
```

```
% cargo run
error[E0277]: cannot add a `f64` to `i32`
 --> src/main.rs:4:17
```

```
  |
4 |     let ss = ii + ff;
  |                  ^ no implementation for `i32 + f64`
```

Rust performs **no implicit numeric conversions**. Not int-to-float, not `u8`-to-`u16`, nothing. If you want a conversion, you write one, using the `as` keyword — and once you do, the rest of the ported figure behaves recognizably, with one telling difference in the last line:

```
# Figure 6: The same figure, with the conversions made explicit

fn main() {
    let ii: i32 = 1;
    let ff: f64 = 2.0;
    let ss = ii as f64 + ff;
    println!("ss: {ss}");
    let dd = ii / ii;
    println!("dd: {dd}");
}
```

```
ss: 3
dd: 1
```

In the Python original, `dd = ii/ii` printed `1.0` — division *always* returned a `float`. In Rust, dividing two integers is integer division: `dd` is `1`, an `i32`, and `7 / 2` would be `3`. If you want the fractional answer, convert to floats first. Neither behavior is right or wrong, but they are different, and scoreboard math is exactly where that difference bites — so it is worth one figure now instead of one confused afternoon later.

The same strictness rewrites the Python book's augmented-assignment figure. In Python, `xx` started as an `int` at 1, and after `xx /= 4` it had silently become a `float` holding 1.5 — the variable changed *type* mid-flight. Watch the Rust version:

```
# Figure 7: Augmented assignments - the type never changes

fn main() {
    let mut xx = 1;
    println!("xx: {xx}");
    xx += 1;
    println!("xx += 1: {xx}");
    xx *= 3;
    println!("xx *= 3: {xx}");
    xx /= 4;
    println!("xx /= 4: {xx}");
}
```

```
xx: 1
xx += 1: 2
xx *= 3: 6
xx /= 4: 1
```

Same operators, same rhythm — Rust has `+=`, `*=`, `/=`, and friends, though like Python it has no `++`. But `xx` was born an `i32` and will die an `i32`; `6 /= 4` gives 1, not 1.5. A binding's type is fixed for the binding's whole life. Which raises an obvious question: what do we do when we genuinely want the Python pattern — same *idea*, new *type*?

Shadowing: same name, new binding

The Python book's constructor figure turned the string `"3.14159"` into a `float` by creating a new object: `pi = float("3.14159")`. Rust's version of that pattern is **shadowing**: declaring a *new* binding, with `let`, that reuses an old name.

```
# Figure 8: Creating a number from a string, by shadowing

fn main() {
    let pi = "3.14159";
    let pi: f64 = pi.parse().expect("not a number");
    println!("pi: {pi}");
}
```

```
pi: 3.14159
```

The first `pi` is a string; the second `pi` is a brand-new binding, a `f64`, whose value came from parsing the first. From that line on, the name `pi` means the number; the string version is shadowed — inaccessible, retired with honors. This is not mutation (nothing was `mut`) and it is not a type change (each binding kept its type); it is the "same idea, new type" idiom, done with two immutable bindings instead of one shape-shifting variable. You will see it constantly in testbench code: parse a string into a number, convert raw bits into a transaction, and keep the natural name at every step.

Two small notes on Figure 8. First, `parse` can fail — `"pi".parse()` has nowhere good to go — so it returns a `Result`, Rust's replacement for exceptions; `.expect("...")` says "give me the value, and halt with this message if it failed." That is a blunt instrument we will trade for proper tools in Chapter 9; the Python original had the same rough edge, raising `ValueError` on `int("3.14159")`. Second, the annotation `: f64` is doing real work: it is how `parse` knows *what* to parse the string into.

`println!` and format strings

You have been reading `println!` output all chapter; now let's look at the format strings themselves, next to the f-strings you know. `{}` is the placeholder, arguments fill placeholders in order, and — the part that makes Rust feel almost Pythonic — a variable name can go directly inside the braces:

```
# Figure 9: Format strings, next to the f-strings you know

fn main() {
    let aa: u8 = 0x2A;
    let bb: u8 = 7;
    println!("aa is {} and bb is {}", aa, bb);  // like str.format()
    println!("aa is {aa} and bb is {bb}");      // like an f-string
    println!("aa in hex: {aa:#04x}");
    println!("aa in binary: {aa:#010b}");
    println!("sum: {}", aa + bb);
}
```

```
aa is 42 and bb is 7
aa is 42 and bb is 7
aa in hex: 0x2a
aa in binary: 0b00101010
sum: 49
```

The format specifiers after the colon will feel familiar from both Python and `$display`: `{aa:#04x}` means hexadecimal, `#` for the `0x` prefix, padded to width 4. The hex and binary forms in Figure 9 are the ones you will reach for when a scoreboard mismatch needs to be read against a waveform.

One genuine difference from f-strings: the braces capture *names only*, not arbitrary expressions. Python lets you write `f"{aa + bb}"`; Rust makes you write the expression as an argument, as in the last line of Figure 9. And the exclamation point still means what Chapter 1 said it means: `println!` is a macro, which is precisely *why* it can type-check your format string against your arguments at compile time — pass one argument too few and the program does not build, where Python's `"{} {}".format(x)` waited until runtime to complain.

Expressions vs. statements

Here is the concept in this chapter most likely to be genuinely new, rather than a stricter spelling of something Python had. Python divides the world into statements (`if`, `for`, assignments) and expressions (things with values), and mostly keeps them apart. In Rust, nearly everything is an **expression** — nearly everything *has a value* — and the language leans on this constantly.

Two demonstrations. First, `if` is an expression, which means it can sit on the right-hand side of a `let`:

```
# Figure 10: if is an expression

fn main() {
    let count: u8 = 42;
    let parity = if count % 2 == 0 { "even" } else { "odd" };
    println!("count is {parity}");
}
```

```
count is even
```

Python has the ternary form `"even" if count % 2 == 0 else "odd"` for exactly this job; Rust simply has no separate ternary, because ordinary `if` already returns a value. The compiler checks that both arms produce the same type — an `if` that gives you a string on Mondays and an integer on Tuesdays does not compile — and an `else` is required when you use the value, because the value must exist either way.

Second, a block — any `{ ... }` — is an expression whose value is its **last expression, written without a semicolon**:

```
# Figure 11: A block is an expression; the semicolon is the switch

fn main() {
    let nn = {
        let doubled = 2 * 3;
        doubled + 1
    };
    println!("nn: {nn}");
}
```

```
nn: 7
```

Look hard at `doubled + 1` — no semicolon. That is not sloppy punctuation; it is load-bearing syntax. A trailing expression without a semicolon is the block's value; add a semicolon and you have turned it into a statement, the block's value becomes the empty "unit" value `()`, and the compiler will greet you with an error message that — helpfully — points straight at the semicolon and suggests removing it. Every Rust programmer alive has been rescued by that message.[2] Once this clicks, you will start to see Rust code as a

tree of nested expressions, each yielding a value to the one above it, and the language's whole shape gets simpler.

[2] Usually within the first hour.

Functions

Which brings us, with suspicious convenience, to functions — because a function body is just another block, and returning a value is just the block-expression rule again.

```
# Figure 12: A TinyALU prediction function

fn predict_add(aa: u8, bb: u8) -> u16 {
    aa as u16 + bb as u16
}

fn main() {
    let sum = predict_add(0xFF, 0xFF);
    println!("predicted sum: {sum:#06x}");
}
```

```
predicted sum: 0x01fe
```

Everything Python left optional is now required, and everything required is now checked. Each parameter declares its type; the `-> u16` arrow declares the return type; and the last expression of the body — `aa as u16 + bb as u16`, no semicolon — is the return value. (An explicit `return` keyword exists for bailing out early, but idiomatic Rust lets the final expression speak for itself.) A function with no `->` returns `()`, Rust's cousin of Python's implicit `None`.

Figure 12 also carries the chapter's TinyALU payload. The Python book's prediction function took two Python `int`s and returned a Python `int`, and the fact that the TinyALU's result port is *sixteen* bits while its operands are *eight* lived only in prose and in the DUT. Here it lives in the signature: `fn predict_add(aa: u8, bb: u8) -> u16` *is* the TinyALU's ADD operation, as a type. `0xFF + 0xFF` overflows a `u8` — which is exactly why the hardware has a wide result port, and exactly why the function converts each operand to `u16` before adding. Try deleting the two `as u16` conversions and read the error you get; the compiler will explain the TinyALU datasheet to you.

And that signature pays one more dividend Python could not offer: the compiler checks every *call*. Pass `predict_add` a 16-bit value, or three arguments, or use its result where a `u8` is expected, and the testbench does not build. In the Python book, a mis-called prediction function was a runtime discovery; here it never gets as far as the simulator.

Comments, briefly

Line comments are `//` to end of line, matching Python's `#` in spirit. Block comments `/* ... */` exist but are rare in practice. What Rust has that Python approximated with docstrings is **doc comments**: lines beginning `///` above a function or type are documentation the toolchain actually compiles into browsable HTML (`cargo doc`). We will start writing them when we write code worth documenting, which is soon.

Summary

This chapter covered Rust's nuts and bolts, each one a deliberate diff against the Python book's basics chapter:

- **`let` bindings** — immutable by default; `mut` is an explicit, visible request for mutability
- **scalar types** — `u8` through `i64`, `f32`/`f64`, `bool`, `char`; bit widths are back, and the TinyALU's operands are honest `u8`s at last
- **no implicit conversions** — mixed-type arithmetic is a compile error; `as` makes conversions visible; integer division stays integer
- **shadowing** — the "same idea, new type" pattern, done with a fresh binding instead of a shape-shifting variable
- **`println!` and format strings** — f-string-like `{name}` captures, plus compile-time checking of the format string itself
- **expressions vs. statements** — `if` and blocks have values; the trailing-expression-without-semicolon rule
- **functions** — typed parameters, declared return types, and signatures the compiler enforces at every call site

Every figure here was straight-line code — no branches worth mentioning, no loops at all. A testbench that never loops is not much of a testbench, and besides, Rust is holding back its best conditional construct: `match`, which Python never had and which the rest of this book will use on nearly every page. Both are waiting in Chapter 4.

Chapter 4: Conditions, Loops, and Match

> **In Python we...** designated blocks with indentation, tested conditions with `if`/`elif`/`else`, and observed that `elif` "provides similar functionality to `case` and `switch` statements, but with more flexibility at the cost of more code." We wrote conditional assignments with the inline ternary — `message = "five_val" if aa == 5 else "other_val"` — and we looped with `while` and `for`, emulating a do-while with `while True:` and a well-placed `break`. Ranges came from the `range()` constructor: `range(stop)`, `range(start, stop)`, `range(start, stop, step)`. — *Python for RTL Verification*, "Conditions and loops" and "Ranges"

This chapter ports all of that to Rust, and then keeps going, because Rust has a construct Python never gave us: `match`. Two-thirds of this chapter is warm-up — conditions and loops transfer from Python almost without friction — and the last third introduces the construct the rest of this book leans on constantly. When we meet `Result` in Chapter 9, `Option` alongside it, and the `Ops` enum in Chapter 7, `match` is how we will take them apart. Learn it well here, in miniature, and every later chapter gets easier.

if and else

Rust's `if` looks like Python's with the punctuation swapped: braces instead of indentation, and no colon. Like Python — and unlike C and SystemVerilog — Rust does not require parentheses around the condition. Unlike everybody, Rust *insists* on the braces, even for one-line bodies.

```
# Figure 1: A Rust if statement

fn main() {
    let name = "Roy";
    if name != "Danny" {
        println!("Hey, you're not Danny.");
    }
}
```

```
--
Hey, you're not Danny.
```

One difference matters more than it looks. Python happily tested any value for truth: `if nn:` meant "if `nn` is nonzero," `if my_list:` meant "if the list is nonempty." Rust conditions must be `bool` — actually, literally `bool`. Write `if nn {` where `nn` is an integer and the compiler stops you:

```
# Figure 2: Rust has no truthiness

fn main() {
    let nn = 5;
    if nn {
```

```
        println!("nonzero");
    }
}
```

```
--
error[E0308]: mismatched types
 --> src/main.rs:3:8
  |
3 |     if nn {
  |        ^^ expected `bool`, found integer
```

You will grumble about this for a week and then remember every testbench where `if dut.done:` silently tested the wrong thing — a handle instead of a value, a list instead of its contents. Rust makes you write `if nn != 0`, and the intent goes on the record.[1]

There is no `elif` keyword. Rust spells it `else if`, and a chain of them ports the Python book's switch replacement directly:

```
# Figure 3: else if as a switch (for now)

fn main() {
    let (a, b) = (5, 5);
    let operation = "divide";
    if operation == "add" {
        println!("A + B = {}", a + b);
    } else if operation == "subtract" {
        println!("A - B = {}", a - b);
    } else if operation == "multiply" {
        println!("A * B = {}", a * b);
    } else {
        println!("Illegal Operation: {operation}");
    }
}
```

```
--
Illegal Operation: divide
```

The Python book called `elif` a switch replacement "at the cost of more code." Hold that thought — the cost is about to be refunded, with interest, in the `match` section.

[1] SystemVerilog veterans have the opposite scar: `if (sig)` on a 4-state signal, where an `X` quietly takes the false branch. Rust's answer to both languages is the same: say what you mean, and the compiler will hold you to it.

if is an expression

Here is the first genuinely new idea. In Chapter 3 we met Rust's distinction between statements and expressions: an expression produces a value. In Rust, `if`/`else` *is an expression* — the whole construct evaluates to the value of whichever branch ran. That means you can bind it with `let`:

```
# Figure 4: Conditional assignment - no ternary needed

fn main() {
    let aa = 5;
    let message = if aa == 5 { "five_val" } else { "other_val" };
    println!("{message}");
}
```

```
--
five_val
```

Python needed special ternary syntax — `x if cond else y` — because its `if` statement produces nothing. Rust needs no ternary because its ordinary `if` already produces a value.[2] Two rules come along with this power. First, both branches must produce the *same type* — a branch handing back a string while the other hands back an integer is a compile error, because `message` must be exactly one type. Second, if you use `if` as an expression, the `else` is mandatory; the compiler will not let you bind a value that might not exist. Both rules are the compiler asking the question Python deferred to runtime: *what, exactly, is this variable?*

[2] C and SystemVerilog programmers may now retire the `?` operator with full honors.

Three loops, one of them new

Python gave us two loops, `while` and `for`. Rust gives us three: `while`, `for`, and `loop`. The first two are your old friends in new clothes; the third is the new idea.

`while` works exactly as you expect, condition first, braces around the body. Here is the Python book's counting loop, ported:

```
# Figure 5: A while loop in action

fn main() {
    let mut nn = 0;
    while nn <= 13 {
        print!("{nn} ");
        nn += 1;
    }
    println!();
}
```

```
--
0 1 2 3 4 5 6 7 8 9 10 11 12 13
```

Two Chapter 3 details show up here: `nn` needs `mut` because we reassign it, and `print!` (without the `ln`) is how Rust says `end=" "` — it prints without the newline. `continue` and `break` exist, spelled the same and meaning the same as in Python; I will not re-teach them.

Now the new one. The Python book emulated a do-while with `while True:` and a `break`. Rust looked at how often programmers write intentionally infinite loops — event loops, polling loops, driver loops that run until the test ends — and decided the pattern deserved its own keyword:

```
# Figure 6: loop - the intentional infinite loop

fn main() {
    let mut nn = 0;
    let first_big_square = loop {
        nn += 1;
        if nn * nn > 200 {
            break nn * nn;
        }
    };
    println!("{first_big_square}");
}
```

```
--
```

225

Look closely at that `break`: it carries a value, and the whole `loop` evaluates to it — which is why we could write `let first_big_square = loop { ... }`. Like `if`, `loop` is an expression. A "run until something happens, then hand back what you found" pattern that took a flag variable and a post-loop read in Python is one construct in Rust. Only `loop` gets this privilege; `while` and `for` loops cannot `break` with a value, because their conditions mean they might never produce one.

If `loop` sounds like a novelty, consider what you already know is coming: every BFM driver loop and monitor loop in the Python book was a `while True:` at heart. In Part II, those become `loop` — the language admitting what the code always meant.

Ranges

Python's `range()` was a constructor with three calling conventions. Rust builds ranges from operators instead:

- `0..8` — start at 0, stop *before* 8. The same half-open interval as `range(0, 8)`.
- `0..=8` — start at 0, stop *at* 8, inclusive. Python had no direct equivalent; you wrote `range(0, 9)` and remembered why.

The `for` loop iterates over a range just as Python's did:

```
# Figure 7: Looping through numbers using a range

fn main() {
    for ii in 0..4 {
        print!("{ii} ");
    }
    println!();
}
```

```
--
0 1 2 3
```

Where is `step`? Ranges are iterators (a concept that transfers straight from the Python book's sequences chapters — Chapter 12 makes it rigorous), and iterators have adapter methods. `range(1, 14, 2)` becomes:

```
# Figure 8: Stepping through a range

fn main() {
    for ii in (1..14).step_by(2) {
        print!("{ii} ");
    }
    println!();
}
```

```
--
1 3 5 7 9 11 13
```

There is also `.rev()` for counting down, and a whole catalog of adapters waiting in Chapter 12. For now: `..` exclusive, `..=` inclusive, adapters for everything else. And notice `0..=8` reads naturally for hardware — an inclusive range is how you think about the values a bus can carry. A TinyALU operand is a `u8`; the values it can take are `0..=255`, and you can write exactly that.

match: the construct Python never had

Now the centerpiece. Python replaced `case`/`switch` with `elif` chains and called the trade "more flexibility at the cost of more code." Rust's `match` refuses the trade: it delivers more flexibility than `case` *and* less code than `elif` — and then adds a property neither language offered, one that this book will spend many chapters collecting dividends on.

Start with the direct port. Figure 3's `else if` chain becomes:

```
# Figure 9: match as a switch

fn main() {
    let (a, b) = (5, 5);
    let operation = "multiply";
    let answer = match operation {
        "add" => a + b,
        "subtract" => a - b,
        "multiply" => a * b,
        _ => panic!("Illegal Operation: {operation}"),
    };
    println!("answer = {answer}");
}
```

```
--
answer = 25
```

Read it as: compare `operation` against each *pattern* on the left of a `=>`; run the code on the right of the first pattern that fits. The underscore `_` is the wildcard — it matches anything, playing the role of `else` or `default`. Three things to notice, each an upgrade over both `elif` and `case`:

1. **`match` is an expression.** Like `if` and `loop`, the whole construct produces a value — here it computes `answer` directly, where Figure 3 could only print from inside each branch. Every arm must produce the same type, same rule as `if`.
2. **There is no fallthrough.** C and SystemVerilog programmers carry decades of missing-`break` scar tissue; `match` arms are separate, always, no `break` required or even possible.
3. **The compiler checks that the patterns cover every case.** This one gets its own section.

Exhaustiveness: the compiler counts your cases

Delete the `_` arm from a `match` and something remarkable happens. Here is a `match` on a raw op code — the TinyALU's four operations, numbered 1 through 4 as they were in the Python book's `Ops` IntEnum — with no wildcard:

```
# Figure 10: The compiler catches missing cases

fn main() {
    let op_code: u8 = 2;
    let name = match op_code {
        1 => "ADD",
        2 => "AND",
        3 => "XOR",
        4 => "MUL",
    };
    println!("{name}");
}
```

```
--
```

```
error[E0004]: non-exhaustive patterns: `0_u8` and `5_u8..=u8::MAX` not covered
 --> src/main.rs:3:22
  |
3 |     let name = match op_code {
  |                      ^^^^^^^ not covered
  |
  = note: the matched value is of type `u8`
```

Sit with that error message for a moment, because it is doing something no Python testbench ever got for free. The compiler enumerated every value a `u8` can hold, subtracted the four we handled, and reported precisely what we missed: zero, and everything from 5 up. An `elif` chain that forgets a case is a runtime surprise — the Python book's Figure 3 only caught its illegal `"divide"` because we remembered to write the `else`. A `match` that forgets a case *does not compile*. The fix is either a `_` arm (an explicit decision to lump the leftovers together) or arms for the missing values (an explicit decision about each) — but it is always a decision, never an oversight.

For a `u8`, exhaustiveness is a nice safety net. The reason this book teaches `match` in Chapter 4 rather than Chapter 14 is what happens when the thing being matched has a *small, meaningful* set of cases. In Chapter 7, `Ops` returns as a true Rust enum with exactly four values, and a `match` on it needs exactly four arms — no wildcard, no dead cases, and if the TinyALU ever grows a fifth operation, **every `match` in the testbench that fails to handle it becomes a compile error**. Your scoreboard, your coverage collector, your predictor: the compiler hands you the complete list of code that must learn about the new op. That is the refactoring-without-fear promise of Chapter 1, delivered by a control-flow statement.

Patterns: ranges, tuples, and taking things apart

The left side of a `match` arm is not limited to constants. Patterns can be ranges — and here the TinyALU gives us a real example. Recall from the Python book that ADD, AND, and XOR complete in one cycle while MUL takes three. With op codes 1 through 4:

```
# Figure 11: Matching on ranges

fn main() {
    let op_code: u8 = 4;
    let cycles = match op_code {
        1..=3 => 1,
        4 => 3,
        _ => panic!("Illegal op code: {op_code}"),
    };
    println!("This operation takes {cycles} cycle(s)");
}
```

```
--
This operation takes 3 cycle(s)
```

Range patterns use the inclusive `..=` form, and you can also combine alternatives with `|`, as in `1 | 3 => ...`. The compiler still does its exhaustiveness arithmetic across all of it.

Patterns can also take structured data apart. Match on a tuple, and each position of the pattern matches the corresponding element — with `_` skipping positions you don't care about and plain names *binding* the values so the arm can use them:

```
# Figure 12: Matching and destructuring a tuple

fn main() {
    let operands: (u8, u8) = (0, 200);
    let comment = match operands {
```

```
        (0, 0) => String::from("both operands zero"),
        (0, _) | (_, 0) => String::from("one operand zero"),
        (a, b) if a == b => format!("equal operands: {a}"),
        (a, b) => format!("ordinary operands: {a}, {b}"),
    };
    println!("{comment}");
}
```

```
--
one operand zero
```

Three new tricks in one figure. The `|` combines two patterns into one arm. The names `a` and `b` in the later arms are bindings — the arm receives the matched values under those names, which is how `match` goes beyond comparing and starts *extracting*. And `if a == b` is a *match guard*, an extra boolean test bolted onto a pattern for the cases patterns alone can't express. You have just watched a `match` classify stimulus into coverage-bin-shaped categories in four lines — remember this figure when we build the coverage collector.

That extraction ability is the real reason `match` anchors this book. Here is a preview of what is coming — do not worry about the details yet. In Chapter 9 you will learn that a fallible operation like looking up a DUT signal returns a `Result`, which is either `Ok(handle)` carrying the goods or `Err(e)` carrying the explanation. The way you get the goods out is a `match`:

```
match dut.child("clk") {
    Ok(clk) => { /* use clk */ }
    Err(e) => { /* the signal wasn't there; e says why */ }
}
```

One construct checks which case you got *and* hands you its contents *and* forces you — at compile time — to say what happens in the failure case you would rather not think about. Python's `try/except` let you skip the `except` and hope; `match` on a `Result` has no such loophole. Exhaustiveness, it turns out, is not a switch-statement garnish. It is how Rust makes error handling mandatory, and Chapters 7 and 9 are where that bill comes due — in our favor.

Summary

Rust's conditions and loops are Python's with braces on: `if/else if/else` instead of `if/elif/else`, `while` and `for` behaving as you expect, `continue` and `break` unchanged. The differences all push the same direction. Conditions must be real `bools` — no truthiness. `if` is an expression, retiring the ternary. `loop` names the intentional infinite loop and can `break` with a value. Ranges are syntax (`0..8` exclusive, `0..=8` inclusive) rather than a constructor, with iterator adapters like `step_by` covering the rest.

And `match` is the construct Python never had: patterns instead of comparisons, expression instead of statement, no fallthrough, destructuring with bindings and guards — and exhaustiveness checking, the compiler's guarantee that every case is a decision and no case is an oversight. We will `match` on integers and tuples this week, on `Ops` in Chapter 7, and on `Option` and `Result` for the rest of our verification careers.

So far, every value we have used has lived and died inside `fn main` without our attention — Python habits, still serving us fine. In the next chapter, we hand a value from one variable to another and discover that Rust has been keeping track of who owns what all along. Chapter 5 is ownership: the idea with no Python-book mirror, and the hinge of the whole language.

Chapter 5: Ownership

Every chapter so far has had a Python twin. `let` had assignment, `match` had `if` chains, `u8` had the integers you already knew. This chapter has no twin, because it answers a question Python never let you hear: *when a value's time is up, who is responsible for destroying it?*

You have created millions of objects in Python — transactions, components, queues full of commands — and you have never once destroyed one. You didn't have to. Python's garbage collector followed you around the testbench like a diligent stagehand, watching which objects still had names pointing at them and quietly disposing of the ones that didn't.[1] It did this job so well, and so silently, that most Python programmers never learn the job exists.

Rust has no garbage collector. There is no stagehand. And yet Rust programs do not leak memory, do not free things twice, and do not touch things after they're freed — the classic sins of C. Rust pulls this off with one idea, enforced by the compiler, and that idea is the hinge of the entire language: **ownership**. Chapter 1 promised that the rules behind Rust's no-runtime-cost safety would bend your brain exactly once. This is the chapter where the bending happens.

> [1] CPython's stagehand is mostly a reference counter — every object carries a count of the names and containers pointing at it, and hits the trash at zero — with a cycle-detecting garbage collector mopping up the cases where two objects point at each other and the counts never fall. The details don't matter here; the silence does.

Who frees this? Python's answer

Let's watch the stagehand work. In Python, a variable is not a box holding a value — it is a name tag stuck onto an object that lives somewhere on the heap. Assignment copies the name tag, never the object. In figure 1, `a` and `b` are two tags on one transaction-ish list, which we can prove by mutating through one name and looking through the other.

```
# Figure 1: Python assignment: two names, one object

a = ["ADD", 5, 3]
b = a
b[0] = "MUL"
print(a)
print(a is b)
```

```
--
['MUL', 5, 3]
True
```

You knew this. The Python book leaned on it constantly — every component holding `self.bfm`, every scoreboard holding the same transaction the monitor held. The part you may never have said out loud is the lifetime question: that list stays alive as long as *any* tag points at it, and it dies whenever the last tag

disappears, at a moment of the garbage collector's choosing. Nobody owns the list. Ownership is smeared across every name that ever touched it, and the runtime keeps the books.

For testbenches this policy is comfortable right up until it isn't: the monitor that holds a stale handle to a component the test rebuilt, the two subscribers that received the "same" transaction and one of them mutated it, the `__del__` method that runs at a time no document will commit to. Python's answer to "who frees this?" is *nobody in particular, eventually.* Rust's answer is one word long.

Rust's answer: one owner

Here is the rule the rest of this book stands on:

> Every value in Rust has exactly one **owner** — the variable (or, later, the struct field) responsible for it. When the owner goes out of scope, the value is destroyed, immediately. Assignment doesn't copy a name tag; it **moves** ownership to the new variable, and the old one is dead.

That last clause is the shock. Let's take the shock now, on purpose, with the compiler watching. Figure 2 is the two-names experiment from figure 1, translated into Rust.

```
# Figure 2: Assignment moves - and the old name is gone

fn main() {
    let a = String::from("ADD 5 3");
    let b = a;
    println!("{a}");
    println!("{b}");
}
```

```
--
error[E0382]: borrow of moved value: `a`
 --> src/main.rs:4:15
  |
2 |     let a = String::from("ADD 5 3");
  |         - move occurs because `a` has type `String`, which does not
  |           implement the `Copy` trait
3 |     let b = a;
  |             - value moved here
4 |     println!("{a}");
  |               ^^^ value borrowed here after move
  |
help: consider cloning the value if the performance cost is acceptable
  |
3 |     let b = a.clone();
  |              ++++++++
```

Read that error the way Chapter 2 taught you, because it is one of the best-written error messages in any compiler and it narrates the whole story. Line 2: the string was created and `a` owned it. Line 3: `value moved here` — the assignment handed ownership to `b`, and `a` stopped being a valid name for anything. Line 4: we tried to use `a` after the move, and the compiler refused to build the program.

Sit with what did *not* happen. The program did not run and print something surprising. It did not crash at 2 a.m. in seed 8,441 of a regression. It never existed as a program at all. In Python, `b = a` gives you two live names and a shrug about lifetimes; in Rust, `let b = a;` is a baton pass — after it, exactly one variable is responsible for that string, and the compiler will name the exact line where responsibility changed hands. One value, one owner, at every moment, provable at compile time. That invariant is the entire trick, and everything Rust does that Python cannot — no GC, no data races, deterministic cleanup — falls out of it.

Notice, too, the compiler's parting suggestion: `a.clone()`. It has read your mind — or at least your options. We'll take it up on that shortly.

Scope is the destructor

If every value has exactly one owner, then "who frees this?" has a mechanical answer: the owner does, at the moment it goes out of scope. Rust calls this **dropping** the value, and it is as deterministic as the closing brace it happens at.

```
# Figure 3: Values die at the closing brace - every time, on time

fn main() {
    {
        let cmd = String::from("MUL 7 6");
        println!("inside the scope: {cmd}");
    }   // <- cmd's owner goes out of scope RIGHT HERE.
        //    The String is dropped, its memory freed, before
        //    the next line runs. No collector. No "eventually."
    println!("after the scope");
}
```

```
--
inside the scope: MUL 7 6
after the scope
```

The output is unremarkable; the guarantee is not. That string's memory was returned *at the brace* — not at the next garbage-collection pause, not when a reference count happened to hit zero, not "at interpreter shutdown, probably." If you have ever tried to close a file, flush a log, or release a queue in a Python `__del__` method, you know what that "probably" costs: the language reference makes carefully hedged promises about when `__del__` runs, and none at all in some shutdown cases.[2] Rust replaces the hedging with a rule you can point to in the source code. When we reach Part IV, this rule will be doing serious work — an objection guard that drops its objection at the closing brace, driver tasks whose cleanup runs the instant they're cancelled — but the whole mechanism is already in front of you in figure 3. Deterministic destruction isn't a feature bolted onto ownership. It *is* ownership, viewed from the value's last moment.

[2] The Python language reference notes that it is "not guaranteed" that `__del__` will be called for objects still alive when the interpreter exits — one of the great load-bearing "not guaranteed"s of our time.

Wait — my integers have been fine

A fair objection: you've been assigning integers back and forth since Chapter 3 without the compiler saying a word about moves. Figure 4 confirms it.

```
# Figure 4: Copy types don't move - small values are simply copied

fn main() {
    let a: u8 = 5;
    let b = a;              // copies the byte; a is still alive
    println!("a = {a}, b = {b}");
}
```

```
--
a = 5, b = 5
```

The error message in figure 2 quietly explained why: the string moved *"because `a` has type `String`, which*

does not implement the `Copy` trait." Types whose values are small, fixed-size, and self-contained — `u8`, `u16`, `bool`, the TinyALU's operands, all the scalars from Chapter 3 — are `Copy` types: assignment duplicates the bits, both variables live, nothing to negotiate. There is no shared object for two names to fight over, so ownership has nothing to protect. (Traits, including how a type comes to be `Copy`, are Chapter 10's business.)

Move semantics governs everything else: `String`, the collections coming in Chapter 8, and — most importantly for us — the structs we build ourselves. Which brings us to the reason this chapter exists.

The monitor and the scoreboard

Every mechanism in this chapter has been abstract enough to shrug at. So let's make it a testbench problem — *the* testbench problem, the one you've written a dozen times: a monitor observes a transaction on the TinyALU's command bus and hands it to a scoreboard.

We need a transaction type. Structs get their own chapter (Chapter 7); for today, read the first four lines of figure 5 as "a Python class with only data and no methods" — fields, types, no ceremony. The `op` field really wants to be a proper enum, and in Chapter 7 it becomes one; a `u8` stands in for now. And we need the two parties: a `scoreboard` function that takes a `Transaction` *by value*, and a `main` that plays the monitor. In pyuvm these would be components connected by an analysis port; here in our cargo playground, two functions are enough to expose the question that matters.

```
# Figure 5: The monitor hands off a transaction - and learns what "hands off" means

struct Transaction {
    a: u8,
    b: u8,
    op: u8,
}

fn scoreboard(t: Transaction) {
    println!("scoreboard checking: {} op {} (code {})", t.a, t.b, t.op);
}   // <- t dropped here: the scoreboard owned it, the scoreboard's
    //    scope ends, the transaction is destroyed. Question answered.

fn main() {
    // main is playing the monitor today.
    let t = Transaction { a: 5, b: 3, op: 1 };
    scoreboard(t);
    println!("monitor logging: a was {}", t.a);
}
```

```
--
error[E0382]: borrow of moved value: `t`
  --> src/main.rs:15:44
   |
13 |     let t = Transaction { a: 5, b: 3, op: 1 };
   |         - move occurs because `t` has type `Transaction`, which
   |           does not implement the `Copy` trait
14 |     scoreboard(t);
   |                - value moved here
15 |     println!("monitor logging: a was {}", t.a);
   |                                           ^^^ value borrowed here after move
   |
note: consider changing this parameter type in function `scoreboard` to
```

```
      borrow instead if owning the value isn't necessary
help: consider cloning the value if the performance cost is acceptable
```

Passing a value to a function moves it, exactly as assignment did — `scoreboard(t)` is a baton pass, and line 15 is the monitor trying to run the next leg without the baton.

Here is what I want you to see: **the compiler is not reporting a syntax mistake. It is asking you a design question.** When the monitor hands this transaction to the scoreboard, what *should* happen? In Python you never had to decide. The analysis port handed every subscriber the same name tag on the same object, ownership belonged to nobody, and the design question got answered by accident — which worked fine until one subscriber mutated the transaction another was still reading, a bug the Python book could only warn you about. Rust makes you answer on purpose, and there are exactly two honest answers.

Answer one: it's a true handoff. The monitor's job was to observe the transaction and pass it on; it has no business touching it afterward. Then the code is wrong and the compiler is right — delete line 15, and the program compiles. Ownership flows monitor → scoreboard, the scoreboard checks it, and when the scoreboard's scope ends the transaction is dropped, on time, by its one owner. The comment on `scoreboard`'s closing brace in figure 5 is the answer to this chapter's title question, sitting in plain sight.

Answer two: the monitor still needs it — to log it, to hand a second copy to a coverage collector. Then the monitor must keep *a real copy*, and Rust has an explicit word for that.

`.clone()`: copies you can see

The compiler suggested it twice, so let's take the hint. Adding `#[derive(Clone)]` above the struct asks the compiler to write a field-by-field copy routine for us (that one magic line gets a full explanation in Chapters 10 and 21; for now, "please generate the copying code" is the whole story). Then `.clone()` makes an independent duplicate wherever we ask.

```
# Figure 6: The monitor keeps a copy - explicitly

#[derive(Clone)]
struct Transaction {
    a: u8,
    b: u8,
    op: u8,
}

fn scoreboard(t: Transaction) {
    println!("scoreboard checking: {} op {} (code {})", t.a, t.b, t.op);
}

fn main() {
    let t = Transaction { a: 5, b: 3, op: 1 };
    scoreboard(t.clone());   // the scoreboard owns the copy...
    println!("monitor logging: a was {}", t.a);   // ...the monitor owns the original
}
```

```
--
scoreboard checking: 5 op 3 (code 1)
monitor logging: a was 5
```

Two transactions now exist, each with exactly one owner, each dropped at its own owner's closing brace. No sharing, no aliasing, no way for the scoreboard's copy and the monitor's original to interfere. And crucially: mutation of one can never surprise the other — the "two subscribers, one mutated object" bug is not merely discouraged, it is unrepresentable in this code.

It's worth pausing on why Rust makes you *spell out* the copy, because Python's policy here was genuinely confusing and you may have stopped noticing. Python copies silently for some things and never for others: rebind an `int` and you get value-like behavior; assign a list or an object and you get an alias, and if you wanted a real copy you had to know to reach for `copy.deepcopy` — knowledge usually acquired from a bug. Rust's policy fits in one breath: trivial bit-copies (`Copy` types) are silent because they cannot matter; every copy that allocates or duplicates real state is written `.clone()`, visible in the diff, greppable in the review. When a testbench is cloning a million transactions a second, you can *find every clone* and decide whether each one earns its cost. In Python the copies — and the accidental non-copies — were invisible either way.

One more note while the handoff is fresh. When we meet channels in Part II, you'll find that sending a transaction to another component takes it by value — a send *is* a move, the monitor-to-scoreboard baton pass made into infrastructure. The decision you just made line-by-line (hand it off, or clone and keep one?) is the same decision you'll make at every analysis port in Part IV, and the compiler will hold you to your answer every time.

The mental model

There is one thread left hanging, and the compiler dangled it in figure 5's output on purpose: *"consider changing this parameter type in function `scoreboard` to borrow instead if owning the value isn't necessary."* Because — be honest — does a scoreboard need to *own* the transaction? It needs to read the fields, compare against a prediction, and be done. Taking ownership just to look at something is like requiring the title to a car in order to check its odometer. Rust has a mechanism for looking without taking, written `&`, and it is called borrowing; it is the whole subject of Chapter 6, and I will not steal that chapter's material beyond telling you it exists.

What I will do is hand you the sentence this book will use, from here to testbench 8.0, whenever an API makes you choose between taking a value and referencing it. Say it with me, and keep it:

> **Ownership is about responsibility. Borrowing is about access.**
>
> Own a value when you are responsible for it — for its storage, its lifetime, its eventual destruction, its answer to "who frees this?" Borrow a value when you merely need access to it for a while — to read it, or briefly change it — while responsibility stays exactly where it was.

Every design decision ahead of us is an application of that sentence. The component hierarchy in Part IV works because parents *own* their children — responsibility flows down the tree. Channels move transactions because a handoff transfers *responsibility*. Monitors will hand scoreboards references or clones depending on whether the scoreboard needs *access* or needs to *keep* something. When you're stuck on a compiler error anywhere in this book, ask the sentence's question first: does this code need responsibility for the value, or just access to it? The answer usually types itself.

You now hold half the model — the responsibility half. Chapter 6 supplies the access half: `&`, the borrow checker Chapter 1 warned you about, and the rule that lets Rust catch at compile time a race condition the Python book could only teach you to fear.

Chapter 6: Borrowing and References

> **In Python we...** got a warning instead of a rule. The Python book's "Coroutines" chapter examined `NullTrigger` and the temptation it represents: two tasks — a monitor and a consumer — sharing a `transaction_data` variable, both waking on the same `RisingEdge`, with no guarantee about which runs first. The consumer awaited a `NullTrigger` to push itself "later," and the book (echoing cocotb's own documentation) told you flatly not to do this — the scheduling order "is not deterministic and should generally not be relied upon" — and to synchronize with an `Event` instead. Python could warn you. It could not stop you. This chapter is about the language feature that stops you. *(book: "Coroutines" chapter, NullTrigger discussion; cocotb: `_base_triggers.py`, `NullTrigger` docstring)*

Chapter 5 ended with a mental model we will now spend a whole chapter earning: *ownership is about responsibility, borrowing is about access.* Ownership answered the question Python's garbage collector answered silently — who frees this transaction? — by declaring exactly one owner, and by making assignment a *move* of that responsibility. When the monitor handed a transaction to the scoreboard, the monitor was done with it, and the compiler enforced its being done with it.

But that model, taken alone, is unlivable. A scoreboard that must *own* every transaction it merely wants to *look at* would be a scoreboard fed entirely by `clone()` calls. A coverage collector that steals the transaction from the scoreboard is not a testbench, it is a relay race. Most of the time a component does not need responsibility for a value. It needs access. Rust's mechanism for access-without-responsibility is the **reference**, and the act of granting one is called **borrowing** — a word chosen carefully, because a borrow, unlike a Python reference, comes with an obligation to give the value back and rules about what you may do while you hold it.

Shared references: many readers

A shared reference is written `&T` — "a reference to a T" — and you create one with `&`:

```
# Figure 1: Shared references - everyone may look, nobody may touch

struct Transaction {
    data: u8,
}

fn report(t: &Transaction) {
    println!("Saw transaction with data {}", t.data);
}

fn main() {
    let t = Transaction { data: 42 };

    let monitor_view = &t;    // a borrow
    let coverage_view = &t;   // another borrow -- readers may alias freely
```

```
    report(monitor_view);
    report(coverage_view);

    println!("Still the owner: {}", t.data);   // t was never moved
}
```

```
--
Saw transaction with data 42
Saw transaction with data 42
Still the owner: 42
```

(I am reusing the `Transaction` struct from Chapter 5. Structs get their full treatment in Chapter 7; for now it is a labeled box holding a `data` field.)

Three things to notice, each a quiet contrast with Chapter 5. First, `report` takes `&Transaction`, not `Transaction` — so calling it does not move `t`. The function borrows the transaction, reads it, and the borrow ends when the function returns. Second, we made *two* references to `t` at the same time and the compiler did not object: shared references may alias freely, any number of them at once. Third, after all that lending, `t` is still ours to use in the final `println!`. Responsibility never changed hands; only access did.

The price of this generosity is stamped into the type: through a `&T` you may *read* and nothing else. Try `monitor_view.data = 7` and the compiler will refuse — not because `t` is immutable (though it is; we never said `let mut`), but because a shared reference never grants write access, period. Everyone may look. Nobody may touch.

If you want a Python analogy, `&T` is what every Python reference *pretended* to be in a well-disciplined codebase: a handle you pass around for reading, trusting that nobody mutates through it. Rust removes the trust and keeps the handle.

Exclusive references: one writer

Sometimes touching is the point. A driver that receives a transaction may legitimately need to fill in a timestamp; a BFM may need to update a field before sending. For write access you need the second kind of reference, `&mut T` — an **exclusive reference**:

```
# Figure 2: An exclusive reference grants write access

struct Transaction {
    data: u8,
}

fn scramble(t: &mut Transaction) {
    t.data = 99;
}

fn main() {
    let mut t = Transaction { data: 42 };   // mut: the owner permits mutation

    scramble(&mut t);   // lend write access, briefly

    println!("After scramble: {}", t.data);
}
```

```
--
After scramble: 99
```

Two spellings of `mut` appear here and they are doing different jobs. `let mut t` is the owner declaring that this value may ever be mutated at all — Chapter 3's immutable-by-default rule. `&mut t` is the owner lending *exclusive, writable* access to somebody else. You cannot create a `&mut` borrow of a variable that was not declared `mut`; permission flows downhill from the owner.

"Exclusive" is the load-bearing word. While a `&mut T` exists, it is the *only* live reference to that value — no other `&mut`, no `&`, and the owner itself may not so much as read the value until the exclusive borrow ends. The writer works alone, with the door locked.

The rule: aliasing XOR mutability

We can now state the rule the whole chapter — arguably the whole language — hangs on. Memorize it in this form:

> **At any moment, a value may have many readers or one writer — never both.**

Rust programmers call this **aliasing XOR mutability**: a value may be aliased (many `&T`), or it may be mutable (one `&mut T`), but never both at once.[1] Everything the borrow checker does is enforcement of this one sentence, plus bookkeeping about *when* each borrow ends.

Why is this the rule worth building a language around? Because every data race you have ever debugged — and, per the sidebar, every data race the Python book warned you about — has the same anatomy: one piece of state, at least one writer, at least one other reader or writer, and no enforced ordering between them. Aliasing XOR mutability makes that anatomy unrepresentable. If there is a writer, there is nobody else; if there is anybody else, there is no writer. The race has no room to exist.

That is a large claim, so let us go back to the scene of the crime.

[1] The rule has the same shape as a conference-room whiteboard policy: any number of people may stand and read it, or exactly one person may hold the marker — but a reader standing at a whiteboard while someone else is erasing it learns nothing trustworthy, and everybody knows it.

The NullTrigger race, rejected at compile time

Recall the pattern the Python book told you never to write: a shared `transaction_data` variable, a monitor task that writes it, a consumer task that reads it, both triggered by the same clock edge, with a `NullTrigger` deployed as a prayer that the reader runs second. In Python, that code runs. It even works, usually, on your simulator, until the scheduler's ordering shifts and it silently doesn't.

We cannot yet write real concurrent tasks in Rust — the executor and `spawn` arrive in Part II — but we do not need them to expose the bug, because the bug was never really about tasks. It was about *two live accessors of one value, one of them a writer, with no enforced order*. We can write exactly that in six lines, giving the monitor its write access and the scoreboard its read access, just as the Python version did:

```
# Figure 3: Two tasks' worth of access to one value -- the borrow checker objects

fn main() {
    let mut transaction_data: Option<u8> = None;

    let monitor = &mut transaction_data;    // the monitor's claim: write access
    let scoreboard = &transaction_data;     // the scoreboard's claim: read access

    *monitor = Some(42);                    // the monitor sees a transaction...

    match scoreboard {                      // ...and the scoreboard checks it
        Some(data) => println!("Scoreboard checking {data}"),
        None => println!("Nothing to check yet"),
```

```
    }
}
```

```
--
error[E0502]: cannot borrow `transaction_data` as immutable because it is also borrowed as mutable
 --> src/main.rs:5:22
  |
4 |     let monitor = &mut transaction_data;
  |                   ---------------------- mutable borrow occurs here
5 |     let scoreboard = &transaction_data;
  |                      ^^^^^^^^^^^^^^^^^ immutable borrow occurs here
...
7 |     *monitor = Some(42);
  |     ------------------- mutable borrow later used here

For more information about this error, try `rustc --explain E0502`.
error: could not compile `borrow_race` (bin "borrow_race") due to 1 previous error
```

(`Option<u8>` is Rust's typed replacement for the Python version's `None`-or-value idiom — read `Some(42)` as "has a value" and `None` as "doesn't." Chapter 9 gives `Option` its full due; here it is set dressing.)

Read the error the way Chapter 2 taught you to — as a collaborator's comment, not a rejection slip. The compiler identifies all three actors in the crime: where the mutable borrow was created (line 4), where the immutable borrow was created while the writer still existed (line 5 — that is the actual error, marked with carets), and — this is the part no Python tool could ever tell you — where the writer is *used again afterward* (line 7), proving the two claims genuinely overlap in time. Writer alive, reader alive, same value, same moment: aliasing AND mutability. The program does not compile.

Sit with what just happened. The Python book documented this bug, explained why it was nondeterministic, and taught you a discipline to avoid it. cocotb's own docstring says **Do not do this** in bold. And still, nothing in the Python toolchain would stop a tired engineer from writing it on a Friday afternoon, and nothing would flag it until a scheduler reordering made a regression flicker, weeks later, in someone else's test. Rust converts the whole affair into three seconds of compile time and an error message with line numbers. Chapter 1 promised that discipline is good but proof is better; Figure 3 is that promise kept.

The fix: make the ordering real

The Python fix was an `Event`: the monitor writes, *then* sets the event; the consumer awaits the event, *then* reads. Notice what the `Event` actually contributed — it forced the write and the read into a definite order, so that the writer was finished before the reader began. The borrow checker demands precisely the same thing, and in straight-line code you provide it the same way: sequence the accesses so they do not overlap.

```
# Figure 4: The same actors, with the ordering made real

fn main() {
    let mut transaction_data: Option<u8> = None;

    let monitor = &mut transaction_data;
    *monitor = Some(42);                    // the monitor writes...
                                            // ...and its borrow ends at its last use

    let scoreboard = &transaction_data;     // now the reader may claim access
    match scoreboard {
        Some(data) => println!("Scoreboard checking {data}"),
        None => println!("Nothing to check yet"),
```

```
    }
}
```

```
--
Scoreboard checking 42
```

The only change from Figure 3 is the order of the lines — the monitor finishes its writing *before* the scoreboard's borrow begins — and that is the entire point. The fix for an unordered write/read conflict is an ordering, in Rust as in Python; the difference is that Rust would not let you skip it.

One subtlety in Figure 4 deserves a sentence, because it will save you fights with the compiler later: `monitor`'s borrow ended at its *last use* (the write on the line above), not at the closing brace of its scope. The borrow checker tracks how long each borrow is actually needed, not merely where the variable was declared. If it worked scope-to-brace, Figure 4 would not compile either; because it works use-to-use, tidy sequential code like this passes without ceremony.

And one honesty note, so you do not over-generalize: when the monitor and scoreboard become genuinely concurrent tasks in Part II, they cannot simply take turns borrowing a local variable — each task needs its own durable handle to shared state, which is exactly the situation `&`/`&mut` alone cannot express. Rust's answers there are the channel (the two tasks never share the value at all — Chapter 16) and, when sharing truly is the design, the `Rc<RefCell<T>>` escape hatch, which moves this chapter's rule from compile time to runtime checking (Chapter 13). Both of those tools are built on top of the rule you just learned, not exemptions from it. The rule is the constant; only the enforcement point moves.

Lifetimes: recognize the syntax, decline the rabbit hole

There is one more piece of borrowing syntax you will meet in the wild, and I want you to be able to greet it without alarm. Sometimes a function signature carries a small tick-marked annotation:

```
fn newer<'a>(x: &'a Transaction, y: &'a Transaction) -> &'a Transaction
```

That `'a` (pronounced "tick-a") is a **lifetime** — a name for *how long a borrow lasts*. Every reference in every program has one; the compiler has been inferring them silently in every figure of this chapter. They surface in the syntax only when the compiler needs your help connecting inputs to outputs: `newer` returns a reference, and the compiler must know whether that returned borrow is tied to `x`, to `y`, or to something else, because whoever *receives* the returned reference is now borrowing from one of them, and the checker must know which value has to stay alive. Writing `'a` on all three says: the returned reference lives no longer than the shorter-lived of the two arguments. That is the whole idea — lifetimes are the paperwork that lets borrow checking work *across* function boundaries.

Here is what you need at this point in the book, in full: recognize `'a` as "a named lifetime," know that it connects the lifetime of an output reference to the lifetimes of input references, and know that when the compiler asks you for one, it is asking "if I hand this reference back to your caller, which argument is it borrowing from?" You will see lifetimes in error messages and in library documentation; you will write them rarely, because the compiler's inference rules cover the overwhelming majority of testbench-shaped code, and rustvm's public API is deliberately designed to keep user-facing lifetimes rare besides.

And here I am going to make the same call the Python book made about metaclasses. That book taught you every Python feature a testbench needed and then, at the door of the metaclass, stopped — noting that pyuvm used them internally but that a testbench author never needed to write one. Lifetimes get the same treatment here, for the same reason. There is real depth behind that tick mark — variance, higher-ranked bounds, a small literature of compiler lore — and none of it, not one line, appears in the testbenches this book builds. When a lifetime annotation is forced on us later, I will explain that occurrence on the spot. Until then: recognize it, read past it, keep going.[2]

[2] Rust folklore holds that you do not truly understand lifetimes until you have argued with the borrow checker about them for a weekend. This book's position is that your weekends belong to

your regressions.

What borrowing buys the testbench

Step back and look at the shape of what you now know. Ownership (Chapter 5) decides who is responsible for every value — who frees the transaction, in a world with no garbage collector. Borrowing (this chapter) decides who may access it meanwhile, under a rule — many readers or one writer, never both — that makes the shared-state race from the Python book's warning sidebar into a compile error with line numbers. Together they are the machinery behind Chapter 1's boldest claim: that a whole class of 2 a.m. testbench bugs becomes a red squiggle at your desk before lunch.

So far, though, our transactions have been a struct with one sad little `u8` in it, and our operations have been strings of field pokes. A real TinyALU transaction has two operands and an *operation* — and "an operation" is a value that is exactly one of ADD, AND, XOR, or MUL, which is a kind of type Python never quite had and Rust does beautifully. Structs, methods, and enums with payloads are Chapter 7, and they are where Rust starts being fun.

Chapter 7: Structs, Enums, and Methods

The Python book opened its Classes chapter with a claim: object-oriented programming is the foundation of the expandability and reusability of UVM-based verification. That claim survives the trip to Rust intact. What does not survive is the machinery. Rust has no `class` statement, no `self` argument sliding invisibly into every method, no `__init__`, and no ability to bolt a data attribute onto an object whenever the mood strikes. In their place it offers two building blocks — structs and enums — and a place to hang behavior on them, the `impl` block. By the end of this chapter you will have both, plus the chapter's real payoff: Rust enums, which are so much more capable than Python's that they will quietly restructure how you think about testbench data.

> **In Python we...** created an object and added data attributes to it on the fly: `walrus = Animal()` followed by `walrus.kg = 1000`, no declaration anywhere. The Classes chapter called this "nothing like SystemVerilog, where the Animal class would have to define the kg variable at definition" — and it warned that the freedom cuts both ways, showing a `yorkie` whose forgotten `kg` attribute surfaced as a runtime `AttributeError`. Later, in *Basic testbench: 1.0*, we met `tinyalu_utils` and its `Ops(enum.IntEnum)`: `ADD = 1, AND = 2, XOR = 3, MUL = 4` — names mapped to opcode integers. Both of those patterns return in this chapter, and both come back changed.

Structs: the data, declared

A Rust struct is the part of a Python class that holds data — and only that part. You declare every field, with its type, up front:

```
# Figure 1: Defining and instantiating a struct

struct Animal {
    kg: f64,
}

fn main() {
    let walrus = Animal { kg: 1000.0 };
    println!("Walrus mass: {}", walrus.kg);
}
```

```
--
Walrus mass: 1000
```

Two things deserve a look. First, the declaration reads more like the SystemVerilog `Animal` from the Python book's figure 3 than like anything Python ever asked of us: the fields are part of the definition, not something we improvise later. Second, instantiation uses a *struct literal* — `Animal { kg: 1000.0 }` — which names

every field and supplies every value. There is no separate "create empty, then populate" step, because an empty `Animal` is not a thing Rust permits to exist.

That last point is not a style preference; it is enforcement. The Python book's figure 7 created a `yorkie`, forgot to set `kg`, and got an `AttributeError` — at runtime, at the moment of use, which in a testbench means mid-simulation. Here is the same mistake in Rust:

```
# Figure 2: Forgetting a field is now a compile error

struct Animal {
    kg: f64,
}

fn main() {
    let yorkie = Animal {};
    println!("Yorkie mass: {}", yorkie.kg);
}
```

```
--
error[E0063]: missing field `kg` in initializer of `Animal`
 --> src/main.rs:6:18
  |
6 |     let yorkie = Animal {};
  |                  ^^^^^^ missing `kg`
```

The program never ran. There is no such thing as an `Animal` with an undefined mass, so the class of bug the Python book had to warn you about is not a bug you can write.[1] This is the pattern of the whole chapter — of the whole book, really: where Python offered a convention and a warning, Rust offers a rule and a compile error.

One freedom is genuinely gone: you cannot add a field to an object after the fact. `walrus.age = 12` on a struct with no `age` field does not compile. Every field a transaction will ever carry is visible in one place, in its definition, forever. For quick scripts this feels confining. For a transaction type that five components and three engineers share, it is exactly what you want.

[1] The walrus, having survived one book already, takes this in stride.

`impl` blocks: where behavior lives

A Python class bundles data and behavior in one indented block. Rust separates them: the struct declares the data, and an `impl` block — short for *implementation* — attaches the behavior. Here is `get_pounds()`, ported:

```
# Figure 3: A method in an impl block

struct Animal {
    kg: f64,
}

impl Animal {
    fn get_pounds(&self) -> f64 {
        self.kg / 2.2
    }
}

fn main() {
    let walrus = Animal { kg: 1000.0 };
```

```
    println!("Walrus weight in pounds {:.2}", walrus.get_pounds());
}
```

```
--
Walrus weight in pounds 454.55
```

The call site is pure Python: `walrus.get_pounds()`. The definition is where the languages diverge, and the divergence is instructive.

Python's `self` is an ordinary argument that the interpreter fills in for you — the Classes chapter demonstrated this by calling `Animal.get_pounds(walrus)` explicitly, passing the object by hand. Rust's `&self` is doing the same job, but it carries more information: that ampersand says this method *borrows* the animal, read-only, exactly as Chapter 6 taught. The method can look at `self.kg`; it cannot modify it, and it does not take ownership of the walrus. The signature tells you the method's relationship to the object before you read a line of the body.

And yes, the explicit form still works: `Animal::get_pounds(&walrus)` is legal Rust and does exactly what `Animal.get_pounds(walrus)` did in Python's figure 5. What Python offered as a party trick, Rust treats as the ordinary meaning that the dot syntax abbreviates.

When a method needs to *change* the object, it says so:

```
# Figure 4: A method that mutates takes &mut self

struct Animal {
    kg: f64,
}

impl Animal {
    fn feed(&mut self, meal_kg: f64) {
        self.kg += meal_kg;
    }
}

fn main() {
    let mut walrus = Animal { kg: 1000.0 };
    walrus.feed(3.5);
    println!("Walrus mass after lunch: {}", walrus.kg);
}
```

```
--
Walrus mass after lunch: 1003.5
```

Notice `let mut walrus`. A method that takes `&mut self` can only be called on a mutable binding — immutability by default, as in Chapter 3, extends all the way into method calls. Skim any `impl` block and the `&self`/`&mut self` split hands you a map of which methods observe and which methods mutate. In Python, discovering that a method quietly modified your transaction was an afternoon with the debugger; here it is a fact printed in the signature. When we build monitors and scoreboards, this distinction stops being philosophy: a scoreboard's check wants `&self`, its update wants `&mut self`, and the compiler holds every caller to it.

Associated functions: `new()` and the end of `__init__`

The Python book introduced `__init__()` as the standard way to force initialization — a magic method, called implicitly when you invoke the class name. Rust has no magic methods and no implicit calls. What it has is the *associated function*: a function in an `impl` block that takes no `self` at all, called through the type name with `::`.

By near-universal convention, the constructor is an associated function named `new`:

```
# Figure 5: The new() associated function

struct Animal {
    kg: f64,
}

impl Animal {
    fn new(kg: f64) -> Self {
        Self { kg }
    }

    fn get_pounds(&self) -> f64 {
        self.kg / 2.2
    }
}

fn main() {
    let yorkie = Animal::new(20.0);
    println!("The Yorkie weighs {:.1} pounds", yorkie.get_pounds());
}
```

```
--
The Yorkie weighs 9.1 pounds
```

This is the Python book's figure 8, translated. A few notes on the translation:

- `Self` (capital S) is shorthand for "the type this `impl` block belongs to" — here, `Animal`. Writing `-> Self` and `Self { kg }` means the code survives a rename.
- `Self { kg }` uses *field init shorthand*: when a variable and a field share a name, you may write the name once instead of `kg: kg`.
- There is nothing special about `new`. It is not a keyword, the language does not call it for you, and you could name it `hatch()` if you enjoyed confusing people. The compiler's only contribution is the guarantee from figure 2: however an `Animal` gets built, every field gets a value.

The Python book sorted methods into three kinds — instance methods, class methods, and static methods — with a decorator apiece. Rust flattens the taxonomy to two: if it takes `self` in some form, it is a *method*, called with a dot; if it does not, it is an *associated function*, called with `::`. Python's `@staticmethod` and `@classmethod` both collapse into the second kind, and the class-variable pattern comes along as an *associated constant*:

```
# Figure 6: Associated constants and functions replace class variables and static methods

struct Triangle;

impl Triangle {
    const SIDE_COUNT: u32 = 3;

    fn print_side_count() {
        println!("I have {} sides.", Self::SIDE_COUNT);
    }
}

fn main() {
    Triangle::print_side_count();
}
```

```
--
I have 3 sides.
```

(`struct Triangle;` with no braces is a *unit struct* — a type with no data, which is all our triangle ever needed.) The Python book gave two reasons to keep a static method inside a class: a logical home, and a consistent calling style. Both reasons apply verbatim here; the `impl` block *is* the logical home, and `Triangle::print_side_count()` is the consistent style.

Enums: the star of the chapter

Now for the construct that earns this chapter its place in the book.

The Python book introduced `Ops` in `tinyalu_utils` as an `enum.IntEnum` — a class that "not only names the operations but also maps the names to the correct opcode integer." That was the best Python could do: an enum there is a set of named constants, each secretly an integer wearing a name tag. Rust enums start at that point and keep going.

Here is `Ops`, ported, together with the prediction function it exists to serve:

```rust
# Figure 7: The Ops enumeration and an exhaustive match

#[derive(Clone, Copy, Debug, PartialEq)]
enum Ops {
    Add = 1,
    And = 2,
    Xor = 3,
    Mul = 4,
}

fn alu_prediction(a: u8, b: u8, op: Ops) -> u16 {
    match op {
        Ops::Add => a as u16 + b as u16,
        Ops::And => (a & b) as u16,
        Ops::Xor => (a ^ b) as u16,
        Ops::Mul => a as u16 * b as u16,
    }
}

fn main() {
    let op = Ops::Add;
    println!("{:?} of 0x0A and 0x05 -> 0x{:04x}", op, alu_prediction(0x0A, 0x05, op));
}
```

```
--
Add of 0x0A and 0x05 -> 0x000f
```

Working through the new pieces:

The `derive` line. `#[derive(Clone, Copy, Debug, PartialEq)]` asks the compiler to generate standard capabilities: copying, comparison with `==`, and the `{:?}` debug printing you see in `main`. These are *traits*, and Chapter 10 gives them their due; for now, read the line as "make this type behave like a sensible value." IntEnum gave Python's `Ops` comparison and printing for free by inheritance; the derive line is where Rust's version of "for free" lives.

The discriminants. `Add = 1` assigns the variant an integer value, just as the IntEnum did, and `Ops::Add as u8` recovers it when the opcode has to go onto a bus. But note which way the equivalence runs. An IntEnum member *is* an int — you can add three to `Ops.ADD` and Python will let you. `Ops::Add` is not a

number that happens to have a name; it is a value of type `Ops`, and arithmetic on it is a compile error. The integer is available on request, one direction only. (Going the other way — from a raw integer read off a bus back to an `Ops` — takes explicit code that must confront the possibility of an illegal opcode. Chapter 9's `Result` is built for exactly that confrontation.)

The `match`. Chapter 4 introduced `match` as the load-bearing construct; here is the load. A `match` on an enum must be *exhaustive*: every variant handled, or the code does not compile. The prediction function cannot silently do nothing for `Mul` the way a Python `if/elif` chain with no `else` can.

That guarantee sounds abstract until the day it saves you. Suppose the TinyALU grows a subtract instruction. In Python you would add `SUB = 5` to the IntEnum, and every `if/elif` over ops in the testbench — the prediction function, the coverage model, the driver — would keep running, silently wrong, until a failing test (or worse, a passing one) sent you hunting. Watch what happens in Rust the moment we add the variant and change nothing else:

```
# Figure 8: The compiler finds every match the new variant breaks

#[derive(Clone, Copy, Debug, PartialEq)]
enum Ops {
    Add = 1,
    And = 2,
    Xor = 3,
    Mul = 4,
    Sub = 5,   // the new operation - and the only edit we made
}
```

```
--
error[E0004]: non-exhaustive patterns: `Ops::Sub` not covered
  --> src/main.rs:11:11
   |
11 |     match op {
   |           ^^ pattern `Ops::Sub` not covered
   |
note: `Ops` defined here
   = note: the matched value is of type `Ops`
help: ensure that all possible cases are being handled by
      adding a match arm with an explicit pattern
```

Stop and appreciate what just happened, because it is the concrete version of this book's central promise. We changed the specification — one line — and the compiler produced a complete list of every place in the testbench that has not yet heard the news, before any simulator license was checked out, before any simulation ran, before any test could pass for the wrong reason. In the Python flow, this bug costs a simulation run at minimum and a shipped escape at maximum. Here it costs one compile, a few seconds, and it *cannot* be skipped. This is the refactoring-without-fear argument from Chapter 1, delivered.[2]

[2] There is an escape hatch — a `_ => ...` wildcard arm matches everything not yet named, and using one forfeits this protection. The idiom this book follows: never use a wildcard when matching an enum you own. Spend the extra lines; they are the tripwire.

Four states, no integers: `Logic`

Every value in the Python testbench was ultimately an integer — `get_int()` existed precisely to coerce simulator values that might contain `x` or `z` into something Python could compute with. But hardware signals are not integers. They are four-state values, and Rust lets us say so directly:

```
# Figure 9: A four-state Logic enum
```

```rust
#[derive(Clone, Copy, Debug, PartialEq)]
enum Logic {
    Zero,
    One,
    X,
    Z,
}

fn to_char(v: Logic) -> char {
    match v {
        Logic::Zero => '0',
        Logic::One => '1',
        Logic::X => 'x',
        Logic::Z => 'z',
    }
}

fn main() {
    let bit = Logic::X;
    println!("The signal reads: {}", to_char(bit));
}
```

```
--
The signal reads: x
```

`Logic` is not a teaching toy: when we reach rustvm-sim in Chapter 17, reading a signal hands you exactly this type, ported from the same four-state value type cocotb defines. Notice what the enum buys us that an IntEnum encoding (say, `X = 2`) never could: there is no integer pretense to leak. Nothing can accidentally add `X` to a running sum, because `X` is not a number — it is one of four states a wire can be in, and any code that consumes a `Logic` must, thanks to exhaustive `match`, say what it does about `x` and `z`. The "forgot to handle the unknown state" bug is unrepresentable.

Variants that carry data

Everything so far, an IntEnum could at least gesture at. This last capability it could not. Rust enum variants can *carry data* — different data per variant — which makes an enum a type that says "this value is exactly one of the following shapes." Computer scientists call this a *sum type*; testbench authors will call it the right way to model outcomes:

```rust
# Figure 10: An enum whose variants carry payloads

#[derive(Debug)]
enum CheckResult {
    Pass,
    Fail { expected: u16, actual: u16 },
}

fn check(expected: u16, actual: u16) -> CheckResult {
    if expected == actual {
        CheckResult::Pass
    } else {
        CheckResult::Fail { expected, actual }
    }
}
```

```rust
fn main() {
    let result = check(0x000f, 0x0005);
    match result {
        CheckResult::Pass => println!("PASS"),
        CheckResult::Fail { expected, actual } => {
            println!("FAIL: expected 0x{expected:04x}, got 0x{actual:04x}")
        }
    }
}
```

```
--
FAIL: expected 0x000f, got 0x0005
```

Read the `match` arms closely, because a new thing is happening: the second arm doesn't just select the `Fail` variant, it *unpacks* it, binding `expected` and `actual` right there in the pattern. A passing check carries nothing; a failing check carries the evidence. In Python we would have modeled this with a boolean plus a couple of variables that are only meaningful when the boolean is `False` — a convention, enforced by hope. The enum makes the convention structural: the payload *exists* only in the variant it belongs to, and the only way to touch it is to admit, in a pattern, which case you are in.

Once you see this shape, you will see it everywhere in Rust, because the standard library's two most important types are exactly this pattern: `Option<T>` is an enum whose variants are `Some(T)` and `None`, and `Result<T, E>` is an enum whose variants are `Ok(T)` and `Err(E)`. Absence and failure, modeled as payload-carrying enums, matched exhaustively — that is the entire story of how Rust replaced both `None`-checking and exceptions, and it gets Chapter 9 to itself.

Summary

Rust splits Python's class into parts and makes each part explicit. A **struct** declares its data completely — every field, every type, no after-the-fact attributes — and the compiler guarantees no instance ever exists with a field unset, retiring the Python book's `AttributeError` demonstration permanently. An **`impl` block** attaches behavior: **methods** take `&self` to observe or `&mut self` to mutate, telling every caller which is which, while **associated functions** take no `self` and answer to the type name — `Animal::new(20.0)` — absorbing the jobs of `__init__`, `@classmethod`, and `@staticmethod` with one mechanism and zero magic.

Enums are the chapter's prize. `Ops` returns from `tinyalu_utils` not as named integers but as a true sum type: exhaustively matched, so that adding a variant produces a compiler-generated to-do list of every site that must change — a testbench bug class eliminated before simulation. `Logic` models a wire's four states without pretending they are numbers. And payload-carrying variants let one type hold differently-shaped alternatives — the pattern behind `Option` and `Result`, and behind more testbench types to come.

Our transactions can now hold data and our enumerations can now hold their own. What we cannot yet do is hold *many* of anything — a queue of commands, a list of results, a scoreboard's worth of expectations. Chapter 8 takes up Rust's collections, where the Python sequences you know are waiting, with ownership along for the ride.

Chapter 8: Collections: Vec, String, and HashMap

> **In Python we...** spent five chapters on sequences and dictionaries. We learned that "Python sequences are iterables that store ordered data and provide the data one at a time," that lists are "Python's workhorse" — mutable, sortable, able to hold anything — that strings are immutable sequences with 45 methods, and that dictionaries "allow us to store a value using a key and retrieve it later," returning their keys in creation order and raising `KeyError` when a key is missing. We counted the letters in `"Mississippi"` with a dictionary and sliced `"abcdefghi"[1:5]` to get `bcde`.

This chapter compresses those five Python chapters into one, and I want to be honest about why: most of what you learned transfers directly. Rust has a growable list (`Vec<T>`), a string type (`String`), and a hash-keyed store (`HashMap<K, V>`). You index with square brackets, you slice with ranges, you loop with `for`, you check membership, you sort. If I walked through every operation the way the Python book did, you would be bored and I would be padding.

So instead this chapter spends its words on the three places where your Python instincts will actively mislead you:

1. A `Vec` *owns* its elements — pushing a value into it is a move, and Chapter 5 comes due.
2. Rust has *two* string types, `String` and `&str`, and until you know which is which, every function signature involving text will look like a typo.
3. Iterating over a collection can either *borrow* it or *consume* it, and the difference is one ampersand.

Everything else — the operations that work the way you expect — gets a fast tour at the end.

Vec: the list that owns its contents

A `Vec<T>` is Rust's growable array: the analog of the Python list, and every bit the workhorse the list was. The first difference is visible in the type: a Python list holds *anything* (`['a', 3, LL]` was a perfectly good list), while a `Vec<T>` holds values of exactly one type `T`. You have met this trade before — it is the same static-typing bargain from Chapter 3, and it is usually what a testbench wanted anyway. A history log of ALU transactions should hold transactions, all of them, and nothing else.

Let's build exactly that. Figure 1 creates a log of the `AluCommand` transactions we defined in Chapter 7 and pushes commands into it, the way a monitor might record everything it sees.[1]

```
# Figure 1: A Vec<AluCommand> as a transaction history log

#[derive(Clone, Copy, Debug, PartialEq)]
enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

#[derive(Clone, Debug, PartialEq)]
struct AluCommand { a: u8, b: u8, op: Ops }
```

```
fn main() {
    let mut log: Vec<AluCommand> = Vec::new();
    log.push(AluCommand { a: 5, b: 3, op: Ops::Add });
    log.push(AluCommand { a: 2, b: 2, op: Ops::Mul });

    println!("{} commands logged", log.len());
    println!("first: {:?}", log[0]);
}
```

```
--
2 commands logged
first: AluCommand { a: 5, b: 3, op: Ops::Add }
```

Familiar territory: `push` is `append`, `len()` is `len()`, `log[0]` is `log[0]`. Note that `log` must be `let mut` — Chapter 3's immutability-by-default applies to collections with no exceptions, which means a testbench data structure cannot be quietly modified by code you didn't expect to modify it. Also note `Vec::new()` gave us an empty vector; the `vec!` macro is the literal syntax, so `vec![1, 2, 3]` is Rust's `[1, 2, 3]`.

Now the part that is genuinely new. In Python, a list holds *references*. When you append a transaction to a list, the list gets one more name for an object that still has all its other names; the monitor keeps its handle, the scoreboard keeps its handle, and the GC sorts out the afterlife. A `Vec` holds *values*. When you push a transaction into a `Vec`, the transaction **moves** into the `Vec` — the vector becomes the owner, exactly as if you had assigned it to a new variable in Chapter 5. Figure 2 shows what happens when we forget.

```
# Figure 2: Pushing is a move

fn main() {
    let mut log: Vec<AluCommand> = Vec::new();
    let cmd = AluCommand { a: 5, b: 3, op: Ops::Add };
    log.push(cmd);
    println!("sent: {:?}", cmd);   // cmd moved into the Vec
}
```

```
--
error[E0382]: borrow of moved value: `cmd`
  --> src/main.rs:12:28
   |
10 |     let cmd = AluCommand { a: 5, b: 3, op: Ops::Add };
   |         --- move occurs because `cmd` has type `AluCommand`,
   |             which does not implement the `Copy` trait
11 |     log.push(cmd);
   |              --- value moved here
12 |     println!("sent: {:?}", cmd);
   |                            ^^^ value borrowed here after move
   |
help: consider cloning the value if the performance cost is acceptable
   |
11 |     log.push(cmd.clone());
   |                 ++++++++
```

Read that error the way Chapter 2 taught you: the compiler names the move, points at the line where it happened, and offers the fix. If you want to log the command *and* keep using it, `push(cmd.clone())` makes the copy explicit — the choice Python's list made for you silently (share the reference) is now a decision you make on the line where it matters. If the log is the transaction's final destination, push the value and let the `Vec` own it. This is the monitor-hands-a-transaction-to-the-scoreboard question from Chapter 5, answered by a container: *whoever holds the `Vec` owns everything in it*, and when the `Vec` is dropped, every transaction

inside is dropped with it. No cycles, no GC, no doubt about who frees what.

One more `Vec` note before we move on: indexing past the end panics, where Python raised `IndexError`. There is also `log.get(7)`, which does not panic but instead returns a value that is either something or nothing — a type called `Option` that keeps appearing in this chapter's corners and gets the full treatment in Chapter 9.

Iteration: borrow or consume, your choice

The `for` loop over a `Vec` looks exactly like Python — and hides the chapter's second lesson in a single character. Figure 3 loops over the log the obvious way and then tries to use it afterward.

```
# Figure 3: A for loop can consume the collection

fn main() {
    let log = vec![
        AluCommand { a: 5, b: 3, op: Ops::Add },
        AluCommand { a: 2, b: 2, op: Ops::Mul },
    ];
    for cmd in log {
        println!("{:?}", cmd);
    }
    println!("{} commands", log.len());   // log is gone
}
```

```
--
error[E0382]: borrow of moved value: `log`
  --> src/main.rs:14:29
   |
9  |     for cmd in log {
   |                --- `log` moved due to this implicit call
   |                    to `.into_iter()`
...
14 |     println!("{} commands", log.len());
   |                             ^^^ value borrowed here after move
   |
help: consider borrowing to avoid moving into the for loop
   |
9  |     for cmd in &log {
   |                +
```

`for cmd in log` *consumes* the vector: ownership of the whole `Vec` moves into the loop, each element moves into `cmd` in turn, and when the loop ends there is nothing left. That is occasionally exactly what you want — a scoreboard draining its queue at end of test, say. But most of the time you want Python's behavior, looking at the elements while leaving the collection intact, and the compiler's `help` line hands you the idiom: borrow it.

```
# Figure 4: Borrowing iteration leaves the Vec intact

fn main() {
    let log = vec![
        AluCommand { a: 5, b: 3, op: Ops::Add },
        AluCommand { a: 2, b: 2, op: Ops::Mul },
    ];
    for cmd in &log {
        println!("{:?}", cmd);
```

```
    }
    println!("{} commands still logged", log.len());
}
```

```
--
AluCommand { a: 5, b: 3, op: Ops::Add }
AluCommand { a: 2, b: 2, op: Ops::Mul }
2 commands still logged
```

With `for cmd in &log`, each `cmd` is a `&AluCommand` — a shared borrow, read-only, governed by Chapter 6's rules. The third form, `for cmd in &mut log`, borrows each element mutably so you can edit in place. The whole story is three spellings:

- `for x in &v` — borrow each element; the loop reads. *This is your Python `for` loop.*
- `for x in &mut v` — mutably borrow each element; the loop edits.
- `for x in v` — take ownership of each element; the loop consumes, and `v` is gone.

Chapter 6's aliasing rule follows you into the loop body, and it quietly retires a classic Python bug: with `for cmd in &log` active, you cannot also `log.push(...)` inside the loop, because that would mutate a collection you are currently borrowing. The Python book could only warn you never to modify a list while iterating over it; the borrow checker rejects the program.

String and &str: the two-string problem

Here is the single most confusing thing this chapter has to teach, so let's take it slowly. Python has one string type. Rust, from where you sit, appears to have two, and every Rust learner arriving from Python spends a bewildered week discovering which functions want which.

The two types are:

- **`String`** — an owned, growable string, allocated on the heap. This is the closest thing to a mutable Python `str`-builder: you can push characters onto it, and whoever owns it is responsible for it, per Chapter 5.
- **`&str`** (pronounced "string slice") — a *borrowed view* of string data that lives somewhere else. It doesn't own anything; it points at characters and knows how many of them it covers. It is Chapter 6's `&T`, specialized for text.

The reason beginners meet the confusing one first: **every string literal is a `&str`.** When you write `"TinyALU"`, those seven bytes are baked into your compiled program itself, and the literal is a borrowed slice pointing into that program memory. It costs nothing, it lives forever, and it is read-only. Figure 5 shows both types and the traffic between them.

```
# Figure 5: String literals are borrowed; String is owned

fn main() {
    let dut: &str = "TinyALU";            // borrowed slice into program memory
    let mut name: String = String::from("Tiny");
    name.push_str("ALU");                 // owned and growable
    let banner = format!("*** Testing the {} ***", name);
    println!("{}", dut);
    println!("{}", banner);
}
```

```
--
TinyALU
*** Testing the TinyALU ***
```

`String::from("Tiny")` copies the literal's characters into a fresh, owned, heap-allocated `String` that we can grow with `push_str`. And `format!` is your f-string: `format!("Testing the {}", name)` builds a new `String` the way `f"Testing the {name}"` built a new `str` — same job, and like Python's strings, Rust's are immutable-by-default; growing one requires `mut`, and replacing text builds a new string rather than editing in place.[2]

Now the question that actually bites: when you write a function that takes text, which type goes in the signature? The rule of thumb is worth memorizing, because it appears throughout rustvm's own API:

> **Store `String`, pass `&str`.** Struct fields that keep text own it as `String`; function parameters that read text borrow it as `&str`.

The reasoning is pure Chapter 6. A function that only *reads* a name has no business demanding ownership of it — that would force every caller to give up (or clone) their string just so you could look at it. A parameter of type `&str` says "lend me a view," and it is maximally accepting: a `&String` coerces to a `&str` automatically, so callers can pass a literal, a slice, or a borrowed `String`, all with no copying. Figure 6 shows the shape.

```
# Figure 6: Take &str; accept everything

fn report_pass(test_name: &str) {
    println!("PASSED: {}", test_name);
}

fn main() {
    let owned = String::from("random_ops");
    report_pass("smoke_test");    // a literal is already a &str
    report_pass(&owned);          // a &String coerces to &str
    println!("still have {}", owned);
}
```

```
--
PASSED: smoke_test
PASSED: random_ops
still have random_ops
```

If instead the function is going to *keep* the text — storing a component's name in a struct field, say — it should take a `String` and own it outright, and the move at the call site honestly documents the handoff. When rustvm asks for `&str` in a signature, it is promising to look and not keep; when it asks for `String`, it is telling you the name is moving in permanently.

One Python habit does not survive the crossing at all: indexing into a string. `line[45]` was everyday Python; in Rust, `s[0]` on a `String` does not compile. Rust strings are UTF-8 encoded, so a character can occupy anywhere from one to four bytes, and Rust refuses to guess whether you want the byte or the character.[3] When you need the characters, say so — `for ch in s.chars()` iterates over them, mirroring the Python book's "string as an iterable" figure — and methods like `split`, `trim` (Python's `strip`), `replace`, and `contains` cover the daily string chores you already know by their Python names.

HashMap: the dictionary, minus the ordering promise

`HashMap<K, V>` is the Python dict: store a value under a key, get it back later. It lives in the standard library's collections module rather than the language itself, so it needs an import — your first `use` statement doing real work.

The Python book's signature dictionary example counted the letters in `"Mississippi"`, using `KeyError` (and then `setdefault`) to handle the first time each letter appeared. Our version counts something a verification engineer actually tallies: how many times the testbench has exercised each ALU operation — the raw

material of functional coverage. Rust's replacement for the whole `try/except KeyError/setdefault` dance is the *entry API*, and it is one line.

```
# Figure 7: A HashMap<Ops, u32> op-frequency counter

use std::collections::HashMap;

#[derive(Clone, Copy, Debug, PartialEq, Eq, Hash)]
enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

fn main() {
    let op_stream = vec![Ops::Add, Ops::Mul, Ops::Add,
                         Ops::Xor, Ops::Add, Ops::Mul];
    let mut freq: HashMap<Ops, u32> = HashMap::new();
    for op in &op_stream {
        *freq.entry(*op).or_insert(0) += 1;
    }
    for (op, count) in &freq {
        println!("{:?}: {}", op, count);
    }
}
```

```
--
Mul: 2
Xor: 1
Add: 3
```

Three things to unpack. First, the derive line grew: a type used as a `HashMap` key must support hashing and full equality, so `Ops` now derives `Eq` and `Hash` alongside the derives from Chapter 7. In Python, hashability was a runtime property you discovered when a `dict` rejected your key; in Rust it is a capability you declare on the type, and using a non-hashable key is a compile error. (What deriving really does, and the family of traits behind it, is Chapter 10's story.)

Second, the entry line: `freq.entry(*op).or_insert(0)` means "find this key's slot, filling it with 0 if it's empty, and hand me access to the value" — `setdefault`, near-verbatim, with the leading `*` saying "increment the value the entry points at," Chapter 6's dereference doing honest work. For simple lookup, `freq.get(&Ops::Add)` plays the role of `players.get(4)`: where Python's `get` returned the value or `None`, Rust's returns that same something-or-nothing `Option` type that `Vec::get` gave us — Chapter 9 is circling closer.

Third — and this one is a genuine behavioral difference, worth a flag in your notes — **look at the output order.** We inserted Add first; the printout led with Mul. The Python book was careful to note (as of Python 3.7) that dicts return keys in insertion order, and if you have written Python since then you have surely leaned on it. `HashMap` makes *no* ordering promise: iteration order is arbitrary, and it is deliberately randomized from run to run, so the same program can print `Add, Mul, Xor` today and `Xor, Add, Mul` tomorrow. Any testbench logic that quietly depends on dict order — golden log files compared line-by-line are the classic offender — must either sort the keys before printing or use `BTreeMap`, `HashMap`'s sibling that keeps keys in sorted order at a small cost. Iterating `for (op, count) in &freq` borrows, exactly per this chapter's rules, and destructures each key-value pair into two variables the way `for number, item in count.items()` did.

The rest of the toolbox, briefly

Here is the promised fast tour of what transfers without drama — the material of the Python book's tuples, ranges, sets, and "commonly used sequence operations" chapters, each in a sentence or two.

Tuples exist and you have already used them: `let pair = (5, Ops::Add);` groups values of different types, `pair.0` indexes them, and `let (a, op) = pair;` destructures — so functions return multiple values exactly as they did in Python. **Ranges** you met in Chapter 4: `0..5` is `range(5)`, `0..=4` includes the end. **Sets** are `HashSet<T>`, with `insert`, `contains`, and the union/intersection/difference operations — and the same no-ordering caveat as `HashMap`, which shares its machinery.

Of the common sequence operations: `v.len()` is `len(v)`; `v.contains(&x)` is `x in v`; `v.sort()` sorts in place like `list.sort()`, with `sort_by_key` playing the `key=` role; `v.iter().max()` and `.min()` return — you can guess by now — an `Option`, because the vector might be empty, a case Python handled by raising `ValueError`. Slicing carries over almost keystroke-for-keystroke: `&log[1..3]` is `log[1:3]`, a borrowed view of elements one and two, and the same range syntax slices strings and arrays. The borrowed-ness is the Rust twist: a slice is a reference into the original, so Chapter 6's rules apply while you hold it. What you will *not* find are `+` and `*` on vectors; Rust spells concatenation and repetition through methods (`extend`, `concat`, `"-".repeat(15)` for the Python book's horizontal bar). And list comprehensions have no literal syntax — their job is done by iterator chains like `map` and `filter`, which are Chapter 12's whole subject and worth the wait.

Summary

Rust's collections are Python's collections with ownership made visible. A `Vec<T>` is the list, but it owns its elements: pushing a transaction moves it in, dropping the `Vec` drops everything inside, and the who-frees-this question of Chapter 5 has a container-sized answer. Iteration comes in three spellings — `&v` borrows, `&mut v` borrows mutably, bare `v` consumes — and the borrow checker enforces the never-mutate-while-iterating rule Python could only put in a warning box. Text comes in two types: owned, growable `String` and borrowed `&str`, with string literals being `&str` slices into your program's own memory, and the rule of thumb *store `String`, pass `&str`*. `HashMap<K, V>` is the dict, with the entry API replacing the `KeyError` dance and one honest regression from Python 3.7: no insertion-order promise, so sort your keys before comparing log files. Tuples, ranges, sets, slices, sorting, and membership tests all transfer nearly unchanged.

Along the way, one type kept appearing in the corners of figures and refusing to explain itself: `Vec::get`, `HashMap::get`, `max`, and `min` all returned an `Option`, Rust's way of saying "there might be nothing here" — in the type, where the compiler can see it. Python answered the same question with `None` checks and `KeyError`; Rust's answer is better, and it comes with a partner named `Result` that replaces exceptions entirely. That is Chapter 9.

---

[1] Standalone playground project, per our Part I convention: `cargo new collections` and everything in this chapter runs in `src/main.rs`. And yes, `vec!` has the exclamation point of a macro, like `println!` — Chapter 21 explains why; until then, enjoy the enthusiasm.

[2] The Python book proved `str` immutability by showing `replace()` returning a new object with a new `id()`. Rust's `s.replace("real", "mall")` likewise returns a fresh `String` and leaves the original alone — same lesson, no `id()` required, because ownership tells you there are two strings.

[3] The moment this stops feeling like pedantry is the moment a signal name, a file path, or a log message contains its first non-ASCII character. Python 2 programmers can tell you stories; Rust decided at birth never to be in those stories.

The randomization is a deliberate defense — a hash table with predictable layout invites pathological (even malicious) key patterns — but the practical takeaway for us is simpler: if your test passes or fails depending on `HashMap` iteration order, the bug is in the test.

Next: Chapter 9, where `Option` finally introduces itself properly, `Result` retires the exception, and the `?` operator makes honest error handling almost as terse as ignoring errors used to be.

Chapter 9: Result, Option, and the End of Exceptions

> **In Python we...** reported errors with exceptions. The Python book explained them with a workplace metaphor: your boss asks you to do a task, you hit an error, and you raise it to your boss. Now it's off your plate. Your boss raises it to their boss, and so on up the chain until somebody handles it — or nobody does, and the error "gets reported to the public" as a crash with a traceback. Exceptions travel up the call stack the same way, and we caught them with `try/except/finally`.

Rust does not have exceptions. Not "discourages them," not "has them but calls them something else" — the mechanism does not exist. There is no `raise`, no `try`, no `except`, and nothing that silently unwinds through your function while it's minding its own business.

Before you mourn, remember what the boss metaphor was papering over. In Python, when you call a function, *nothing about that function tells you it might raise.* `nice_div()` looks exactly like a function that always returns a number. The fact that it can instead fling a `ZeroDivisionError` through your code is invisible — undocumented control flow that you discover at runtime, in our world usually forty minutes into a simulation. Chapter 1 promised that Rust moves discoveries like this to the compile step. This chapter is where that promise gets paid for errors.

Rust's answer is almost embarrassingly simple: **errors are ordinary values, and functions that can fail say so in their return type.** Two enums from the standard library do all the work:

- `Option<T>` — a value that might not be there.
- `Result<T, E>` — an operation that might fail, and if so, why.

You already have the tool for consuming both: `match`, from Chapter 4. This chapter is `match` earning its keep.

Option: a value that might not be there

Python has one value for "nothing here": `None`. It shows up as a return value (`players.get(4)` returns `None` when number 4 isn't in the dictionary), as a default, and as a sentinel. And it carries a famous failure mode: `None` is a perfectly good Python object right up until you use it like the thing you expected, at which point you get an `AttributeError` — often far from the line that produced the `None`, which is what makes it such a satisfying bug to chase.[1]

Rust refuses to let "maybe nothing" hide inside an ordinary type. A `u8` is always a number; a `String` is always a string. When a value might be absent, the type says so:

```
enum Option<T> {
    Some(T),
    None,
}
```

This is just an enum with a payload, exactly like the ones you built in Chapter 7 — it happens to be defined in the standard library and generic over any type `T` (generics get their full treatment in Chapter 11; for now, read `Option<u8>` as "maybe a `u8`").

You met `Option` briefly in Chapter 8, because `HashMap` lookups return one. Let's port the Python book's football-players dictionary directly. In Python, `players.get(4)` returned `None`, and `players.get(4, "Not in database")` supplied a default. Figure 1 is the same program in Rust.

```
# Figure 1: A HashMap lookup returns Option

use std::collections::HashMap;

fn main() {
    let mut players = HashMap::new();
    players.insert(7, "Beckham");
    players.insert(10, "Messi");
    players.insert(11, "Salah");

    match players.get(&4) {
        Some(name) => println!("Number 4? {name}"),
        None => println!("Number 4? Not in database"),
    }

    let player = players.get(&4).copied().unwrap_or("Not in database");
    println!("Number 4? {player}");
}
```

```
--
Number 4? Not in database
Number 4? Not in database
```

The two halves of figure 1 do the same job two ways. The `match` is the fundamental move: an `Option` is an enum, so we take it apart with `match`, and the compiler *requires* both arms. You cannot forget the `None` case — leaving it out is a compile error, not a 2 a.m. discovery. The second half uses `unwrap_or`, one of a family of convenience methods on `Option` that package up the common matches; it is the exact analog of passing `get()` a default in Python.

Here is the part worth slowing down for. In Python, the `None` from a failed lookup is the same shape as any other value — you can pass it along, store it in a transaction field, and only find out three function calls later. In Rust, an `Option<&str>` *is not* a `&str`. You cannot print it as a name, compare it to a name, or hand it to a function expecting a name. The compiler stops you at the line where you forgot to handle absence — which is to say, the `AttributeError`-far-from-the-bug failure mode is not merely discouraged. It doesn't compile.

When you only care about one arm, `if let` from Chapter 4 reads better than a `match` with an empty arm:

```
if let Some(name) = players.get(&10) {
    println!("Found: {name}");
}
```

Result: an operation that can fail

`Option` says "there might be nothing." `Result` says "this might fail, and here is why":

```
enum Result<T, E> {
    Ok(T),
    Err(E),
```

```
}
```

`T` is the type you get on success; `E` is the error type you get instead. And now we can port the Python book's exceptions chapter scenario for scenario, starting where it started: dividing by zero.

```
# Figure 2: You still can't divide by zero

fn main() {
    let divisor = 0;
    println!("3/0 = {}", 3 / divisor);
}
```

```
--
thread 'main' panicked at src/main.rs:3:26:
attempt to divide by zero
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

That is a **panic** — Rust's crash. We will come back to panics later in the chapter, because they have a specific job, and "report a divide by zero to the caller" is not it. When the possibility of failure is part of a function's honest contract, the function should return it. The standard library agrees: alongside the `/` operator, integers provide `checked_div`, which returns an `Option` — `None` instead of a crash.[2]

The Python book's `nice_div()` caught `ZeroDivisionError` and returned `math.inf`. Our integer version can't return infinity, but it can do something better: return a `Result` that names what went wrong. Figure 3 is `nice_div`, Rust edition.

```
# Figure 3: nice_div returns a Result instead of raising

#[derive(Debug)]
enum DivError {
    DivideByZero,
}

fn nice_div(dividend: u32, divisor: u32) -> Result<u32, DivError> {
    match dividend.checked_div(divisor) {
        Some(result) => Ok(result),
        None => Err(DivError::DivideByZero),
    }
}

fn main() {
    match nice_div(33, 2) {
        Ok(result) => println!("nice_div(33, 2) = {result}"),
        Err(err) => println!("You screwed up your division, human: {err:?}"),
    }
    match nice_div(3, 0) {
        Ok(result) => println!("nice_div(3, 0) = {result}"),
        Err(err) => println!("You screwed up your division, human: {err:?}"),
    }
}
```

```
--
nice_div(33, 2) = 16
You screwed up your division, human: DivideByZero
```

Read the signature first: `fn nice_div(dividend: u32, divisor: u32) -> Result<u32, DivError>`. That return type is the whole philosophy in one line. In Python, `nice_div`'s ability to fail was a secret

between the function body and whoever read the documentation. In Rust it is in the signature, which means the *compiler* knows, which means every caller is forced to acknowledge it. The `match` in `main` is our `except` block — except it cannot be forgotten, because you cannot get the `u32` out of a `Result<u32, DivError>` without going through it.

Notice also who's who in the port: the `Err` arm of the `match` is playing the role of Python's `except ZeroDivisionError` block, and constructing `Err(DivError::DivideByZero)` is playing the role of `raise`. Same drama, but now the error travels as a return value, in plain sight.

The exception that vanished

The Python book's next scenario was `nice_div(3, "zero")` — dividing an `int` by a `str`, which raised a `TypeError`, which required a second `except` block to catch. Figure 4 ports that call to Rust.

```rust
# Figure 4: The TypeError scenario, ported to Rust

fn main() {
    match nice_div(3, "zero") {
        Ok(result) => println!("nice_div = {result}"),
        Err(err) => println!("Error: {err:?}"),
    }
}
```

```
--
error[E0308]: mismatched types
 --> src/main.rs:4:24
  |
4 |     match nice_div(3, "zero") {
  |           -------- ^^^^^^ expected `u32`, found `&str`
  |           |
  |           arguments to this function are incorrect
```

It doesn't compile. Of the two failure categories the Python book needed `except` blocks for, one has simply left the building: passing the wrong *type* is not a runtime error to be caught, it is a program the compiler refuses to build. The Python chapter's figures 4 through 6 — the uncaught `TypeError`, the second `except` block, catching two exception types on one line — have no Rust equivalents, because the bug they handle cannot occur. Keep a tally of moments like this; the book has several more coming.[3]

The ? operator: propagation you can see

Back to the boss metaphor. The genuinely good idea inside exceptions was *delegation*: a low-level function shouldn't have to decide what a divide-by-zero means for the whole program. It should hand the problem upward. Python's `raise` did that invisibly. Rust does it with one visible character.

Suppose we're writing a little TinyALU-flavored utility: compute a ratio of two accumulated counts as a percentage. It calls `nice_div`, and if the division fails, our function can't succeed either — the error should go up to *our* caller. Written longhand, that's a `match` where the `Err` arm just re-returns the error. Written idiomatically, it's figure 5.

```rust
# Figure 5: The ? operator sends the error up the stack

fn percent(numerator: u32, denominator: u32) -> Result<u32, DivError> {
    let ratio = nice_div(numerator * 100, denominator)?;
    Ok(ratio)
}
```

```
fn main() {
    match percent(40, 0) {
        Ok(pct) => println!("{pct}%"),
        Err(err) => println!("percent failed: {err:?}"),
    }
}
```

```
--
percent failed: DivideByZero
```

The `?` after `nice_div(...)` means: *if this is `Ok(value)`, unwrap it and keep going; if it is `Err(e)`, return `Err(e)` from this function, right now.* That is exception propagation — the error bubbles up the call stack, each function passing it to its boss — with two differences that change everything:

1. **It's visible at the call site.** Every fallible call in a Rust function is marked with a `?` (or an explicit `match`). Scanning a function body tells you exactly where it can bail out early. A Python function body gives you no such list; any line might throw.
2. **It's visible in the signature.** You can only use `?` in a function that itself returns a `Result` (the compiler enforces this, with a helpful error if you forget). So the ability to fail is contagious *in the types*: if `percent` propagates `nice_div`'s errors, then `percent`'s signature says `Result`, and its callers are on notice too. The chain of bosses is written down.

The Python book showed re-raising: catch an exception, print a snarky message, then `raise` to send it upward anyway. The Rust spelling of that pattern is a `match` (or an `inspect_err` call) that logs and then returns the `Err` — nothing new to learn, just values. And when the error types along the chain differ, `?` will convert between them automatically if you've told it how; that hook is a trait called `From`, and traits are the very next chapter, so we'll leave that thread hanging deliberately.

One more nicety: `main` itself can return a `Result`. When it returns an `Err`, the program prints the error and exits with a failing status — the last boss in the chain has a sensible default. In Part II you'll see that rustvm tests work the same way: a test is an `async fn` returning `Result<(), TestError>`, and an `Err` fails the test *(design doc: §0.6)*. The `?` operators sprinkled through a testbench are little arrows pointing at everything that can end the test.

Designing an error enum

`DivError` had one variant, which made it a demo. Real error types earn their keep when there are several ways to fail and the caller might care which one happened — exactly the job Python's exception *class hierarchy* did, with `except ZeroDivisionError` and `except TypeError` selecting by type. In Rust, the error type is an enum, the variants are the failure modes, payloads carry the evidence, and `match` does the selecting.

Let's build one the TinyALU will actually need. Think ahead to a testbench utility that decodes an operation field from the DUT into our `Ops` enum from Chapter 7. Two things can go wrong: the bits might not encode any legal operation, or the DUT might never hand us the value at all before the clock runs out. That's a two-variant enum, one variant carrying the offending byte:

```
# Figure 6: A custom error enum for the TinyALU

use std::fmt;

#[derive(Debug, Clone, Copy, PartialEq)]
enum AluError {
    InvalidOp(u8),
    Timeout,
}
```

```rust
impl fmt::Display for AluError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self {
            AluError::InvalidOp(bits) => {
                write!(f, "invalid ALU op code: {bits:#04x}")
            }
            AluError::Timeout => write!(f, "timed out waiting for the DUT"),
        }
    }
}

fn decode_op(bits: u8) -> Result<Ops, AluError> {
    match bits {
        1 => Ok(Ops::Add),
        2 => Ok(Ops::And),
        3 => Ok(Ops::Xor),
        4 => Ok(Ops::Mul),
        _ => Err(AluError::InvalidOp(bits)),
    }
}

fn main() {
    for bits in [1, 4, 7] {
        match decode_op(bits) {
            Ok(op) => println!("{bits} decodes to {op:?}"),
            Err(err) => println!("decode failed: {err}"),
        }
    }
}
```

```
--
1 decodes to Add
4 decodes to Mul
decode failed: invalid ALU op code: 0x07
```

Walk through what each piece buys:

- `#[derive(Debug, ...)]` gives us the `{err:?}` developer-facing printout for free — the same derive habit from Chapters 7 and 8.
- **The `Display` implementation** is the human-facing message, the counterpart of the string you'd pass when raising a Python exception (`raise ValueError(f"invalid op: {bits}")`). Writing `impl fmt::Display for AluError` is our first hand-written trait implementation; Chapter 10 will explain the machinery you just used. For now: implementing `Display` is what makes `{err}` (no `:?`) work, and it's the conventional courtesy every public error type pays.
- **The payload on `InvalidOp(u8)`** carries the evidence to wherever the error is handled. Python attached this data to the exception object; we attach it to the variant. No fishing it back out of a message string.
- **Callers select with `match`.** A caller who wants to retry on `Timeout` but fail hard on `InvalidOp` writes a two-arm match — the moral equivalent of two `except` blocks, checked for exhaustiveness by the compiler. Add a third variant to `AluError` next month, and every such `match` in the codebase becomes a compile error until it says what to do about the new case. Try getting *that* from an exception hierarchy.

This pattern — an enum of failure modes, `Debug` derived, `Display` implemented, payloads where useful — is the whole craft of error design in application code, and it's the shape rustvm's own errors take (`HandleError`

for signal lookups, `TestError` for test outcomes — you'll meet them in Chapter 17).

panic!: for bugs, not for failures

Now we can return to figure 2's crash. `panic!` is Rust's mechanism for *this program has a bug*: it prints a message and location, unwinds the current thread, and by default takes the program down. You can invoke it yourself:

```
panic!("driver state machine reached an impossible state");
```

The panicking family has members you will use daily:

- **`assert!(condition, "message...")`** — panic if the condition is false. `assert_eq!(a, b)` panics if two values differ, and prints both.
- **`.unwrap()`** — on an `Option` or `Result`: give me the value, and panic if it's `None`/`Err`.
- **`.expect("message")`** — `unwrap` with a message you choose, which makes it strictly better in code you keep.

Figure 7 ports the Python book's `assert` scenario — the `xor_bytes` function that rejected values that don't fit in a byte. The port sharpens the example nicely: in Rust, `xor_bytes` takes `&[u8]`, so a value over 255 can't even reach the function (the `TypeError` disappearance, again). What still *can* go wrong is a claim about our own logic — say, this version that folds in a parity check the surrounding testbench relies on:

```
# Figure 7: assert! guards an invariant

fn xor_bytes(bytes: &[u8]) -> u8 {
    let mut xor = 0;
    for b in bytes {
        xor ^= b;
    }
    xor
}

fn main() {
    let frame = [0x08, 0x09, 0x10];
    let checksum = xor_bytes(&frame);
    println!("checksum: {checksum:#04x}");
    assert!(
        xor_bytes(&[checksum, 0x11]) == 0,
        "checksum self-test failed: {checksum:#04x} ^ 0x11 != 0"
    );
    assert!(
        xor_bytes(&[checksum, 0x12]) == 0,
        "checksum self-test failed: {checksum:#04x} ^ 0x12 != 0"
    );
    println!("all self-tests passed");
}
```

```
--
checksum: 0x11
thread 'main' panicked at src/main.rs:16:5:
checksum self-test failed: 0x11 ^ 0x12 != 0
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

The first assertion passes silently; the second one stops the program at the line where the impossible happened, with our message. That is what you want from an invariant check: loud, early, and located.

The Python book closed its `assert` discussion with a trap: `assert (8 < 7, "Obviously false")` never fires, because the parentheses build a two-element tuple, and a non-empty tuple is truthy. I am pleased to report the Rust translation of that bug: `assert!((8 < 7, "Obviously false"))` does not compile, because a tuple is not a `bool`, and `assert!` insists on a `bool`. The trap is not merely avoided; it is unrepresentable.

So when do you panic and when do you return `Err`? The line is intent:

- **`Result::Err` is for failures the design expects.** A signal that might not exist, an operand that might be out of range, a check that might not pass. These are outcomes, and the caller gets to decide what they mean.
- **`panic!`/`assert!` are for states the design promises are impossible.** If one fires, the code — not the input, not the DUT — is wrong, and there is no sensible way to continue.

`.unwrap()` and `.expect()` sit exactly on this line, which is why they need judgment: each one converts an `Err`/`None` into a panic, so each one is a small signed statement that says "I claim this cannot fail here." In playground code and examples, unwrap freely. In a testbench you'll run for a year, prefer `expect` with a message that will make sense to whoever reads the panic — probably you, later, in a worse mood.

One Python comfort has no direct Rust twin: `finally`. Rust's guarantee of "this cleanup runs no matter what" doesn't live in error-handling syntax at all — it lives in ownership. When a scope ends, by `return`, by `?`, or by panic-unwinding, values are dropped and their destructors run, as you saw in Chapter 5. Cleanup-on-any-exit is not something you remember to write in Rust; it's where the `Drop` happens.

The taxonomy this book will live by

Everything above compresses into one convention, and it is load-bearing: the rest of this book — and the design of rustvm itself — assumes it *(design doc: §7.3, testing conventions)*.

> **The failure taxonomy. `Result::Err` is for checks** — the DUT did something wrong. A scoreboard comparing predicted against actual and finding a mismatch produces an `Err`. The test fails, which is the test doing its job. This is *expected fallibility*: finding these is why we come to work. **`panic!`/`assert!` are for testbench bugs** — *we* did something wrong. A driver calling `item_done` twice, a queue that is empty when the protocol guarantees it can't be, a state machine in a state the match arms say is impossible. The testbench is broken, and no result it reports can be trusted until it's fixed.

Both fail the test — rustvm catches panics at the task boundary and scores them as failures, just as cocotb caught a stray exception in any task — but the report distinguishes them, because the reader of the report must react differently: an `Err` sends you to the waveform viewer; a panic sends you to your own source code. It is the difference between the lab reporting that the chip failed and the lab reporting that the thermometer is broken.

Python's world blurred this line: an `AssertionError` from a scoreboard check and an `AttributeError` from a testbench typo were both just exceptions rising through the same machinery, distinguished only by convention and by reading the traceback. Rust gives the two categories different mechanisms, different types, and different syntax — so from Chapter 18 on, when you see a scoreboard whose check returns `Result` and a driver studded with `assert!`, you are seeing this chapter's taxonomy at work, not a stylistic accident.

You now hold both halves of Rust's honesty policy: types that admit absence (`Option`), and signatures that admit failure (`Result`). Along the way, you implemented your first trait — that `Display` on `AluError` — mostly on faith. Chapter 10 replaces the faith with understanding: traits are how Rust does everything Python did with inheritance and dunder methods, and they are the last big idea between you and real testbench components.

[1] The Python book's dictionaries chapter used `players.get(4)` returning `None` for a missing hockey player. The bug arrives when you then write `player.upper()` — and Python tells you `'NoneType' object has no attribute 'upper'`, three files away from the lookup.

[2] The Python book returned `math.inf` for division by zero, on the theory that it's the mathematically correct answer. Rust's *floating-point* division agrees — `3.0 / 0.0` is `inf`, per IEEE 754, no panic — so on this one point the two books are in complete accord and only the integers object.

[3] Chapter 27 is essentially this moment stretched to a full chapter: every ConfigDB failure mode from the Python book, replayed as a compile error.

A colleague of mine calls `.unwrap()` "a panic with no commit message."

End of Chapter 9 draft.

Chapter 10: Traits

Every chapter so far has taken something Python did at runtime and shown Rust doing it at compile time. This chapter takes the biggest one yet: the machinery of object-oriented programming itself. The Python book spent three chapters on inheritance, `super()`, and the method resolution order, because pyuvm is built on them — `uvm_driver` extends `uvm_component`, which extends `uvm_object`, and every `__init__()` dutifully calls `super().__init__()` up the chain.

Rust has no inheritance. None. There is no base class, no child class, no `super()`, no method resolution order to print. When I first learned this I assumed Rust must be missing something essential, and I was wrong in an instructive way: inheritance was never the *goal*. It was one language's mechanism for two separate goals — sharing behavior across types, and treating different types uniformly. Rust delivers both with a single feature called a **trait**, and once you see the two goals pulled apart, you may find you don't miss the mechanism.

> **In Python we...** used inheritance to avoid copying code. "Inheritance allows us to avoid copying code and modifying it by creating a base class that contains shared behavior" — we recognized that a dog is an animal and a cat is an animal, wrote `make_sound()` once in `Animal`, and extended it with `class Dog(Animal)`. When a child overrode `__init__()`, we called `super().__init__()` so the parent could set its data attributes, and Python found every method by walking the method resolution order — the MRO — from child to parent to `object`. The book made it a habit, and pyuvm made it a requirement: *always call `super().__init__()`*.

This chapter rebuilds that material piece by piece: the Animal menagerie, the `super()` question, multiple inheritance, and then the payoff for verification — how traits replace the dunder methods (`__eq__`, `__str__`, `__lt__`) that the Python book taught you to write on every transaction class.

Shared behavior without a base class

A trait is a named list of method signatures that a type can opt into. It declares *what* a type can do; a separate `impl` block declares that a particular type does it. Here is the Python book's Animal example, rebuilt.

```
# Figure 1: Shared behavior through a trait, not a base class

trait Animal {
    fn species(&self) -> &str;
    fn sound(&self) -> &str;

    fn make_sound(&self) {
        println!("The {} says '{}'", self.species(), self.sound());
    }
}

struct Dog;
```

```
struct Cat;

impl Animal for Dog {
    fn species(&self) -> &str { "dog" }
    fn sound(&self) -> &str { "bow bow" }
}

impl Animal for Cat {
    fn species(&self) -> &str { "cat" }
    fn sound(&self) -> &str { "miāo" }
}

fn main() {
    Dog.make_sound();
    Cat.make_sound();
}
```

```
--
The dog says 'bow bow'
The cat says 'miāo'
```

Read the trait from the bottom up. `species()` and `sound()` are *required* methods — they have signatures but no bodies, so any type implementing `Animal` must provide them. `make_sound()` has a body, which makes it a **default method**: implementors get it for free and may override it. The shared behavior the Python book put in the `Animal` base class lives in the default method; the per-type differences the Python book put in `__init__()` live in each `impl` block.

Notice what is *not* here. `Dog` does not mention `Animal` in its definition — `struct Dog;` stands alone, and the relationship is declared separately, in `impl Animal for Dog`. In Python, `class Dog(Animal)` fused "what Dog is" with "what Dog can do" into one statement. Rust keeps them apart, and the separation has a consequence you will come to rely on: you can implement a new trait for an existing type without touching the type's definition. When Chapter 32 needs a coverage collector to receive transactions, it will not edit the transaction — it will implement `Subscriber` on the collector, and the transaction never knows.

One more absence: data. A Python base class could set `self.species = None` in its `__init__()` and let children fill it in. A trait *cannot contain fields* — only method signatures and default bodies. If shared behavior needs data, it asks for the data through a required method, exactly as `make_sound()` asks for `species()`. This will feel like ceremony for about one chapter, and then it will feel like honesty: the trait's signature documents precisely what it needs from you, instead of quietly reaching into your attribute namespace and hoping.

Where did super() go?

The Python book's most instructive inheritance bug was `SmallDog`. It extended `Dog`, overrode `__init__()` to set `self.sound = "yap yap"`, forgot to call `super().__init__()`, and blew up with an `AttributeError` because `self.species` was never set — data attributes, unlike methods, are not inherited. The fix was `super().__init__()`, and the book told you to make it a habit.

Rust closes this bug at both ends. First: struct fields are declared in the struct, not created by whichever initializer happens to run. There is no execution order that leaves a `SmallDog` half-built, because a struct that is missing a field does not compile. Second: when a `SmallDog` wants to reuse `Dog`'s behavior, it does so by *containing* a `Dog` — composition — and delegating to it explicitly.[1]

```
# Figure 2: SmallDog by composition - delegation replaces super()

struct SmallDog {
```

```
    dog: Dog,    // a SmallDog HAS the dog parts
}

impl Animal for SmallDog {
    fn species(&self) -> &str { self.dog.species() }   // delegate to Dog
    fn sound(&self) -> &str { "yap yap" }              // override
}

fn main() {
    let sd = SmallDog { dog: Dog };
    sd.make_sound();
}
```

```
--
The dog says 'yap yap'
```

Look at `self.dog.species()`. That line is doing the job `super().__init__()` did in Python, but spelled as what it actually is: a call to a specific method on a specific value. There is no MRO to walk, no rule about which class comes next in the search, no way to forget the call and find out at runtime — if `SmallDog`'s `impl` doesn't provide `species()` one way or another, the compiler rejects the `impl` block on the spot, naming the missing method. The Python book's figure 4 — the `AttributeError` that "would surprise a SystemVerilog programmer" — has no Rust equivalent to show you. The program that produces it cannot be built.

You may remember that the Python book also taught `Dog.__init__(self)` — calling the parent's method explicitly through the class object — as the alternative whose "advantage is that you know which copy you are calling." Delegation is that idea, promoted from alternative to only option. Rust decided that knowing which copy you are calling is not an advantage but a requirement.

Wearing many hats

The Python book used multiple inheritance for Pat, the firefighter with kids: `class FirefighterWithKids(Parent, Firefighter)`, followed by a careful discussion of the MRO and a design rule borrowed from *Highlander* — of `__init__()` methods, there can be only one.

In Rust, "Pat has two roles" is simply "Pat implements two traits." There is nothing to diagram.

```
# Figure 3: Multiple roles as multiple trait implementations

trait Parent {
    fn kiss(&self);
}

trait Firefighter {
    fn hose(&self);
}

struct Pat {
    name: String,
}

impl Parent for Pat {
    fn kiss(&self) { println!("{} gives the baby a kiss.", self.name); }
}

impl Firefighter for Pat {
```

```
    fn hose(&self) { println!("{} sprays water.", self.name); }
}

fn main() {
    let pat = Pat { name: String::from("Pat") };
    pat.kiss();
    pat.hose();
}
```

```
--
Pat gives the baby a kiss.
Pat sprays water.
```

A type can implement as many traits as it likes, and because traits carry no data, the classic multiple-inheritance hazards — which parent's field wins, which `__init__()` runs, what order the MRO searches — never arise. `Pat` has exactly one set of fields, declared in exactly one place, and each trait bolts a capability onto it. The Highlander rule enforced itself.[2]

[1] The Python book credits the SmallDog change to Ron Swanson of Parks and Recreation. I see no reason to withdraw the attribution.

[2] Rust examined the method resolution order and politely declined to have one.

Default methods: pyuvm's no-op pattern, formalized

Here is where this chapter starts paying rent for Part IV. Think back to how pyuvm's phases worked: `uvm_component` defined `build_phase()`, `connect_phase()`, `run_phase()` and the rest as *empty methods*, and your components overrode only the phases they cared about. The base class's no-op bodies existed so the phase machinery could call every phase on every component without checking what each component had bothered to define.

That pattern — "here is the full interface; override what you use" — is exactly what default methods are for, and it is how rustvm's `Component` lifecycle trait is designed. A preview, signatures only (Chapter 24 does this properly):

```
# Figure 4: The shape of rustvm's Component trait (preview - signatures only)

pub trait Component {
    fn start(&mut self, ctx: &mut RunCtx);            // required: every component runs
    fn check(&mut self, errors: &mut CheckSink) {}    // default: do nothing
    fn report(&self) {}                               // default: do nothing
}
```

A scoreboard overrides `check()`; a driver doesn't, and inherits the empty default — the same division of labor the Python book taught in its `uvm_component` chapter, with one upgrade. In pyuvm, the empty methods lived in a base class you extended, so getting them required joining the inheritance hierarchy. In rustvm, they live in a trait you implement, so a component is just a plain struct — its fields are its children, per Chapter 24 — that opts into the lifecycle. Same methodology, no family tree.

Deriving: the compiler writes the boring impls

The Python book taught you a set of magic methods — the dunders — because transactions need them: `__eq__` so the scoreboard can compare a prediction to a result, `__str__` so logs are readable, and it was your job to write each one on every transaction class. Rust maps each dunder to a standard trait, and for most of them the compiler will write the implementation for you. The keyword is `#[derive(...)]`, an attribute

you place on a struct or enum, listing traits whose implementations the compiler should generate from the fields.

Let's put the TinyALU's transaction under the microscope.

```
# Figure 5: Derived traits on the TinyALU command transaction

#[derive(Clone, Copy, Debug, PartialEq)]
enum Ops {
    Add = 1,
    And = 2,
    Xor = 3,
    Mul = 4,
}

#[derive(Clone, Debug, PartialEq)]
struct AluCommand {
    a: u8,
    b: u8,
    op: Ops,
}

fn main() {
    let cmd = AluCommand { a: 0xAA, b: 0x55, op: Ops::Xor };
    let copy = cmd.clone();
    println!("equal? {}", cmd == copy);
    println!("{:?}", cmd);
}
```

```
--
equal? true
AluCommand { a: 170, b: 85, op: Xor }
```

One line above the struct bought us three implementations. `Clone` gives `cmd.clone()`, a field-wise copy. `PartialEq` gives `==`, a field-wise comparison — in Python this was you writing `__eq__` by hand, remembering to compare every field, and remembering again when you added a field. The derived version regenerates from the struct definition on every compile, so it *cannot* fall out of sync with the fields. `Debug` gives the `{:?}` format you see in the output: a machine-ish rendering of the whole struct, which is `__repr__`'s job.[3]

That leaves `__str__` — the human-friendly rendering. Its Rust counterpart is the `Display` trait, and here the compiler makes you write it yourself, on purpose: Rust's position is that a machine can guess how to *dump* your type but not how to *present* it. Implementing `Display` is our first hand-written implementation of a standard-library trait, and it looks like every trait impl you've seen this chapter:

```
# Figure 6: Implementing Display by hand - the __str__ of Rust

use std::fmt;

impl fmt::Display for AluCommand {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "cmd: a=0x{:02x} b=0x{:02x} op={:?}", self.a, self.b, self.op)
    }
}

fn main() {
    let cmd = AluCommand { a: 0xAA, b: 0x55, op: Ops::Xor };
    println!("{}", cmd);
```

```
}
```

```
--
cmd: a=0xaa b=0x55 op=Xor
```

Once `Display` exists, `{}` in any format string works, exactly as defining `__str__` made `print()` work. The `write!` macro is `println!`'s cousin that writes into the formatter instead of to the screen, and the `fmt::Result` return is Chapter 9's `Result` making a cameo — formatting can fail, and the signature says so.

Here is the full mapping, the table this chapter exists to give you. Left column: what the Python book taught you to write. Right columns: where it went.

Python magic method | The Python book used it for | Rust trait | How you get it |

|—|—|—|—| | `__eq__` | comparing transactions in the scoreboard | `PartialEq` | `#[derive(PartialEq)]` | | `__repr__` | unambiguous debug output | `Debug` (`{:?}`) | `#[derive(Debug)]` | | `__str__` | human-readable output, `convert2string` | `Display` (`{}`) | written by hand | | `__lt__`, `__le__`, `__gt__`, `__ge__` | ordering and sorting | `PartialOrd`, `Ord` | `#[derive(...)]` | | `__hash__` | using objects as dict keys | `Hash` | `#[derive(Hash)]` | | `__add__`, `__sub__`, … | operator overloading | `std::ops::Add`, `Sub`, … | written by hand | | `copy.deepcopy()` / `do_copy` | duplicating transactions | `Clone` | `#[derive(Clone)]` |

Two rows deserve a comment. `PartialOrd` and `Ord` split Python's ordering dunders in half because some types have values that refuse to be ordered (floating-point `NaN` is the culprit — hence *partial*); for transaction structs of integers and enums, you derive both and move on. And the operator traits in `std::ops` mean Rust has real operator overloading — `cmd_a + cmd_b` can be made to work — but it is opt-in per trait, per type, with no `__radd__`-style reflection rules to memorize.

If that table feels like it just dissolved most of a chapter of the Python book into an attribute line, hold the thought: Chapter 35 shows that it also dissolves most of `uvm_object`. The field-wise `do_copy` and `do_compare` machinery that pyuvm hand-rolled by walking `__dict__` at runtime is precisely what `derive` generates at compile time. A rustvm transaction is a plain struct with `#[derive(Clone, Debug, PartialEq)]` on top — no base class required.

[3] The derived `Debug` prints `a: 170` rather than `a: 0xaa` because it renders a `u8` as decimal — another small argument for writing `Display` yourself when humans will read the result.

Two kinds of polymorphism

We now have `Dog`, `Cat`, and `SmallDog`, each implementing `Animal`, and one question left — the question inheritance answered with "they share a base class": how do we write code that works on *any* animal?

Rust gives two answers, and choosing between them is a skill this book will exercise from here to the final chapter.

Answer one: generics. Write a function with a type parameter, and *bound* the parameter by the trait:

```
# Figure 7: Generic function - dispatch resolved at compile time

fn check_in<T: Animal>(animal: &T) {
    animal.make_sound();
}

fn main() {
    check_in(&Dog);
    check_in(&Cat);
}
```

```
--
The dog says 'bow bow'
The cat says 'miāo'
```

`<T: Animal>` reads "for any type T that implements Animal." This is duck typing with the ducks counted before the program runs: where Python said "just call `make_sound()` and hope," the bound *documents* the requirement in the signature and the compiler *checks* it at every call site. Behind the scenes the compiler compiles a separate copy of `check_in` for `Dog` and for `Cat` — a process called **monomorphization** — so each call dispatches directly, as fast as if you had written the two functions by hand. Generic code costs nothing at runtime. Chapter 11 is entirely about this machinery, so I'll leave it warm rather than cooked.

Answer two: trait objects. Sometimes you need one collection holding a mixture of types — a Python list held anything, and a kennel holds whatever shows up. For that, Rust erases the concrete type behind a pointer:

```
# Figure 8: Trait objects - dispatch resolved at runtime

fn main() {
    let kennel: Vec<Box<dyn Animal>> = vec![
        Box::new(Dog),
        Box::new(Cat),
        Box::new(SmallDog { dog: Dog }),
    ];
    for animal in &kennel {
        animal.make_sound();
    }
}
```

```
--
The dog says 'bow bow'
The cat says 'miāo'
The dog says 'yap yap'
```

`dyn Animal` is a **trait object**: "some type, I'm not saying which, that implements Animal." Because the compiler no longer knows the concrete type, two things follow. The value must live behind a pointer — here Chapter 13's `Box` — since different animals have different sizes and a `Vec` needs uniform elements. And each call to `make_sound()` is dispatched at runtime through a **vtable**, a small table of function pointers riding along with the object. If that sounds like how Python method calls or SystemVerilog virtual methods work — yes, exactly, except that Rust makes you *ask* for dynamic dispatch by writing `dyn`, and hands you static dispatch everywhere else.

The trade, in one breath: generics are faster and fully checked but require the concrete types to be knowable where the code is compiled; trait objects accept types chosen at runtime but pay a pointer, an allocation, and an indirect call. The rule of thumb this book follows: **reach for generics first, and reserve `dyn` for collections that genuinely must mix types.**

You will see rustvm make both choices, each where it belongs. The driver is `Driver<REQ, RSP>` — generic over its transaction types, so handing the wrong transaction to a driver is a compile error rather than the runtime type explosion pyuvm checked for by hand. Subscribers are a bound, `T: Subscriber`, the "anything with a `write()` method" of the analysis chapters. And trait objects appear where heterogeneity is the point: a config struct's maker closure returns `Box<dyn DriverLike>` so a test can substitute driver implementations at runtime (Chapter 29), and a sequencer stores its queued sequences as trait objects because sequences of different types wait in one line (Chapter 36). Known types: generics. One slot, many possible occupants: `dyn`.

Summary

Rust replaces inheritance with traits: named, explicit interfaces that a type opts into with an `impl` block. Required methods state what the type must provide; default methods carry shared behavior, doing the job of base-class methods — including pyuvm's empty-phase-method pattern, which becomes default methods on rustvm's `Component` trait. Code reuse comes from composition and delegation, and the delegating call replaces `super()` with an ordinary, visible method call. Multiple roles come from implementing multiple traits, with no MRO and no diamond.

The dunder methods the Python book taught map one-for-one onto standard traits — `__eq__` to `PartialEq`, `__repr__` to `Debug`, `__str__` to `Display`, the ordering dunders to `PartialOrd`/`Ord` — and `#[derive(...)]` makes the compiler write most of them from your struct's fields, which is how a rustvm transaction gets by with no base class at all. Finally, "code that works on any implementor" comes in two flavors: generic functions with trait bounds, monomorphized and free at runtime, and trait objects behind `dyn`, dispatched through a vtable when one collection must hold many types.

We have been leaning on that `<T: Animal>` notation while promising the details later. Later has arrived — Chapter 11 opens up generics: type parameters, bounds, and why `Driver<REQ, RSP>` is the most honest thing a driver has ever said about itself.

Chapter 11: Generics

Chapter 10 ended with a promissory note. We wrote `fn check_in<T: Animal>(animal: &T)`, waved at the angle brackets, and promised that generics — code with type parameters, resolved at compile time — would get their own chapter. This is that chapter. By the end of it you will know what the compiler actually does with `<T: Animal>`, why the result runs exactly as fast as code without it, and why `Driver<REQ, RSP>` is, as promised, the most honest thing a driver has ever said about itself.

Here is the small confession first: you have been using generics since Chapter 8. `Vec<T>` is a generic struct. So are `Option<T>` and `Result<T, E>` — every time you wrote `Vec<AluCommand>` or `Result<u16, AluError>`, you were filling in someone else's type parameters. This chapter teaches you to write your own.

> **In Python we...** never told a function what types it accepted, because Python functions accept everything. The Python book built its classes in that spirit — "you create the object and start adding data attributes to it" — and its testbench code ran on the same faith: a scoreboard's `write()` method took whatever the analysis port delivered, an `__eq__` compared whatever showed up, and if the object had the right attributes, everything worked. This philosophy has a name, *duck typing*: if it walks like a duck and quacks like a duck, treat it as a duck. Nobody checks for feathers until the moment of the quack.

Duck typing is genuinely pleasant to write. Its cost is *when* the feather-check happens: at every call, at runtime, and only on the paths your test actually exercised. Rust keeps the pleasant part — one function, many types — and moves the check. If it implements the trait bound, it's a duck, and you find out at compile time, for every path, including the ones your test forgot.

A function for any type

Suppose we want the largest value in a slice. For the TinyALU's `u8` operands we could write it directly, but the moment we also want the largest `u16` result, or the alphabetically last test name, we are copying the function and changing one type annotation — exactly the copy-and-modify busywork the Python book invoked inheritance to avoid. Rust's answer is a **type parameter**: a placeholder type, declared in angle brackets, that the caller fills in.

Let's write it the way you would naively write it, because the failure is the lesson.

```
# Figure 1: A generic function, first attempt - the compiler wants proof

fn largest<T>(list: &[T]) -> &T {
    let mut largest = &list[0];
    for item in list {
        if item > largest {
            largest = item;
        }
    }
    largest
}
```

```
--
error[E0369]: binary operation `>` cannot be applied to type `&T`
 --> src/main.rs:4:17
  |
4 |         if item > largest {
  |            ---- ^ ------- &T
  |            |
  |            &T
  |
help: consider restricting type parameter `T`
  |
1 | fn largest<T: std::cmp::PartialOrd>(list: &[T]) -> &T {
  |             ++++++++++++++++++++++
```

Read the declaration first: `fn largest<T>` says "this function is defined for some type T, to be named later," and then `&[T]` and `&T` use the placeholder as if it were a real type. Python would have shrugged and run this. Rust refuses, and its objection is philosophically the whole chapter: *you said T could be any type, and then you compared two of them with >. Not every type can do that.* What do we know about a type we know nothing about? Nothing — so we may call nothing on it.

The fix is in the compiler's own `help` text, and it is Chapter 10's vocabulary: a **trait bound**. `T: PartialOrd` narrows "any type" to "any type that implements `PartialOrd`" — the ordering trait from Chapter 10's dunder table, the one behind `<` and `>`.

```
# Figure 2: The bound is the fix - and the documentation

fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut largest = &list[0];
    for item in list {
        if item > largest {
            largest = item;
        }
    }
    largest
}

fn main() {
    let operands: Vec<u8> = vec![0x22, 0xAA, 0x07];
    let results: Vec<u16> = vec![0x0154, 0x7100, 0x00FF];
    let tests = vec!["alu_add_test", "alu_xor_test", "alu_mul_test"];

    println!("largest operand: 0x{:02x}", largest(&operands));
    println!("largest result:  0x{:04x}", largest(&results));
    println!("last test name:  {}", largest(&tests));
}
```

```
--
largest operand: 0xaa
largest result:  0x7100
last test name:  alu_xor_test
```

One function body, three element types, and no type mentioned at any call site — the compiler infers `T` from the argument, the same way it has been inferring types for you since Chapter 3.[1] Notice too that we return `&T`, a reference, not `T`: Chapter 5 taught us that returning the value itself would mean moving it out of the caller's slice, and the borrow is both cheaper and honest about who still owns the data.

The bound does double duty, and this is worth slowing down for. To the *compiler* it is a permission slip:

inside `largest`, exactly the methods of `PartialOrd` may be called on `T`, no more. To the *reader* it is documentation you can trust: the signature `fn largest<T: PartialOrd>(list: &[T]) -> &T` tells you everything this function will ever demand of your type. Python's equivalent contract lived in the docstring, the tribal knowledge, or the stack trace.

More than one requirement

A bound can require several traits at once, joined with `+`. Here is a function a scoreboard might want — compare an expected transaction against an actual one, and complain legibly on a mismatch. Comparing needs `PartialEq`; complaining legibly needs `Debug`. Both go in the contract:

```
# Figure 3: Multiple bounds with a where clause

use std::fmt::Debug;

fn check_match<T>(expected: &T, actual: &T) -> bool
where
    T: PartialEq + Debug,
{
    if expected == actual {
        true
    } else {
        println!("MISMATCH: expected {:?}, actual {:?}", expected, actual);
        false
    }
}

fn main() {
    check_match(&0x54u16, &0x54u16);
    check_match(&0x54u16, &0x55u16);
}
```

```
--
MISMATCH: expected 84, actual 85
```

We could have written `fn check_match<T: PartialEq + Debug>(...)` and it would mean the same thing; the `where` clause is the same information moved out of the angle brackets so the signature stays readable. Convention, and this book, use the inline form for one short bound and `where` for anything longer. Either way, look at what this function is: the Python book's duck-typed checker — "anything with an `__eq__` and a `__repr__`" — with the duck requirements written down and enforced. A transaction type that forgot to derive `PartialEq` doesn't fail in hour three of a regression; it fails to compile, with an error pointing at the missing derive.

Generic structs

Type parameters work on structs the same way, and you already know the syntax from the consumer side — `Vec<T>` — so producing one holds no surprises. Here is a register model wide enough for any leg of the TinyALU:

```
# Figure 4: A generic struct - one definition, many widths

struct Register<T> {
    value: T,
}
```

```rust
impl<T: Copy> Register<T> {
    fn new(value: T) -> Self {
        Register { value }
    }
    fn read(&self) -> T {
        self.value
    }
    fn write(&mut self, value: T) {
        self.value = value;
    }
}

fn main() {
    let mut a_reg: Register<u8> = Register::new(0x00);   // the A leg is 8 bits
    let mut result: Register<u16> = Register::new(0x0000); // the result is 16

    a_reg.write(0xAA);
    result.write(0x7100);
    println!("A: 0x{:02x}  result: 0x{:04x}", a_reg.read(), result.read());
}
```

```
--
A: 0xaa  result: 0x7100
```

Two spellings deserve a comment. `struct Register<T>` declares the parameter; `impl<T: Copy> Register<T>` declares it *again* for the methods, and that is where the bound lives — `read()` hands back a copy of the value, so the methods require `T: Copy`, which every integer satisfies. Keeping the struct definition unbounded and putting requirements on the `impl` is idiomatic: the data doesn't care what `T` can do; the *operations* do.

And note what the type system now knows that Python's never did: `Register<u8>` and `Register<u16>` are **different types**. Try to write a `u16` result into the 8-bit A register and the program does not compile. The Python book's testbenches kept the TinyALU's operand widths straight by discipline and masking; here the widths are in the types, and the compiler is the one doing the masking-related worrying. This — a struct parameterized by the type it carries — is precisely what `Vec<T>` has been all along, and it is the shape rustvm's plumbing takes: when Part IV connects components with typed channels, a FIFO of ALU commands is a `TlmFifo<AluCommand>`, and connecting it to a component expecting results is a compile error, not a 3 a.m. discovery.

Monomorphization, or: where did the ducks go?

Now the question a performance-minded verification engineer should be asking. Python's duck typing has a runtime cost, and it is not small: *every* method call, every attribute access — `item.compare(other)`, `self.scoreboard.write(txn)` — is a lookup by name, at runtime, every single time, because until the moment of the call Python genuinely does not know what the object is. Twenty million transactions means twenty million rounds of "does this object have a `write`?" That is the interpreter overhead Chapter 1 put on the ledger.

So what does `largest` cost? It is one function that works on three types — surely *something*, somewhere, is checking which type showed up?

Nothing is, and the reason is the best idea in this chapter. At compile time, the compiler finds every concrete type your program actually uses with `largest`, and generates a separate, specialized copy of the function for each one — a process called **monomorphization**, "making single-formed." Your generic source code is a stencil; the compiler stamps it out once per type. Conceptually, figure 2 compiles as if you had written:

```
# Figure 5: What the compiler generates from figure 2 (conceptually - you never see this)

fn largest_u8(list: &[u8]) -> &u8 {
    // ... same body, with T = u8 throughout
}

fn largest_u16(list: &[u16]) -> &u16 {
    // ... same body, with T = u16 throughout
}

fn largest_str(list: &[&str]) -> &&str {
    // ... same body, with T = &str throughout
}
```

Each call site in `main` is wired directly to its own stamped-out copy. When `largest(&operands)` runs, there is no lookup, no vtable, no "which type is this?" — the `u8` version was chosen before the program existed as a binary, and the call is exactly as fast as the hand-written `u8`-only function you refused to copy-paste three times. The same happens to structs: `Register<u8>` and `Register<u16>` compile to two independent struct definitions, each with its own specialized methods. This is what the design of rustvm means by *generic code costs nothing at runtime*: you write the abstraction once and pay for it never.[2]

Set the two models side by side, because this is the chapter's picture. Python answers "which `write()` do I call?" by looking it up when the call happens — maximally flexible, paid for on every transaction of every test of every regression. Rust answers the same question once, at compile time, by generating the exact code each caller needs — and the flexibility you give up is precisely the flexibility of being wrong. Chapter 10's trait objects (`Box<dyn Animal>`, the vtable, the runtime dispatch) remain available for the cases that genuinely need runtime choice; monomorphized generics are what you get everywhere else, which in a testbench is almost everywhere.

Honesty requires the other side of the ledger, and it is real but modest: stamping out copies takes compile time and makes the binary larger. A generic function used with thirty types is compiled thirty times. For testbench code — a handful of transaction types, not thirty — you will notice this approximately never, but now you know why heavily generic Rust code compiles slower than it runs.

The destination: Driver<REQ, RSP>

Part I keeps promising that these language chapters are load-bearing, so let me show you the load. In pyuvm, `uvm_driver` had a `seq_item_port`, and what came out of `get_next_item()` was — whatever the sequencer sent. If a misconfigured test wired a memory sequence to the ALU agent, the driver discovered it at runtime, usually as an `AttributeError` deep in `run_phase()`, and the Python book taught you the discipline to avoid it.

rustvm's driver states its requirements in its name. Signatures only — Part IV builds this for real:

```
# Figure 6: The shape of rustvm's driver (preview - signatures only)

pub struct Driver<REQ, RSP = REQ> {
    pub seq_item_port: SeqItemPort<REQ, RSP>,
    // ...
}
```

Everything in this chapter is in that one line. Two type parameters, because a driver's conversation has two directions: `REQ` is the request transaction it pulls from the sequencer, `RSP` the response it may send back. Type parameters can have *defaults* — `RSP = REQ` says "if you don't name a response type, it's the same as the request," so the common no-response driver is just `Driver<AluCommand>`. And the port inside is a generic struct over the same two parameters, so the driver, its port, and the sequencer on the far end must all

agree on the transaction types *or the testbench does not compile*. The wrong-sequence-on-the-wrong-agent bug does not become an error message. It becomes unwritable.

That is the trade this book keeps making, in its purest form yet: the duck-typed flexibility of pyuvm's driver — any port, any transaction, sort it out at runtime — exchanged for a signature that is documentation, contract, and proof all at once. And thanks to monomorphization, `Driver<AluCommand>` compiles to code as direct as a driver hand-written for ALU commands alone, because that is literally what the compiler makes of it.

Summary

Generics let one definition serve many types: type parameters in angle brackets stand for a type named later, on functions (`fn largest<T: PartialOrd>(list: &[T]) -> &T`) and on structs (`Register<T>`, and the `Vec<T>`, `Option<T>`, and `Result<T, E>` you have used since Chapter 8). Trait bounds are the contract — `T: PartialOrd`, or several requirements joined with `+`, or a `where` clause when the list grows — and they replace Python's duck typing with the same idea checked at compile time: if it implements the bound, it's a duck, and the compiler verifies the feathers before the program runs. Monomorphization is why none of this costs anything at runtime: the compiler stamps out a specialized copy of the generic code for each concrete type used, so every call dispatches directly, in exchange for some compile time and binary size. And `Driver<REQ, RSP = REQ>` is where the book is taking all of it — a driver whose transaction types are part of its type, so mismatched testbench plumbing fails at compile time instead of mid-regression.

We now have types that carry proofs. What we do not yet have is Rust's way of passing *behavior* around — the thing Python did every time it handed a function to `sorted(key=...)` or stored a coroutine to run later, and the thing rustvm's factory will do for a living. Chapter 12 takes up closures and iterators, and if you enjoyed watching the compiler specialize your generics for free, you are going to like what it does to a `for` loop.

[1] When inference can't decide — or you want to be explicit — you can name the type at the call site with `largest::<u8>(&operands)`. The `::<>` operator is universally called the *turbofish*, it is official Rust culture, and no, nobody has come up with a better name. Swim on.

[2] C++ programmers will recognize this as what templates do, with one upgrade worth appreciating: a Rust generic function is type-checked once, against its bounds, when it is *defined* — not re-checked per instantiation with errors erupting from the template's guts. The compiler in figure 1 complained about our signature, in our terms, before any caller existed.

Chapter 12: Closures and Iterators

The Python book taught two features that made testbench code shorter and stranger at the same time: comprehensions, which built a whole list in one bracketed line, and generators, which used `yield` to produce values one at a time without ever building the list at all. Both were ways of saying *here is a stream of values and a recipe for making them* — and both, you may remember, took a chapter to stop looking like magic.

Rust has the same two ideas, reorganized around one feature: the **iterator**. And driving the iterator machinery is a smaller feature that this book has been saving up for eleven chapters, because it is quietly one of the most important in the language: the **closure**. Closures matter far beyond this chapter. When Part IV rebuilds the UVM factory, the mechanism that replaces the entire override registry will turn out to be a closure stored in a struct field. This is the chapter where you learn why that sentence makes sense.

> **In Python we...** created generators using the `yield` statement. "The yield statement is like the return statement in that it yields a number to the caller. Unlike return, yield continues running in the function like any other statement." We wrote `my_range()` with a `while` loop around a `yield`, and a Fibonacci generator with two `yield` statements, and used both directly in `for` loops. And in the Lists chapter we met the list comprehension — `[nn**2 for nn in range(11) if nn % 2 == 0]` — four parts in square brackets that replaced a four-line `for` loop, with dictionary and set comprehensions following the same pattern in curly braces.

Closures: functions as values

A closure is a function without a name, written inline, that can use the variables around it. Python had these in two flavors: `lambda x: x + 1` for one-liners, and nested `def` for anything longer. Rust has one syntax for both. The parameters go between vertical bars, and the body follows.

```
# Figure 1: A closure is an unnamed function in a variable

fn main() {
    let add_one = |x: u32| x + 1;

    let describe = |aa: u8, bb: u8| {
        let sum = aa as u16 + bb as u16;
        format!("{aa} + {bb} = {sum}")
    };

    println!("{}", add_one(41));
    println!("{}", describe(0xFF, 0x01));
}
```

```
--
42
255 + 1 = 256
```

`add_one` holds a function the way a variable holds a number. A single expression needs no braces; a multi-line

body takes braces and, like every Rust block, evaluates to its last expression. Notice there is no return type written on either closure — the compiler infers closure types from how you use them, which is why closures usually look lighter than `fn` declarations. In fact the parameter types are usually optional too; I wrote `x: u32` for clarity, but `let add_one = |x| x + 1;` compiles fine once the compiler sees a call that pins the type down.

So far this is `lambda` with different punctuation. The interesting part — the part Python never asked you to think about — is what happens when the closure uses a variable it did not declare.

Capture, through the ownership lens

A Python closure that mentions an outer variable just… uses it. Every Python name is a reference, so the closure captures a reference, silently, and if two pieces of code mutate the same captured object at the same time, that is your problem to discover at runtime. The Python book's NullTrigger race — two tasks sharing `transaction_data` — was exactly this bug wearing a coroutine costume.

Rust closures also capture outer variables, but here is the difference: *capturing is subject to the ownership rules from Chapters 5 and 6, like everything else.* A closure that reads a variable borrows it with `&T`. A closure that mutates one borrows it with `&mut T`. A closure that consumes one takes ownership. The compiler looks at the closure's body, picks the least drastic mode that works, and then enforces it — visibly, in the type system, at compile time.

Watch the three modes in order. First, a closure that only reads.

```
# Figure 2: A closure that reads captures by shared borrow

fn main() {
    let ops = vec!["ADD", "AND", "XOR", "MUL"];

    let show = || println!("ops under test: {ops:?}");

    show();
    show();
    println!("still mine: {} ops", ops.len());  // ops was only borrowed
}
```

```
--
ops under test: ["ADD", "AND", "XOR", "MUL"]
ops under test: ["ADD", "AND", "XOR", "MUL"]
still mine: 4 ops
```

`show` borrowed `ops` the way any `&Vec` would, so we can call it repeatedly and still use `ops` afterward. Second, a closure that mutates.

```
# Figure 3: A closure that mutates captures by exclusive borrow

fn main() {
    let mut errors = Vec::new();

    let mut log_error = |msg: &str| errors.push(msg.to_string());

    log_error("ADD result mismatch");
    log_error("XOR result mismatch");

    println!("{} errors: {errors:?}", errors.len());
}
```

```
--
2 errors: ["ADD result mismatch", "XOR result mismatch"]
```

Two things changed. The closure itself must be declared `let mut`, because calling it mutates the captured `errors` — mutation is never invisible in Rust, not even here. And while `log_error` is alive, it holds the *exclusive* borrow of `errors`; if we tried to `println!("{errors:?}")` between the two calls, the compiler would refuse, citing the aliasing-XOR-mutability rule from Chapter 6. One writer, no readers alongside. The race the Python book could only warn you about is structurally impossible to write.

Third, a closure that takes ownership. Sometimes a closure must own its captures — most often because it will outlive the scope it was created in, which is precisely the situation when you store a closure in a struct or hand it to another task. The `move` keyword forces the transfer.

```
# Figure 4: A move closure takes ownership of its captures

fn main() {
    let test_name = String::from("alu_smoke_test");

    let banner = move || format!("*** {test_name} ***");

    println!("{}", banner());
    println!("{}", test_name);  // ERROR: test_name moved into the closure
}
```

```
--
error[E0382]: borrow of moved value: `test_name`
 --> src/main.rs:8:20
  |
4 |     let banner = move || format!("*** {test_name} ***");
  |                  ------- value moved into closure here
...
8 |     println!("{}", test_name);
  |                    ^^^^^^^^^ value borrowed here after move
```

The error message tells the whole story: `test_name` moved into `banner`, and Chapter 5's rule applies — after a move, the old name is dead. Delete the last `println!` and the program compiles. This is the same monitor-hands-transaction-to-scoreboard reasoning you already know; the only novelty is that the new owner is a closure instead of a function parameter.

Rust names these three capture behaviors with three traits, and you will meet them constantly in documentation: **`Fn`** for closures that can be called any number of times through a shared borrow (figure 2), **`FnMut`** for closures that mutate and need exclusive access to call (figure 3), and **`FnOnce`** for closures that consume something and therefore can only be called once.[1] You do not choose among them by annotation — the compiler classifies each closure from its body — but you will *read* them in every function signature that accepts a closure, and later in this chapter you will write one into a struct field.

> [1] The names describe how the closure may be *called*, and they nest: every `Fn` is also an `FnMut`, and every `FnMut` is also an `FnOnce` — a closure you may call many times can certainly be called once. Interviewers love this; day-to-day code mostly just needs you to recognize the three names.

Iterator adapters: comprehensions, unrolled

Now the payoff. The Python book built `even_squares` twice — once with a `for` loop and `append()`, then in one line:

```
even_squares = [nn**2 for nn in range(11) if nn % 2 == 0]
```

A comprehension has four parts: an expression, a variable, an iterator, and a filter. Rust has no comprehension syntax. Instead it lets you take those same four parts and chain them left to right as method calls on the iterator — each method taking, naturally, a closure.

```
# Figure 5: The list comprehension, as an iterator chain

fn main() {
    let even_squares: Vec<u32> = (0..=10)
        .filter(|nn| nn % 2 == 0)
        .map(|nn| nn * nn)
        .collect();

    println!("even squares {even_squares:?}");
}
```

```
--
even squares [0, 4, 16, 36, 64, 100]
```

Read the chain aloud and it is the comprehension in sentence order: take the range zero through ten, *filter* it down to the even numbers, *map* each survivor to its square, and *collect* the results into a `Vec`. The `|nn| ...` closures are the comprehension's expression and filter parts, now explicit values passed as arguments. Where the Python book had to teach you the four positions inside the brackets, the Rust version wears its structure on the outside — and when a chain grows too clever, it splits across lines exactly as figure 5 shows, no special multi-line dispensation required.

The Python book's dictionary comprehension ports the same way. `{ii : ii**3 for ii in range(4)}` becomes a chain that maps each number to a `(key, value)` pair and collects into a `HashMap`:

```
# Figure 6: The dictionary comprehension, collected into a HashMap

use std::collections::HashMap;

fn main() {
    let cubes: HashMap<u32, u32> = (0..4)
        .map(|ii| (ii, ii.pow(3)))
        .collect();

    println!("cubes: {cubes:?}");
}
```

```
--
cubes: {2: 8, 0: 0, 3: 27, 1: 1}
```

Two things to notice. `collect()` is doing something quietly remarkable: the *same method* built a `Vec` in figure 5 and a `HashMap` here, steered entirely by the type annotation on the left — the generics machinery from Chapter 11 earning its keep. And look at that output order. Chapter 8 warned you that Rust's `HashMap`, unlike the Python dict you knew, promises nothing about iteration order, and here is the proof; your run will likely print a different scramble.

One more adapter completes the everyday set. Where `map` transforms each element and `filter` drops some, `fold` boils the whole stream down to a single value: it takes a starting accumulator and a closure that combines the accumulator with each element in turn. Here is a scoreboard-flavored example — counting mismatches in a list of (expected, actual) pairs:

```
# Figure 7: fold reduces a stream to one value

fn main() {
    let results = [(0x55u16, 0x55u16), (0x100, 0x100), (0x0FE, 0x0FF)];
```

```
    let mismatches = results
        .iter()
        .fold(0, |errs, (exp, act)| if exp == act { errs } else { errs + 1 });

    println!("mismatches: {mismatches}");
}
```

```
--
mismatches: 1
```

In truth you will reach for `fold` less often than you expect, because the standard library pre-packages the common folds: `.sum()`, `.count()`, `.max()`, `.any()`, `.all()`. The chain `results.iter().filter(|(exp, act)| exp != act).count()` does figure 7's job and reads better. But `fold` is the general case the shortcuts are made of, and knowing it makes the shortcuts unmysterious.

Lazy, like a generator

Here is the fact that connects comprehensions to generators, and it deserves its own paragraph: **iterator adapters do nothing until something consumes them.** The chain `(0..=10).filter(...).map(...)` computes no squares. It builds a small struct that *describes* the computation, and only when `collect()` — or a `for` loop, or `sum()` — starts pulling values through does any work happen, one element at a time, no intermediate list anywhere.

If that sounds familiar, it should. It is exactly the property that made Python generators worth a chapter: `my_range(1_000_000)` didn't build a million-entry list, it produced values on demand. In Rust, *every* iterator chain behaves that way by default. Python made laziness an opt-in feature with special syntax; Rust made it the only behavior and never needed the syntax.

Which raises the obvious question: what happened to `yield`?

Where generators went

Rust has no `yield` statement.[2] What it has instead is the `Iterator` trait — one required method, `next()`, which returns `Some(value)` until the stream is exhausted and `None` thereafter. That `Option` should ring a bell from Chapter 9: where Python generators signal exhaustion with a `StopIteration` exception behind the scenes, Rust signals it in the return type, in the open.

Any type that implements `Iterator` works in a `for` loop, chains with every adapter above, and collects into collections. The Python book's favorite generator was Fibonacci, so let us port it. Where Python kept `lastnumb` and `numb` alive between `yield`s inside a paused function, Rust keeps them as fields in a struct, and `next()` advances the state one step per call.

```
# Figure 8: The Fibonacci generator, as an Iterator implementation

struct Fibonacci {
    curr: u64,
    next: u64,
}

impl Iterator for Fibonacci {
    type Item = u64;

    fn next(&mut self) -> Option<u64> {
        let result = self.curr;
        self.curr = self.next;
```

```
        self.next = result + self.next;
        Some(result)
    }
}

fn main() {
    let fib = Fibonacci { curr: 0, next: 1 };
    for numb in fib.take(8) {
        print!("{numb} ");
    }
    println!();
}
```

```
--
0 1 1 2 3 5 8 13
```

The state that Python hid in a suspended stack frame is now a two-field struct you can see, and the resumption that Python performed by magic is now an ordinary method call. Notice that this iterator never returns `None` — it is *infinite*, which is fine, because it is lazy; `.take(8)` is an adapter that cuts the stream off after eight values. An infinite generator was a slightly daring trick in Python. In Rust it is Tuesday.

Writing an `impl Iterator` block for every one-off stream would get old, though, and here Chapter 11's `impl Trait` makes its second appearance — this time in a return position. A function can declare that it returns *some* iterator without naming the struct, and inside, you build that iterator out of ranges and adapters. This is the closest Rust idiom to "a function with `yield` in it," and it is how this book will write value streams from here on.

Suppose a test wants every combination of small A and B operands for the TinyALU. In Python:

```
def operand_pairs(n):
    for aa in range(n):
        for bb in range(n):
            yield (aa, bb)
```

And in Rust, as a function returning `impl Iterator`:

```
# Figure 9: A TinyALU operand-pair stream, replacing a generator function

fn operand_pairs(n: u8) -> impl Iterator<Item = (u8, u8)> {
    (0..n).flat_map(move |aa| (0..n).map(move |bb| (aa, bb)))
}

fn main() {
    for (aa, bb) in operand_pairs(3) {
        print!("({aa},{bb}) ");
    }
    println!();
}
```

```
--
(0,0) (0,1) (0,2) (1,0) (1,1) (1,2) (2,0) (2,1) (2,2)
```

`flat_map` is the nested-loop adapter: for each `aa` it produces a whole inner stream and splices the streams end to end. And look — there are our two `move` keywords from figure 4, no longer a curiosity. The inner closure must *own* its copy of `aa`, and the outer must own `n`, because these closures ride out of the function inside the returned iterator and live on long after `operand_pairs`'s local variables are gone. The capture rules you learned through the ownership lens are what make it safe to return a paused computation from a

function. Python's generators did the same thing by keeping the whole stack frame alive on the heap; Rust does it by moving exactly the values needed, and the compiler will name each one if you forget.

[2] Generator syntax has been experimented with in unstable Rust for years, but stable Rust — the Rust this book teaches — does not have it, and honestly, between adapters and `impl Iterator`, you will rarely feel the gap.

Closures you keep: a seed for Part IV

Everything so far has passed closures *downward* — into `map`, into `filter`, used and forgotten. The last idea in this chapter is the one with the longest reach in this book: a closure is a value, and like any value, it can be **stored in a struct field** and called later, by code that has no idea what the closure does inside.

Because every closure has its own unwritable type, storing one takes a trait object — Chapter 10's `dyn`, boxed up: `Box<dyn Fn(u8, u8) -> u16>` is "some heap-allocated callable, taking two `u8`s, returning a `u16`; I don't know or care which one." Here is a toy checker whose *prediction function is data*:

```
# Figure 10: A struct that carries its behavior as a closure

struct Checker {
    predict: Box<dyn Fn(u8, u8) -> u16>,
}

impl Checker {
    fn check(&self, aa: u8, bb: u8, actual: u16) {
        let expected = (self.predict)(aa, bb);
        if expected == actual {
            println!("PASS: ({aa}, {bb}) -> {actual}");
        } else {
            println!("FAIL: ({aa}, {bb}) expected {expected}, got {actual}");
        }
    }
}

fn main() {
    let adder_check = Checker {
        predict: Box::new(|aa, bb| aa as u16 + bb as u16),
    };
    adder_check.check(0xFF, 0x01, 0x100);

    let and_check = Checker {
        predict: Box::new(|aa, bb| (aa & bb) as u16),
    };
    and_check.check(0x0F, 0x35, 0x0006);  // wrong on purpose
}
```

```
--
PASS: (255, 1) -> 256
FAIL: (15, 53) expected 5, got 6
```

Two `Checker` values, one struct definition, two completely different behaviors — selected not by inheritance, not by overriding a virtual method, but by *which closure was placed in the field at construction time*. Sit with that for a moment, because it is the seed of something large. The UVM factory — the machinery the Python book spent a chapter on, with its registries and overrides, `create()` calls and override tables — exists to answer one question: *how does a test change what the testbench builds without editing the testbench?* Python and SystemVerilog needed a registry because they could not comfortably pass constructors around

as values. Rust can. When Chapter 29 rebuilds the factory's job, the answer will be a config struct carrying closure fields much like `predict` — a constructor in a box, replaced by the test in three visible lines, checked end to end by the compiler. You now hold the entire mechanism; Part IV supplies the methodology.

Summary

Closures are unnamed functions in variables: `|x| x + 1`, with braces for multi-line bodies and types mostly inferred. They capture surrounding variables under the ordinary ownership rules — shared borrow to read, exclusive borrow to mutate, ownership when `moved` — and the traits `Fn`, `FnMut`, and `FnOnce` name those three calling contracts in signatures. Iterator adapter chains — `filter`, `map`, `flat_map`, `fold`, `collect` — replace Python's list, set, and dictionary comprehensions part for part, and they are lazy by default, doing no work until consumed. Python's generators map to the `Iterator` trait: implement `next()` on a struct for full control, or return `impl Iterator` built from adapters for the everyday case, with `move` closures carrying the captured state out of the function. Finally, closures are values: boxed as `Box<dyn Fn(...)>`, they can live in struct fields and be swapped at construction time — the mechanism Chapter 29 will grow into rustvm's replacement for the UVM factory.

That `Box` in figure 10 was the first time this book put a value on the heap on purpose, and I slipped it past you with one sentence of explanation. It deserves better — because `Box` has two siblings, `Rc` and `RefCell`, and among them they answer the question every pyuvm refugee eventually asks: *how do two components share one scoreboard?* Chapter 13 pays that debt.

Chapter 13: Smart Pointers: Box, Rc, and RefCell

> **In Python we...** protected attributes instead of values. The Python book's "Protecting attributes" chapter told the story of a `Temperature` class and a user who kept reaching past its methods to poke `tt.temp` directly — first deterred by a single underscore (a convention, unenforced), then by double-underscore name mangling (enforced, grudgingly), and finally civilized by `@property` accessors that let the class decide *how* its data gets read and written. The whole chapter was about one question: who gets to touch this value, and on whose terms? Python answered with naming conventions and accessor methods, because it could not answer with the language itself. *(book: "Protecting attributes" chapter)*

Chapters 5 and 6 handed you two rules and told you the rest of the book stands on them: every value has exactly one owner, and at any moment a value may have many readers or one writer, never both. Since then, you may have been quietly carrying a worry. You wrote pyuvm testbenches. You *know* what a testbench looks like inside: an environment holding an agent, a monitor and a scoreboard both holding the same BFM, an analysis port fanning one transaction out to three subscribers. Shared access everywhere, and — be honest — shared *mutable* access more often than the diagrams admit. If Rust's rules are absolute, how does any of that survive the port?

The answer is that the rules are absolute but the *enforcement point* is negotiable. Rust provides a small set of standard-library types — the community calls them **smart pointers** — that let you buy back Python-style sharing, one capability at a time, each purchase visible in your code and each with a price tag attached. This chapter teaches the three you will actually meet: `Box<T>`, `Rc<T>`, and `RefCell<T>`. By the end you will be able to read `Rc<RefCell<T>>` and recognize an old friend wearing a name tag: a Python object reference, made visible.

`Box<T>`: the heap, on request

The simplest smart pointer barely earns the name. `Box<T>` puts a value on the heap and owns it — one owner, moved and dropped exactly like everything in Chapter 5. The only thing that changed is *where the bytes live*.

```
# Figure 1: A Box owns its value on the heap - everything else is Chapter 5

struct Transaction {
    a: u8,
    b: u8,
    op: u8,
}

fn main() {
    let t = Box::new(Transaction { a: 5, b: 3, op: 1 });
```

```
    println!("boxed transaction: {} op {}", t.a, t.b);
}   // t goes out of scope; the Box is dropped; the heap memory is freed.
    // One owner, one drop, on time - nothing new.
```

```
--
boxed transaction: 5 op 3
```

Notice `t.a` works without ceremony — a `Box` hands through field access as if the box weren't there. So why would you ever ask for one? In Python you never chose; every object lived on the heap and every variable was a pointer to it. Rust defaults to the stack and lets you opt into the heap, and there are two situations where you must. The first is a type whose size the compiler cannot pin down — a recursive type, like a linked list whose node contains another node; boxing the inner node gives the compiler a fixed-size pointer to reason about. The second you have already met: **trait objects**. Chapter 10 introduced `Box<dyn Component>` as the way to store "some type implementing `Component`, decided at runtime" — and since the compiler cannot know that type's size, onto the heap it goes, behind a `Box`.

That is genuinely the whole story. `Box<T>` is single ownership with a heap address. When you see one in rustvm — `Box<dyn DriverLike>` in a config struct, say — read it as "an owned value whose concrete type or size wasn't known at compile time" and move on. The interesting escape hatches are the next two.

`Rc<T>`: Python's refcount, made visible

Here is a piece of CPython trivia that is about to stop being trivia: every object you ever created in Python carried a hidden integer — a **reference count** — and every assignment, argument pass, and container insert incremented it, while every scope exit and `del` decremented it. When the count hit zero, the object died.[1] You never saw this machinery; it ran on every single Python statement you ever executed, silently, whether you needed it or not. We can even catch it in the act:

```
# Figure 2: The refcount Python never showed you

import sys

a = ["ADD", 5, 3]
print(sys.getrefcount(a))
b = a
print(sys.getrefcount(a))
```

```
--
2
3
```

(`getrefcount` reports one higher than you'd guess, because the act of calling it lends the list one more temporary reference. Even the inspection tool participates in the scheme.)

Rust's `Rc<T>` — *reference counted* — is exactly this mechanism, with two differences: you opt into it per value instead of paying for it everywhere, and the increments happen only where you can see them. `Rc::new` wraps a value and starts the count at one. `Rc::clone` does *not* copy the value — it copies the *handle* and increments the count, which is precisely what Python's `b = a` did behind your back. When each `Rc` handle is dropped, the count decrements; at zero, the value is dropped. It is CPython's memory management, offered à la carte.

```
# Figure 3: Rc::clone increments a refcount - on purpose, where you can see it

use std::rc::Rc;

fn main() {
```

```
    let bfm = Rc::new(String::from("TinyALU BFM"));
    println!("owners: {}", Rc::strong_count(&bfm));

    let driver_handle = Rc::clone(&bfm);
    {
        let monitor_handle = Rc::clone(&bfm);
        println!("owners: {}", Rc::strong_count(&bfm));
        println!("monitor sees: {monitor_handle}");
    }   // monitor_handle dropped here: count decrements

    println!("owners: {}", Rc::strong_count(&bfm));
    println!("driver sees: {driver_handle}");
}
```

```
--
owners: 1
owners: 3
monitor sees: TinyALU BFM
owners: 2
driver sees: TinyALU BFM
```

Read figure 3 as a negotiation with Chapter 5. That chapter's rule — one value, one owner — has not been repealed; it has been *satisfied cleverly*. The `Rc` bookkeeping block is the value with the single owner story; the handles share it, and "who frees this?" is answered mechanically: the last handle out turns off the lights. This is why the method is spelled `Rc::clone(&bfm)` rather than hiding inside an assignment — the Rust convention is to make every refcount bump greppable, so that when you audit a testbench you can find each place ownership got shared and ask whether it earned its keep. Python bumped refcounts on every line and could not tell you where; Rust bumps them only where the code says `Rc::clone`.

[1] Mostly. CPython also runs a cycle-detecting garbage collector to rescue objects that point at each other and hold their mutual counts above zero forever. Hold that thought — `Rc` has the same weakness and no rescue squad, and it is going to matter later in this chapter.

So can several components share a BFM this way? Almost. There is a catch, and it is the same catch Chapter 6 stamped into `&T`: sharing is a *reader's* privilege.

```
# Figure 4: Shared owners are readers - Rc will not hand out write access

use std::rc::Rc;

struct Scoreboard {
    errors: u32,
}

fn main() {
    let sb = Rc::new(Scoreboard { errors: 0 });
    let handle = Rc::clone(&sb);
    handle.errors += 1;
    println!("errors: {}", sb.errors);
}
```

```
--
error[E0594]: cannot assign to data in an `Rc`
 --> src/main.rs:9:5
  |
9 |     handle.errors += 1;
```

```
  |     ^^^^^^^^^^^^^^^^^^ cannot assign
  |
  = help: trait `DerefMut` is required to modify through a dereference,
          but it is not implemented for `Rc<Scoreboard>`
```

Of course it refused. Two handles alive means two potential accessors, and *many readers or one writer, never both* applies to shared owners exactly as it applied to references. `Rc<T>` gives you Python's sharing but only the reading half of Python's behavior. For the writing half, we need the strangest and most instructive type in this chapter.

`RefCell<T>`: the borrow checker, moved to runtime

Everything the borrow checker has done so far, it has done at compile time, by proving things about your source code. But some sharing patterns are true in ways a compiler cannot see from the source — *these two components take turns, I promise* — and for those, Rust offers a deal: **`RefCell<T>` keeps the borrowing rules but checks them at runtime.** Same law, different courtroom.

A `RefCell<T>` wraps a value and replaces `&`/`&mut` with two accessor methods: `borrow()` returns a read handle, `borrow_mut()` returns a write handle, and the cell *counts* its outstanding loans. Many readers, or one writer — enforced by a counter at the door instead of a proof in the compiler.

```
# Figure 5: Interior mutability - mutation through an immutable binding

use std::cell::RefCell;

struct Scoreboard {
    errors: u32,
}

fn main() {
    let sb = RefCell::new(Scoreboard { errors: 0 });   // note: no `mut`

    sb.borrow_mut().errors += 1;    // the write handle lives for this line only

    println!("errors: {}", sb.borrow().errors);
}
```

```
--
errors: 1
```

Look hard at the first line of `main`: there is no `mut`. Since Chapter 3, that has meant untouchable — yet we mutated `errors` anyway. This trick has a name, **interior mutability**: from the outside, `sb` is an immutable value you can share freely; on the inside, the cell hands out exclusive write access to one caller at a time, checked at the moment of the call. If that sounds familiar, it should — it is the "Protecting attributes" chapter's move, replayed at the level of the type system. Python's `Temperature` class hid `__temp` behind properties so the class could enforce its rules at every access; `RefCell` hides its value behind `borrow` and `borrow_mut` so the *borrowing* rules get enforced at every access. Both designs answer the same question — who gets to touch this value, and on whose terms? — by making all traffic pass through a gatekeeper. The difference is what the gatekeeper checks: Python's properties checked whatever validation you wrote by hand; `RefCell` checks aliasing XOR mutability, the one rule this whole language is built on.

And what happens when the rule is violated? At compile time, a violation was a program that never existed. At runtime, the program exists — it is halfway through a simulation — so the only honest response left is to stop:

```
# Figure 6: The borrow checker at runtime - a panic replaces the compile error
```

```rust
use std::cell::RefCell;

struct Scoreboard {
    errors: u32,
}

fn main() {
    let sb = RefCell::new(Scoreboard { errors: 0 });

    let reader = sb.borrow();          // a reader is at the whiteboard...
    let mut writer = sb.borrow_mut();  // ...and a writer grabs the marker

    writer.errors += 1;
    println!("reader saw: {}", reader.errors);
}
```

```
--
thread 'main' panicked at src/main.rs:12:25:
already borrowed: BorrowMutError
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

Set figure 6 next to Chapter 6's figure 3. They are the *same program* — one value, a live reader, a live writer, overlapping. In Chapter 6 the compiler rejected it with error E0502 and three annotated line numbers, before anything ran. Here it compiled without a murmur and then panicked at line 12, at runtime. That is the entire trade, in two figures: `RefCell` does not weaken the rule — a reader and a writer still cannot coexist, and the race Chapter 6 buried stays buried — but it moves the *discovery* of a violation from your desk to the running program. In a testbench, "the running program" means seed 8,441, forty minutes in, with the license checked out. You have not escaped the borrow checker. You have volunteered to meet it later, at a worse time, with a stack trace instead of a source annotation.[2]

That framing tells you exactly when `RefCell` is legitimate: when you *know* the accesses cannot overlap — because your components take turns at await points, because the write handle lives for one expression as in figure 5 — but the knowledge lives in the design rather than in anything the compiler can verify from the source. Then the runtime check is not a time bomb; it is an assertion, permanently guarding an invariant you believe, and the panic is the assertion firing on the day you turn out to be wrong.

> [2] A panic in a rustvm task does not take down the simulator, for the record — it is caught at the task boundary and scored as a test failure, the same way cocotb catches a stray exception per task. Cold comfort at seed 8,441, but comfort.

`Rc<RefCell<T>>`: a Python reference, made visible

Now stack them. `Rc` gave us shared ownership with read-only access; `RefCell` gave us gate-checked mutation of a shared value. Compose them — `Rc<RefCell<T>>` — and you have shared handles, any of which can mutate the value, with the borrow rules enforced at each access. Which is to say: you have rebuilt the Python object reference, the thing every variable in every Python program you ever wrote actually was.

Chapter 5 opened with a Python figure that proved `b = a` gives two names for one mutable object, and then showed Rust refusing to compile the same experiment. We have been in debt to that figure for eight chapters. Time to pay it off:

```rust
# Figure 7: Chapter 5's Python experiment, finally legal in Rust - with its costs itemized

use std::cell::RefCell;
use std::rc::Rc;
```

```rust
#[derive(Debug)]
struct Transaction {
    op: String,
    a: u8,
    b: u8,
}

fn main() {
    let a = Rc::new(RefCell::new(Transaction {
        op: String::from("ADD"),
        a: 5,
        b: 3,
    }));
    let b = Rc::clone(&a);          // two names...

    b.borrow_mut().op = String::from("MUL");   // ...mutate through one...

    println!("{:?}", a.borrow());              // ...observe through the other
    println!("same object: {}", Rc::ptr_eq(&a, &b));
}
```

```
--
Transaction { op: "MUL", a: 5, b: 3 }
same object: true
```

Line for line, this is Chapter 5's figure 1: two names, one object, a mutation through `b` visible through `a`, and `Rc::ptr_eq` standing in for Python's `is`. What Python gave you invisibly and unconditionally, Rust sells you piecewise, each purchase named in the source: `Rc::new` (this value will be shared), `Rc::clone` (here is another owner), `borrow_mut()` (I want the marker, check me at the door), `borrow()` (just reading, count me). A reviewer can see every one of those decisions. In the Python version, there was nothing to see — which is precisely why the Python book needed a chapter begging users not to touch `_temp`.

The bill

I owe you the honest price list, because "just wrap it in `Rc<RefCell<>>`" is the single most common way for a new Rust programmer to dig a hole.

Runtime checks that can panic. Every `borrow()` and `borrow_mut()` is a check that can fail, and figure 6 showed what failure looks like: a panic, at runtime, in whatever seed happens to trip it. The compile-time guarantee you have enjoyed since Chapter 6 is gone for this value; you are back to *testing* for the bug instead of being proven free of it.

Bookkeeping overhead. Refcount increments, borrow-flag checks — each is tiny, and unlike Python you pay only on the values you wrapped. But "tiny" is a per-access word, and monitors touch shared state per transaction, per seed, per regression. It adds up honestly; budget for it consciously.

Reference cycles leak. Here the footnote from earlier comes due. CPython backs its refcounts with a cycle-collecting garbage collector; `Rc` has no such backstop. If a parent holds an `Rc` to its child and the child holds an `Rc` back to the parent, the counts never reach zero and the memory never frees — a leak, silent and permanent. (The standard library's `Weak` type exists to break such cycles; rustvm's design never needs it, for reasons one section away, so this book leaves it at a mention.)

The temptation. This is the real cost. Once you know `Rc<RefCell<T>>` exists, every borrow-checker error acquires an easy exit: wrap it, clone it, move on. Do that habitually and you will have written Python with worse syntax — every shared value a potential runtime panic, every refcount a small tax, and the compiler's proofs traded away for the debugging sessions this book keeps promising you left behind. The discipline is

the same one the Python book taught for private attributes, pointed the other way: reach for the escape hatch deliberately, at a designed boundary, and let the default remain the default.

Why rustvm barely needs any of this

Which brings us to the question this chapter has been building toward: how much `Rc<RefCell<T>>` will the testbenches in Part IV actually contain? You know how pyuvm is built — every child holds `_parent`, every parent holds `_children`, and a global `component_dict` holds a reference to everything. That is a graph full of exactly the parent-and-child cycles that make `Rc` leak, and it works in Python only because the garbage collector untangles what the refcounts cannot. Port that structure naively and you would be signing up for `Rc<RefCell<T>>` everywhere, `Weak` back-references to break the cycles, and a runtime borrow check on every component access — Python's architecture with Rust's ceremony, the worst page of both books.

rustvm's design refuses the premise: **the component tree *is* the ownership tree** (design doc: §0.2 and the component-hierarchy discussion, §5.2). An environment is a struct; its children are its fields; the parent owns them the way any struct owns its fields, and drops them at its own drop, in the way you have understood since Chapter 5. There is no back-pointer to a parent, no global registry, no cycle — so there is nothing for a refcount to get wrong and no shared mutation for a `RefCell` to referee. When the monitor needs to hand the scoreboard a transaction, it will not reach through a shared reference to poke scoreboard state; it will send the transaction down a channel, moving ownership the way Chapter 5's baton pass always wanted to (design doc: §5.6). The hierarchy that forced Python into pervasive implicit sharing simply is not shaped that way here.

What survives is the genuinely shared resource — the thing that really does have several users and really is one object. The canonical example, previewed now and built in Part IV: one `TinyAluBfm` wrapping the DUT interface, needed by a driver *and* a monitor at once. The design doc's answer is a config struct carrying `bfm: Rc<TinyAluBfm>` — the test creates the BFM once, and each component that needs it receives a counted handle through its constructor (design doc: §5.4). Notice which half of this chapter that uses: `Rc` alone, no `RefCell`, because the BFM's async methods take `&self` and the sharing is read-shaped from the outside. Shared mutability, where it appears in rustvm at all, is deliberate, boundary-marked, and rare — every use called out in the design rather than ambient in the architecture (design doc: §0.2). The escape hatch exists, you now know exactly what it costs, and the framework's job is to make sure you almost never reach for it.

That completes the Rust you need for values: how they are owned, borrowed, shaped, collected, and — this chapter — shared on purpose. What you do not yet know is how Rust programs are *organized*: where `use std::rc::Rc` has been coming from all this time, what a crate actually is, and how `cargo` turns a directory of files into the testbench library a simulator can load. Chapter 14 is about modules, crates, and cargo — and it ends with something Python never quite gave you: unit tests that run in milliseconds, no simulator required.

Chapter 14: Modules, Crates, and Cargo

We have spent thirteen chapters writing Rust in single files, the way the Python book spent its early chapters typing into Jupyter cells. Real testbenches do not live in single files. The Python book's testbenches grew a `tinyalu_utils.py` holding the `Ops` enum, the prediction function, and the logger setup, and every testbench imported it. This chapter is about where code *lives* in Rust — modules within a project, crates between projects, and `cargo` orchestrating all of it — and it ends with a payoff I have been promising since Chapter 1: running real unit tests against testbench logic with no simulator anywhere in sight.

> **In Python we...** imported types and values from external files called *modules*, and called a directory of modules a *package* — "cocotb, for example, is delivered as a package." The `import` statement ran the module's code and added the module's name to our scope; `from pyuvm import FIFO_DEBUG` pulled a single name in; and `from pyuvm import *` pulled in everything, PEP-8 be damned, so that Python UVM code would look like SystemVerilog UVM code. Python found modules by searching the directories listed in `sys.path`, and when it couldn't find `tinyalu_utils`, we helped it: `sys.path.insert(0, str(Path("..").resolve()))`.

Hold on to that last line, because it is the one this chapter deletes. Rust has no `sys.path`, no import-time code execution, and no possibility of a testbench that works on your machine but not on the farm because of a path environment variable. What Rust has instead is more ceremonial — I will not pretend otherwise — and in exchange the compiler knows exactly where every name comes from and exactly who is allowed to use it.

Modules: namespaces inside a crate

A **module** in Rust is a named scope you declare with the `mod` keyword. Unlike Python, where every file automatically *is* a module, a Rust module is something you declare explicitly — and you can declare one right in the middle of a file. Let's start there, because it makes the concept visible before any files get involved.

We will need a home for the TinyALU's prediction logic — the pure function that computes what the DUT *should* produce. This was `tinyalu_utils.py`'s most important resident, and it will be our example for the rest of the chapter.

```
# Figure 1: A module declared inline, in the middle of main.rs

mod predictor {
    #[derive(Clone, Copy, Debug, PartialEq)]
    pub enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

    pub fn alu_prediction(a: u8, b: u8, op: Ops) -> u16 {
        match op {
            Ops::Add => a as u16 + b as u16,
```

```
            Ops::And => (a & b) as u16,
            Ops::Xor => (a ^ b) as u16,
            Ops::Mul => a as u16 * b as u16,
        }
    }
}

fn main() {
    let sum = predictor::alu_prediction(0xFF, 0x01, predictor::Ops::Add);
    println!("0xFF + 0x01 = {sum:#06x}");
}
```

```
--
0xFF + 0x01 = 0x0100
```

Everything between the braces of `mod predictor` lives in the `predictor` namespace, and code outside reaches it with `::` — `predictor::alu_prediction` — exactly the job Python's `.` did in `pyuvm.FIFO_DEBUG`. Note in passing that the prediction itself is honest about widths in a way the Python version never had to be: the TinyALU's operands are `u8` and its result bus is sixteen bits wide, so `Add` and `Mul` cast up to `u16` *before* operating. `0xFF + 0x01` carries into bit eight instead of wrapping to zero. Chapter 3 planted that seed; here it flowers.

use: bringing paths into scope

Writing `predictor::Ops::Add` at every call site gets old, and Rust's answer is the `use` declaration — the direct descendant of Python's `from ... import ...`.

```
# Figure 2: use brings names into scope, like Python's from-import

mod predictor {
    #[derive(Clone, Copy, Debug, PartialEq)]
    pub enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

    pub fn alu_prediction(a: u8, b: u8, op: Ops) -> u16 {
        match op {
            Ops::Add => a as u16 + b as u16,
            Ops::And => (a & b) as u16,
            Ops::Xor => (a ^ b) as u16,
            Ops::Mul => a as u16 * b as u16,
        }
    }
}

use predictor::{alu_prediction, Ops};

fn main() {
    println!("AND: {:#06x}", alu_prediction(0xF0, 0x3C, Ops::And));
    println!("XOR: {:#06x}", alu_prediction(0xF0, 0x3C, Ops::Xor));
}
```

```
--
AND: 0x0030
XOR: 0x00cc
```

The mapping to Python is nearly one-to-one:

Python | Rust |

|—|—| | `import pyuvm` | *(nothing needed — see below)* | | `import pyuvm as p` | `use pyuvm as p;` (aliasing works the same way) | | `from pyuvm import FIFO_DEBUG` | `use pyuvm::FIFO_DEBUG;` | | `from pyuvm import *` | `use pyuvm::*;` (a *glob import*) |

The first row deserves a word. In Python, `import` did two jobs: it *ran the module's code* and it added a name to your scope. Rust's `use` does only the second, because there is no first — modules do not "run" when you name them. All the code in every module of your program was compiled together before the program started; `use` is purely a naming convenience, with no import-time side effects, no circular-import deadlocks, and no module-level code sneaking configuration in behind your back.[1]

Rust's style community shuns glob imports for the same reason PEP-8 did — nobody reading the file can tell where a name came from. And it carves out the same exception the Python book carved out for `from pyuvm import *`: crates may export a **prelude**, a curated module explicitly designed to be glob-imported. You have been using one all along — `std`'s prelude is why `String`, `Vec`, and `Option` never needed a `use`. When we reach Part II, `use rustvm::prelude::*;` will open every testbench, doing the job `from pyuvm import *` did in the last book — one line, sanctioned by convention, because a testbench that starts by importing its methodology library is a testbench you can read.

[1] Readers who have debugged a cocotb testbench that behaved differently depending on *import order* may pause here for a private moment of celebration.

pub: privacy the compiler enforces

You may have noticed the `pub` keywords sprinkled through figures 1 and 2. They are not decoration. In Rust, **everything in a module is private by default** — invisible to code outside the module — and `pub` is how an item opts into being part of the module's public interface.

Recall how the Python book handled this. Python has no private anything: every name in every module is importable by anyone, and the convention is that an underscore prefix (`_my_helper`) means *please don't*. The book was frank that this is etiquette, not enforcement — pyuvm's internals are full of underscored names that nothing actually stops you from reaching. Rust replaces the etiquette with a compile error. Let's provoke one. Suppose the predictor grows a private helper that the outside world has no business calling:

```
# Figure 3: Private by default - the underscore convention, enforced

mod predictor {
    pub enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

    fn widen(x: u8) -> u16 {     // no pub: private to this module
        x as u16
    }

    pub fn alu_prediction(a: u8, b: u8, op: Ops) -> u16 {
        match op {
            Ops::Add => widen(a) + widen(b),
            Ops::And => widen(a & b),
            Ops::Xor => widen(a ^ b),
            Ops::Mul => widen(a) * widen(b),
        }
    }
}

fn main() {
    let w = predictor::widen(0xFF);  // reaching for a private helper
```

```
    println!("{w}");
}
```

```
--
error[E0603]: function `widen` is private
  --> src/main.rs:20:24
   |
20 |     let w = predictor::widen(0xFF);  // reaching for a private helper
   |                        ^^^^^ private function
   |
note: the function `widen` is defined here
  --> src/main.rs:5:5
   |
5  |     fn widen(x: u8) -> u16 {     // no pub: private to this module
   |     ^^^^^^^^^^^^^^^^^^^^^^
```

Delete the offending line and the program compiles; `alu_prediction`, living *inside* the module, calls `widen` freely. This is encapsulation with teeth. When you mark a helper private, you are not requesting politeness — you are making a promise the compiler keeps for you: *nothing outside this module depends on this function, so I may rewrite it tomorrow without checking.* Refactoring the guts of a monitor stops requiring a grep across the testbench for who might have reached in. Note also that privacy applies to struct *fields* individually: a `pub struct` can keep its fields private, forcing outsiders through its methods — which is where the Python book's "Protecting attributes" chapter went, and why Chapter 13's discussion of controlled access kept using the word *visibility*.

Modules in files: `mod foo;` finds `foo.rs`

Inline modules taught the concept, but real code puts modules in files. Here Rust asks for one more line of ceremony than Python did — and the line matters, so let's be precise about it.

In Python, creating `tinyalu_utils.py` *was* creating a module; the filesystem was the module system. In Rust, a file does not become part of your program until some module declares it. Writing `mod predictor;` — with a semicolon, no braces — tells the compiler: *there is a module named `predictor`, and its body lives in the file `predictor.rs` next to me.* Our playground project, reorganized:

```
# Figure 4: The file-to-module mapping

alu_playground/
   Cargo.toml
   src/
       main.rs          <-- contains the line: mod predictor;
       predictor.rs     <-- the body of the module: Ops, alu_prediction
--
% cargo run
   Compiling alu_playground v0.1.0
    Finished `dev` profile
     Running `target/debug/alu_playground`
AND: 0x0030
XOR: 0x00cc
```

Everything that was inside `mod predictor { ... }` moves into `src/predictor.rs` (dropping the wrapper braces and one level of indentation), `main.rs` declares `mod predictor;`, and nothing else changes — the `use` lines, the calls, the visibility rules are exactly as before. If `predictor` later needs child modules of its own, it graduates to a directory: `src/predictor.rs` alongside `src/predictor/history.rs`, with `mod history;` declared inside `predictor.rs`. (You will also meet an older layout, `predictor/mod.rs`, in existing code;

the two styles are equivalent, and the older one's chief legacy is an editor tab bar reading `mod.rs`, `mod.rs`, `mod.rs`.[2])

Why the extra declaration? Because in Rust the *module tree is part of the program*, not an emergent property of whatever files happen to be on a search path. There is no `sys.path` to populate, no `PYTHONPATH` to export on every farm machine, no "works in this directory, fails in that one." The compiler starts at the crate root — `main.rs` for a program, `lib.rs` for a library — follows the `mod` declarations outward, and compiles precisely those files. A file nobody declares is not compiled at all. It is more ceremony than Python's file-is-a-module simplicity, one honest line more per module, and what the line buys is that your program's structure is written down in the program.

[2] A directory full of files all named `mod.rs` is nobody's favorite piece of Rust history. Use the `predictor.rs`-plus-directory style in new code.

Crates: the unit of compilation and sharing

One level up from modules sits the **crate** — the thing `cargo new` has been making for us all along. A crate is Rust's unit of compilation and distribution: the compiler compiles a crate at a time, and crates are what you publish, version, and depend on. Where Python drew a fuzzy line between "module," "package," and "distribution" (three concepts, two of which share the name *package*), Rust draws one line: a crate is a tree of modules with a single root, and it compiles to a single artifact.

Crates come in two flavors you already half-know. A **binary crate** has a `main.rs` with a `fn main()` and compiles to a program — every playground so far. A **library crate** has a `lib.rs` and no `main`; it compiles to a library for other crates to use. cocotb and pyuvm are the Python analogs of library crates; your testbench was the analog of a binary crate importing them.

When a project outgrows one crate, cargo scales up with a **workspace**: several crates developed side by side in one repository, sharing a build directory and a single lock file. This is not a distant abstraction — it is the shape of the very tools this book builds toward. `rustvm-sim` (the cocotb analog) and `rustvm` (the pyuvm analog) live as separate crates in one workspace, for the same reason cocotb and pyuvm are separate packages: you can write a Part II-style testbench against `rustvm-sim` alone, no UVM machinery in sight, and the dependency arrow only points one way. Your own testbenches, though, stay simple — one crate, depending on published ones. Which raises the question: *how* does a crate depend on another? That is `Cargo.toml`'s job.

Cargo.toml: the manifest

Every `cargo new` wrote a `Cargo.toml` we have been politely ignoring for thirteen chapters. It is the crate's **manifest** — the one file that says what the crate is and what it needs. Time to look inside, and to add our first dependency while we're there. Python's `random` came in the standard library; Rust's standard library is deliberately lean, and random numbers live in a crate called `rand` on **crates.io**, the community registry that plays the role of PyPI. Where Python needed `pip install` (into the right virtual environment, remember) plus a line in `requirements.txt`, cargo does both jobs with one command:

```
# Figure 5: Adding a dependency with cargo add

% cargo add rand
    Updating crates.io index
      Adding rand v0.9.1 to dependencies
--
# Cargo.toml, after:

[package]
name = "alu_playground"
```

```
version = "0.1.0"
edition = "2024"

[dependencies]
rand = "0.9.1"
```

Two sections, both readable at sight. `[package]` names and versions the crate itself. `[dependencies]` lists what it needs — and this section *is* the requirements file, living in the project, checked into version control, impossible to forget on the farm machines. The next `cargo build` (or `run`, or `test`) downloads `rand`, compiles it, and links it; there is no separate install step, no environment to activate, and no way to run against a different version than the manifest declares. Alongside the manifest cargo maintains `Cargo.lock`, recording the *exact* versions resolved, so that every machine that builds this crate builds it with identical dependencies — the reproducibility that `pip freeze` approximated, produced automatically and kept current. With `rand` declared, using it is just a path — `rand::random::<u8>()` gives the random operands our tests are about to want. No import dance; the dependency's name is the root of its module tree, available everywhere in your crate.

The payoff: `cargo test`

Now the capability I flagged in Chapter 1 as worth the price of admission. It has been sitting quietly inside cargo the whole time, waiting for us to have code worth testing. We do: `alu_prediction` is a pure function — values in, value out, no DUT, no signals, no simulator. In the Python testbench, logic like this could only be exercised by running the whole cocotb stack against the simulator, or by setting up pytest scaffolding alongside it, which the book's flow never did. In Rust, testing it is built into the language and the tool. You write functions marked `#[test]`, and `cargo test` finds and runs them. The convention is a `tests` submodule at the bottom of the file whose code it tests:

```
# Figure 6: Unit tests live beside the code they test

// src/predictor.rs

#[derive(Clone, Copy, Debug, PartialEq)]
pub enum Ops { Add = 1, And = 2, Xor = 3, Mul = 4 }

pub fn alu_prediction(a: u8, b: u8, op: Ops) -> u16 {
    match op {
        Ops::Add => a as u16 + b as u16,
        Ops::And => (a & b) as u16,
        Ops::Xor => (a ^ b) as u16,
        Ops::Mul => a as u16 * b as u16,
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn add_carries_into_bit_eight() {
        assert_eq!(alu_prediction(0xFF, 0xFF, Ops::Add), 0x01FE);
    }

    #[test]
    fn and_masks_operands() {
```

```
        assert_eq!(alu_prediction(0xF0, 0x3C, Ops::And), 0x0030);
    }

    #[test]
    fn xor_finds_differing_bits() {
        assert_eq!(alu_prediction(0xF0, 0x3C, Ops::Xor), 0x00CC);
    }

    #[test]
    fn mul_needs_the_full_result_bus() {
        assert_eq!(alu_prediction(0xFF, 0xFF, Ops::Mul), 0xFE01);
    }
}
```

Every piece of this figure is machinery you already own. `mod tests` is an inline module, straight from figure 1. `use super::*;` is a glob import whose path `super` means "my parent module" — it pulls `Ops` and `alu_prediction` into the tests' scope, and it is the second sanctioned use of a glob import, because a test module importing everything it tests is exactly as readable as a testbench importing its methodology library. `#[cfg(test)]` tells the compiler to build this module only when compiling tests, so the shipping artifact carries no test code. And `assert_eq!` you met in Chapter 9, along with the taxonomy that governs it: assertion failures are for *bugs*, and a failed test is a bug by definition.

```
# Figure 7: Running the unit tests

% cargo test
   Compiling alu_playground v0.1.0
    Finished `test` profile [unoptimized + debuginfo]
     Running unittests src/main.rs
--
running 4 tests
test predictor::tests::add_carries_into_bit_eight ... ok
test predictor::tests::and_masks_operands ... ok
test predictor::tests::mul_needs_the_full_result_bus ... ok
test predictor::tests::xor_finds_differing_bits ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

Read that last line the way it deserves to be read: *finished in 0.00s*. Four checks of the TinyALU's prediction logic, on a laptop, in less time than a simulator takes to print its banner. No license checked out, no design elaborated, no waveform dumped — because nothing here needed one. And when a test fails, the report is a diagnosis, not a stack trace. Suppose a future refactor forgets the width lesson and writes the addition as `(a + b) as u16`, wrapping at eight bits:

```
# Figure 8: A failing test names the culprit

% cargo test
--
running 4 tests
test predictor::tests::add_carries_into_bit_eight ... FAILED
test predictor::tests::and_masks_operands ... ok
test predictor::tests::mul_needs_the_full_result_bus ... ok
test predictor::tests::xor_finds_differing_bits ... ok

failures:
```

```
---- predictor::tests::add_carries_into_bit_eight stdout ----
assertion `left == right` failed
  left: 254
 right: 510

test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured
```

`254` is `0xFF + 0xFF` wrapped to eight bits; `510` is the truth. The bug that would have surfaced as a scoreboard miscompare forty minutes into a regression — with the *DUT* as the initial suspect — instead surfaced in milliseconds, correctly attributed to the predictor, before any simulator ran.

Savor what just changed, because it is a genuinely new capability, not a nicer version of an old one. Your testbench's pure logic — predictors, transaction arithmetic, coverage binning, anything that computes without touching a signal — now has its own test suite that runs on every build, for free. The Python stack kept all testbench verification inside the simulation; the testbench was only ever as tested as your last regression. From here on, this book runs `cargo test` habitually: when Part IV builds scoreboards and coverage collectors, their logic arrives with unit tests beside it, and the simulator's time is spent on the only thing that actually needs a simulator — the DUT.

Summary

In this chapter we gave Rust code a place to live. `mod` declares modules — inline for small things, `mod foo;` pointing at `foo.rs` for real ones — and the module tree is written in the program rather than discovered on a search path. `use` brings paths into scope the way `from ... import` did, glob imports and all, with preludes as the sanctioned exception. Everything is private until `pub` says otherwise, turning Python's underscore etiquette into a compiler-kept promise. Crates are the unit of compilation and distribution — Python's module/package/distribution muddle, resolved into one concept — with workspaces gathering related crates, as rustvm's own crates will be gathered. `Cargo.toml` declares dependencies, `cargo add` fetches them from crates.io, and `Cargo.lock` makes every build reproducible. And `cargo test` runs unit tests against pure testbench logic in milliseconds, no simulator required — a capability we will never stop using.

This chapter also closes Part I, so take a step back and look at what you now own. You came in knowing no Rust. You now hold the whole toolkit this book's testbenches are built from: ownership and borrowing, the responsibility-and-access model that replaces the garbage collector and turns data races into compile errors; structs, enums, and `match`, which gave `Ops` and four-state logic honest types; `Result` and `Option`, where exceptions used to be; traits, doing the work of inheritance and the dunder methods; generics, closures, and iterators, which will carry typed drivers and the factory's job; the smart pointers that make sharing explicit; and now modules, crates, and cargo to organize and test all of it. Every one of these landed on ground the Python book prepared, and every one was chosen because a coming chapter needs it. What you cannot yet do is *wait* — no Timer, no RisingEdge, no way to say "pause this task until the clock rises." That is Part II's business. Chapter 15 picks up exactly where the Python book's "Coroutines" chapter left off — same Rogue game, same event loop — and shows how `async`/`await` works when the language gives you the syntax but hands *you* the engine. The simulator is finally in sight.